

HORUS UNIVERSITY - EGYPT
Faculty of Engineering
Department of Communications and Electronics

ADVANCED SYSTEM FOR DRONE DETECTION, TRACKING, AND NEUTRALIZATION

SUBMITTED BY

Islam Emad Ahmed Ismail	1211108
Abdelaziz Eltohami Mohamed	1211105
Rafeek Marzouk El-Fauomy	1233007
Rawan Mohammed Elbialy	1211174
Sagy Shawky Abo El-Naga	1221602
Islam Hesham Hamed	1201040

SUPERVISED BY

Prof. Dr. Amr Hussein Hussein Abdullah
Assoc. Prof. Dr. Mervat Mohamed Hassan Elseddek
Assis. Prof. Dr. Rania Hamdy El-Abd

Academic Year 2025/2026

Acknowledgement

The architectural conceptualization and mathematical formulation of this system represents a rigorous synthesis of academic theory and applied engineering. This monograph serves as the foundational design document for our capstone research.

We owe a profound debt of gratitude to our supervisors, **Prof. Dr. Amr Hussein Hussein Abdullah**, **Assoc. Prof. Dr. Mervat Mohamed Hassan Elseddek**, and **Assis. Prof. Dr. Rania Hamdy El-Abd**. Your relentless pursuit of academic excellence, rigorous critique of our mathematical models, and unwavering support during the system design phase were the guiding forces behind this research.

We extend our sincere appreciation to the faculty teaching committee for their technical insights, which were invaluable in structuring our theoretical approach to embedded systems and deep learning deployment. Finally, to the members of the research team: this work is the manifestation of our collective persistence and multidisciplinary logic.

Contents

Abstract	5
List of Tables	6
List of Abbreviations	7
List of Figures	9
List of Codes	10
1 Introduction, Problem Formulation, and Theoretical Background	11
1.1 Introduction: The Proliferation of Asymmetric Aerial Threats	11
1.2 Problem Formulation: The Granularity Gap and Economic Asymmetry	12
1.2.1 The Granularity Gap and Thermal Submersion in Radar Systems	12
1.2.2 The Cost-Exchange Ratio (CER) Collapse	12
1.3 Electromagnetic Foundations: Antenna Theory and Characteristics	12
1.3.1 Radiation Pattern and Isotropic Radiators	12
1.3.2 Directivity (D) and Antenna Gain (G)	13
1.4 Spectrum Sensing and the Spectrum Analyzer Architecture	14
1.4.1 The Superheterodyne Spectrum Analyzer	14
1.4.2 Resolution Bandwidth (RBW) and Displayed Average Noise Level (DANL) .	14
1.5 Electronic Countermeasures: Jamming and Microwave Integrity	14
1.5.1 Mechanics of RF Jamming (The J/S Ratio)	15
1.5.2 Effect of Jamming on Microwave Receivers	15
1.5.3 Anti-Jamming (AJ) Technologies in Modern UAS	15
1.6 Proposed Cyber-Physical Architecture	16
1.7 Project Objectives	16
2 Problem Statement, Literature Review, and Analysis	18
2.1 Introduction	18
2.2 Problem Statement: The Physics of the Granularity Gap	18
2.2.1 Thermal Submersion and Swerling Target Models	18
2.2.2 Clutter Statistics and the Doppler Blind Zone	19
2.3 Literature Review: Spectrum Sensing in C-UAS	19
2.3.1 Superheterodyne Receiver Architecture	19
2.3.2 Mathematical Paradigms of Detection Algorithms	19
2.4 Electronic Countermeasures (ECM): Jamming Physics	20
2.4.1 The Burn-Through Range and J/S Ratio	20
2.4.2 Impact on Microwave Circuits: IP3 and IMD	20
2.5 Electronic Protection (EP): Anti-Jamming Architectures	20

2.5.1	Frequency Hopping Spread Spectrum (FHSS)	21
2.5.2	Direct Sequence Spread Spectrum (DSSS)	21
2.5.3	Spatial Filtering and Null Steering	21
2.6	Computer Vision and the Small Object Detection (SOD) Dilemma	22
2.6.1	The Evolution to Single-Stage Detectors (YOLOv8)	22
2.6.2	The Mathematics of Small Object Detection (SOD)	22
2.7	Analysis and Identification of the Research Gap	23
3	Proposed Cyber-Physical Architecture and Integration	24
3.1	Introduction: The Edge-Native Framework	24
3.2	Centralized Management and Inter-Process Communication	24
3.2.1	The Shared State Bus and Mutex Synchronization	24
3.2.2	Proactive Configuration Validation and Auto-Recovery	24
3.3	Acquisition Layer: Multimodal Fusion and Memory Architecture	25
3.3.1	Pre-Event Recording via Rolling Buffer Architecture	25
3.3.2	On-Screen Display (OSD) Telemetry Overlay	26
3.4	Network Engineering and Auto-Discovery Protocols	26
3.4.1	SADP Protocol and Parallel Port Scanning	26
3.4.2	Dual-NIC Routing Conflict Resolution	26
3.5	Control Theory and Actuation Kinematics	27
3.5.1	Shortest-Path Algorithm and Auto-Sweep	27
3.5.2	Discrete Kalman Filter (DKF) Smoothing	27
3.5.3	Stepper Motor Half-Step Sequencing	27
3.6	Command & Control (C2) and System Watchdog	28
3.6.1	Tactical Dashboard and WebSocket Streaming	28
3.6.2	Encrypted Telegram Alerts and Cooldown Matrix	28
3.6.3	The Thermal and Process Watchdog	29
4	System Implementation and Pre-Deployment Validation	30
4.1	Introduction	30
4.2	Hardware Realization: Component Analysis and Integration	30
4.2.1	Mechanical Infrastructure: Base and Axis Alignment	30
4.2.2	Passive RF Sensing Node: Early Warning Acquisition	31
4.2.3	Optical Perception and Edge Computing Core	32
4.2.4	Active Neutralization Payload: The Electronic "Weapon"	33
4.2.5	Complete System Integration: The Final Assembly	34
4.3	Software Architecture and Concurrent Engineering	34
4.3.1	Centralized State Management and Mutex Synchronization	35
4.3.2	Zero-Copy Video Acquisition and AI Perception	35
4.3.3	Closed-Loop Kinematics and Actuation	35
4.4	Command, Control (C2), and System Telemetry	35
4.4.1	Tactical Dashboard and WebSocket Integration	35
4.4.2	Remote Alerting and the Event Cooldown Matrix	36
4.5	Fault Tolerance and Autonomous Recovery	36

4.5.1	Pre-Flight Configuration Validation	36
4.5.2	Thermal Watchdog and Thread Auto-Recovery	36
4.6	Pre-Deployment Validation and Simulation Analytics	36
4.6.1	Latency Budget Profiling (τ_{sys})	37
4.6.2	Deep Learning Training Metrics and Classification	37
4.7	Chapter Summary	38
5	Future Roadmap and System Evolution: Towards Cognitive Air Defense	39
5.1	Introduction: The Paradigm Shift in Asymmetric Warfare	39
5.2	Hardware Evolution: Heterogeneous Computing and Multi-Spectral Fusion	39
5.2.1	Multi-Spectral Perception (EO/LWIR Fusion)	39
5.2.2	Heterogeneous Edge SoC (FPGA + NPU Architecture)	40
5.3	Cognitive Electronic Warfare (CEW) and 6G Integration	40
5.3.1	Adaptive Spatial Beamforming and Null Steering	40
5.3.2	Machine Learning in Spectrum Warfare	41
5.4	Autonomous Swarm Defense and Multi-Object Tracking	41
5.4.1	The Unscented Kalman Filter (UKF)	41
5.4.2	Joint Probabilistic Data Association (JPDA)	42
5.5	Final Conclusion	42
A	Software Repository and Source Code	43
A.1	Core Execution and Orchestration	43
A.2	Perception and Computer Vision Subsystem	50
A.3	Kinematic Actuation and Control Theory	54
A.4	RF Sensing and Network Operations	59
A.5	Command & Control (C2) and Data Logging	66
A.6	System Health and Fault Tolerance	90
A.7	System Configuration and OS Deployment	103
A.8	Perception and Computer Vision Subsystem	105
A.9	Kinematic Actuation and Control Theory	108
A.10	RF Sensing and Network Operations	110
A.11	System Health and Fault Tolerance	111
A.12	System Configuration and OS Deployment	112
A.13	Command & Control (C2) and Data Logging	112
A.14	System Configuration and OS Deployment	114

Abstract

The democratization of aerial maneuverability has exposed a critical vulnerability in modern air defense known as the "Granularity Gap." Conventional Air Surveillance Radars, optimized for high-velocity, large-RCS ballistic targets, are mathematically constrained by the inverse fourth-power attenuation law, rendering them blind to micro-drones ($\sigma < 0.01 \text{ m}^2$) operating within dense urban ground clutter.

This monograph establishes the theoretical architecture and mathematical framework for an autonomous, edge-native Cyber-Physical System designed for drone detection, tracking, and theoretical neutralization. The proposed topology abandons monolithic single-sensor dependency in favor of a multimodal approach. The system incorporates an Omnidirectional Antenna and a 6G-ready Spectrum Analyzer to detect ambient control link anomalies. Upon cueing, the perception layer utilizes an INT8-quantized YOLOv8 convolutional neural network, hosted on a Raspberry Pi 5 computing node, to process visual streams from a high-gain PTZ optical sensor.

To resolve the non-linear kinematics of highly maneuverable targets and compensate for mechanical actuation latency, the architecture mathematically derives and implements an Extended Kalman Filter (EKF). The system's hardware infrastructure encompasses a Display Screen for tactical Human-Machine Interfacing, a high-density LiFePO₄ Battery for autonomous power distribution, and theoretically maps the integration of a Directional Antenna coupled with a Gallium Nitride (GaN) Power Amplifier for active RF jamming. This comprehensive framework offers a rigorous blueprint for neutralizing asymmetric aerial threats using accessible Commercial Off-The-Shelf (COTS) components, establishing the protocols for upcoming empirical field validations.

List of Tables

1.1	Comparison between Legacy Aerial Threats and Class I Asymmetric Threats (Micro-Drones).	11
2.1	Comparative Analysis of Single-Sensor Modalities against Class I UAS.	23
3.1	Core System Processes and Concurrency Allocation Matrix.	25
3.2	Comparison of Stepper Motor Excitation Modes.	28
3.3	Event Notification Cooldown Matrix to Prevent Network Saturation.	28
4.1	Hardware-in-the-Loop (HIL) Latency Budget Profiling Breakdown.	37
5.1	Comparison of Target Tracking Filter Algorithms for High-G UAS Maneuvers. . . .	42

List of Abbreviations

Acronym	Description
C-UAS	Counter-Unmanned Aerial Systems
CER	Cost-Exchange Ratio
CIP	Critical Infrastructure Protection
CNN	Convolutional Neural Network
CONOPS	Concept of Operations
COTS	Commercial Off-The-Shelf
EKF	Extended Kalman Filter
EIRP	Effective Isotropic Radiated Power
GaN	Gallium Nitride
HMI	Human-Machine Interface
HUD	Heads-Up Display
KPI	Key Performance Indicator
PA	Power Amplifier
PTZ	Pan-Tilt-Zoom
RCS	Radar Cross Section
YOLO	You Only Look Once

List of Figures

1.1	2D Cartesian projection of a directional antenna radiation pattern, illustrating the Main Lobe, Side Lobes, Back Lobe, and the Half-Power Beamwidth (HPBW).	13
1.2	Block Diagram of a Superheterodyne Spectrum Analyzer. The incoming f_{RF} is mixed with a swept f_{LO} to produce a constant f_{IF} ($f_{IF} = f_{RF} - f_{LO} $), allowing the RBW filter to isolate specific spectral components.	14
1.3	Time-Frequency grid illustrating Frequency Hopping Spread Spectrum (FHSS) operating against a Partial-Band Barrage Jammer. The drone rapidly changes its carrier frequency based on a pseudo-random code ($f_k = f_0 + c_k \Delta f$).	16
2.1	Detailed block diagram of a Superheterodyne Spectrum Analyzer front-end. The Resolution Bandwidth (RBW) filter determines the noise floor according to the equation: $DANL = -174 + 10 \log_{10}(RBW) + NF$	19
2.2	Frequency spectrum illustrating Third-Order Intermodulation Distortion (IMD). The high-power jamming signals (f_1, f_2) force the non-linear LNA to generate IP3 products that fall directly into the receiver's IF bandpass, irrecoverably corrupting the data link.	21
2.3	Adaptive Beamforming and Null Steering. The UAS phased array dynamically adjusts element weights to maximize gain toward the friendly Ground Control Station (GCS) while forcing an antenna null ($Gain \approx -\infty$ dB) exactly on the Angle of Arrival of the Jamming signal.	22
2.4	The Structural Degradation in Small Object Detection (SOD). Deep CNN pooling layers expand the Receptive Field but catastrophically compress the spatial dimensions. A drone initially occupying 16×16 pixels is reduced to less than 1 pixel deep in the network, destroying feature extractability.	23
3.1	The Circular Rolling Buffer Architecture. The system continuously writes high-resolution video frames to the RAM buffer. Upon an AI detection trigger, the system commits the trailing 30 seconds of frames to persistent storage, preserving critical pre-engagement footage while saving disk space.	25
3.2	Sequence Diagram of the Network Auto-Discovery Module. The edge node utilizes a UDP broadcast to locate the PTZ hardware, followed by a targeted TCP scan to verify the availability of the RTSP video stream, automatically updating the system's YAML configuration.	26
3.3	Closed-Loop Kinematic Block Diagram. The noisy AI observation is smoothed by the Kalman Filter. The spatial error $e(t)$ between the predicted target and the optical center drives the PID controller to output real-time velocity commands [70, 71]. . . .	27
4.1	The Mechanical Skeleton. The fluid-damped tripod provides absolute terrestrial stability, while the L-Bracket acts as the structural spine, ensuring perfect spatial alignment between the optical and RF axes.	31

4.2	The Passive RF Acquisition Node. The omnidirectional antenna intercepts ambient drone telemetry, which is mathematically parsed by the SA6 sweeping receiver to identify FHSS/DSSS signatures.	31
4.3	The Perception and Processing Array. The Hikvision camera feeds high-speed RTSP video via Ethernet to the Raspberry Pi 5, which executes YOLOv8 INT8 inference and issues closed-loop kinematic commands.	32
4.4	The Tactical Edge LCD. Connected directly to the Pi 5, it renders the military-styled web dashboard, providing real-time telemetry and optical verification to the local operator.	33
4.5	The ECM Neutralization Chain. The HT004A boosts the synthesized jamming signal by +30 dB, which is then violently focused into a tight spatial beam by the Alfa directional antenna to overwhelm the drone's receiver.	33
4.6	The fully assembled Cyber-Physical System. The image illustrates the successful electro-mechanical integration of the Kingjoy base, the structural L-Bracket, the Hikvision optical node, and the active RF payload, forming a unified, autonomous defense platform.	34
4.7	Stacked bar chart detailing the deterministic execution blocks derived from laboratory HIL profiling. The total computational loop latency remains strictly below the 200 ms mechanical instability threshold.	37
5.1	Proposed Heterogeneous Edge Architecture. The FPGA fabric handles deterministic, cycle-accurate pixel fusion and ISP operations, routing the unified tensor directly to the NPU. The CPU is completely offloaded, dedicated exclusively to stochastic filtering and motor kinematics.	40
5.2	Cognitive MVDR Beamforming. The Massive MIMO array dynamically adjusts complex element weights ($\mathbf{w}_{opt}$) to focus the jamming energy exclusively on the hostile target while forcing radiation nulls toward friendly infrastructure, preventing collateral RF damage.	41

List of Codes

A.1	Central Execution Engine (core/main.py)	43
A.2	Optical Tracker and DKF (core/camera_tracker.py)	50
A.3	Kinematic PID and ISAPI Actuation (core/motor_controller.py)	54
A.4	Network Auto-Discovery (utils/network_scanner.py)	59
A.5	Tactical HMI Server (core/dashboard.py)	66
A.6	Event Cooldown Matrix (utils/alert_manager.py)	81
A.7	Multi-Format Database Logger (utils/data_logger.py)	85
A.8	Thermal and Hardware Monitor (utils/system_health.py)	90
A.9	Configuration Validation (utils/config_validator.py)	94
A.10	System Configuration Matrix (config/config.yaml)	103
A.11	Deep Learning Inference Engine (core/yolo_detector.py)	105
A.12	Zero-Copy Video Acquisition (core/stream_recorder.py)	106
A.13	Sensor Triangulation Mathematics (utils/triangulation.py)	108
A.14	Spectrum Analyzer Interfacing (core/rf_monitor.py)	110
A.15	Thread Watchdog Recovery (utils/watchdog_recovery.py)	111
A.16	Command Line Interface (core/cli_interface.py)	112
A.17	Remote Telegram Alerting (core/telegram_bot.py)	113
A.18	Systemd Daemonization Service (c_uas.service)	114
A.19	Python Dependencies (requirements.txt)	115

Introduction, Problem

1 Formulation, and Theoretical Background

1.1 Introduction: The Proliferation of Asymmetric Aerial Threats

Historically, the doctrine of airspace dominance and Critical Infrastructure Protection (CIP) was predicated on a fundamental assumption: the operational sky was the exclusive domain of state actors operating highly visible, supersonic, and technologically complex platforms. Consequently, global air defense grids were engineered as colossal sentinels designed to detect large Radar Cross Section (RCS) targets, such as intercontinental ballistic missiles, strategic bombers, and conventional tactical aircraft [1, 3].

However, the rapid convergence and commercialization of micro-electromechanical systems (MEMS), high-density Lithium-Polymer (Li-Po) energy storage, and advanced brushless DC (BLDC) motor efficiency have catalyzed a profound paradigm shift. The third decade of the 21st century marks the democratization of aerial maneuverability. The mass proliferation of Class I Unmanned Aerial Systems (UAS)—commonly referred to as micro-drones or loitering munitions—has introduced a devastatingly effective modality of asymmetric security threats [23, 29].

These Commercial Off-The-Shelf (COTS) devices, originally engineered for civilian topography and photography, have been rapidly weaponized. Their defining physical and kinematic characteristics—extremely low procurement cost, hyper-agility exceeding $4g$ instantaneous acceleration, and a minimal electromagnetic footprint ($\sigma < 0.01 m^2$)—render them practically invisible to legacy early-warning radar systems, necessitating a paradigm shift towards multimodal sensing and Edge Artificial Intelligence (AI) [28, 25].

Table 1.1: Comparison between Legacy Aerial Threats and Class I Asymmetric Threats (Micro-Drones).

Characteristic	Legacy Threats (Fighters/Missiles)	Class I UAS (Micro-Drones)
Radar Cross Section (RCS)	Large ($> 1.0 m^2$)	Extremely Small ($< 0.01 m^2$)
Kinematics	Supersonic, High Altitude	Low-Slow-Small (LSS), Hovering
Procurement Cost	\$10M – \$100M+	\$500 – \$2,000 (COTS)
Electromagnetic Signature	High-power military radar/comms	Weak ISM band telemetry (2.4/5.8 GHz)
Detection Vulnerability	Conventional Early-Warning Radar	Multi-modal (RF Sensing + Edge AI)

1.2 Problem Formulation: The Granularity Gap and Economic Asymmetry

The core vulnerabilities exposed by Class I UAS represent a catastrophic intersection of electromagnetic physics constraints and economic asymmetry.

1.2.1 The Granularity Gap and Thermal Submersion in Radar Systems

Legacy surveillance radars rely on the standard Radar Range Equation, where the maximum detection range R_{max} is governed by the fourth root of the target's RCS (σ):

$$R_{max} = \sqrt[4]{\frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 P_{min} L_{sys}}} \quad (1.1)$$

For a micro-drone target ($\sigma < 0.01 \text{ m}^2$), the reflected power P_r drops precipitously below the receiver's thermodynamic noise floor (kTB) [2, 7]. The signal is entirely submerged within the ground clutter reflectivity (σ_c^0) [10, 12]. Furthermore, micro-drones executing hovering maneuvers possess a radial velocity $v_r \approx 0$, causing them to fall precisely into the radar's Moving Target Indicator (MTI) Zero-Velocity Notch Filter, essentially blinding the radar network [14].

1.2.2 The Cost-Exchange Ratio (CER) Collapse

The secondary vulnerability is fundamentally economic. Defense economics governing traditional Surface-to-Air Missiles (SAM) operate on a highly unfavorable Cost-Exchange Ratio (CER) when deployed against autonomous micro-drones [33].

$$CER = \frac{C_{intercept}}{C_{target}} \approx \frac{\$3,000,000}{\$1,500} = 2000 : 1 \quad (1.2)$$

Intercepting low-cost swarm assets with multi-million-dollar kinetic interceptors dictates an unsustainable attrition warfare model. Defense mechanisms must therefore leverage directed electromagnetic energy (RF Jamming) and localized Edge AI to bring the cost of interception down to the cost of standard electricity.

1.3 Electromagnetic Foundations: Antenna Theory and Characteristics

To effectively detect RF telemetry and subsequently neutralize the target via jamming, a rigorous understanding of antenna electromagnetics is imperative. An antenna acts as a reciprocal transducer matching the impedance of free space ($Z_0 \approx 377 \Omega$) to the transmission line [6].

1.3.1 Radiation Pattern and Isotropic Radiators

The radiation pattern is a mathematical function or graphical representation of the radiation properties of the antenna as a function of space coordinates (spherical coordinates θ, ϕ). A theoretical *Isotropic*

Radiator radiates power uniformly in all directions (4π steradians). Real antennas, however, focus energy into specific lobes.

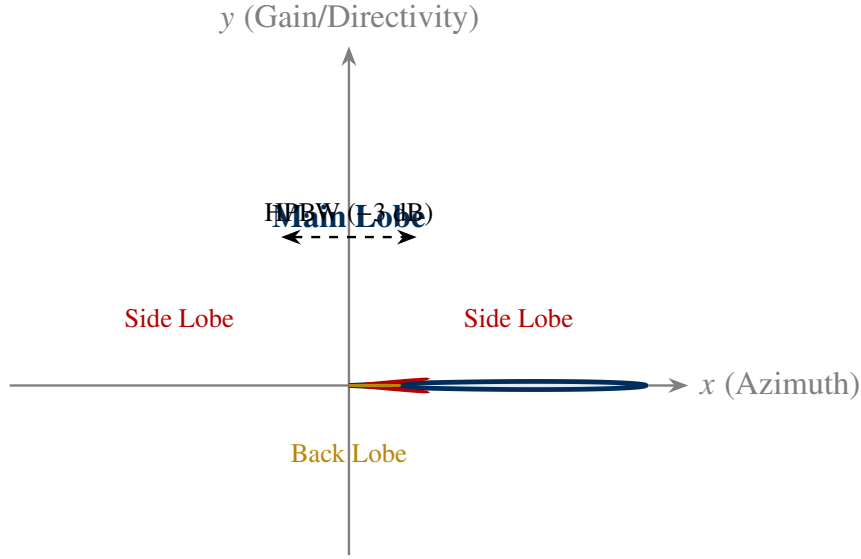


Figure 1.1: 2D Cartesian projection of a directional antenna radiation pattern, illustrating the Main Lobe, Side Lobes, Back Lobe, and the Half-Power Beamwidth (HPBW).

1.3.2 Directivity (D) and Antenna Gain (G)

Directivity (D) is defined as the ratio of the radiation intensity in a given direction to the radiation intensity averaged over all directions [8]. Let $U(\theta, \phi)$ be the radiation intensity (W/sr) and P_{rad} be the total radiated power:

$$D(\theta, \phi) = \frac{U(\theta, \phi)}{U_{avg}} = \frac{4\pi U(\theta, \phi)}{P_{rad}} \quad (1.3)$$

The maximum directivity D_{max} usually occurs at the center of the main lobe.

Antenna Gain (G) is a more comprehensive metric than directivity as it accounts for the ohmic and dielectric losses within the antenna structure. If η_{rad} is the radiation efficiency coefficient ($0 \leq \eta_{rad} \leq 1$), the Gain is mathematically expressed as:

$$G(\theta, \phi) = \eta_{rad} \cdot D(\theta, \phi) \quad (1.4)$$

In the context of the proposed system:

- **Omnidirectional Antennas (Low Gain, 360° coverage):** Utilized in the spectrum sensing phase to detect drone telemetry from any unknown azimuth.
- **Directional Antennas (High Gain, Narrow Beamwidth):** Utilized during the jamming phase to focus maximum Effective Isotropic Radiated Power (EIRP) strictly towards the targeted drone, minimizing collateral interference.

1.4 Spectrum Sensing and the Spectrum Analyzer Architecture

To overcome the radar blind spot, passive RF Spectrum Sensing is employed. Drones continuously emit RF energy for telemetry, video downlinking (e.g., FPV), and C2 (Command and Control) synchronization, primarily within the 2.4 GHz and 5.8 GHz ISM bands [35, 40].

1.4.1 The Superheterodyne Spectrum Analyzer

The core device for capturing these signals is the Spectrum Analyzer (SA). Modern high-frequency SAs (such as the HTOOL-SA-6G) operate on the **Superheterodyne** principle, which translates incoming high-frequency RF signals down to a lower, fixed Intermediate Frequency (IF) for precise filtering and analysis.

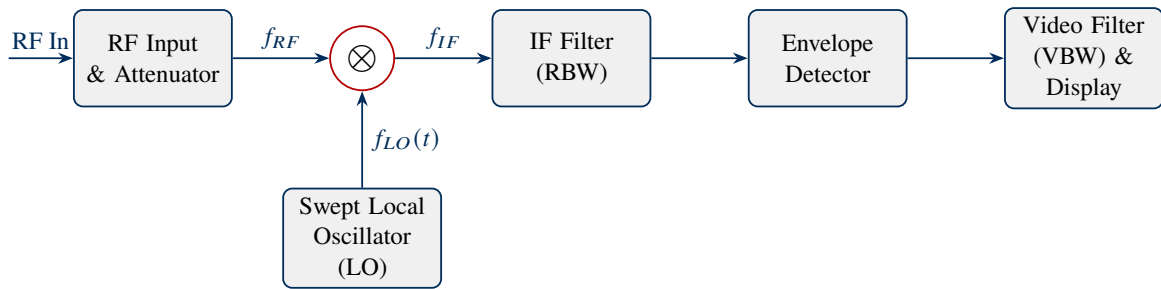


Figure 1.2: Block Diagram of a Superheterodyne Spectrum Analyzer. The incoming f_{RF} is mixed with a swept f_{LO} to produce a constant f_{IF} ($f_{IF} = |f_{RF} - f_{LO}|$), allowing the RBW filter to isolate specific spectral components.

1.4.2 Resolution Bandwidth (RBW) and Displayed Average Noise Level (DANL)

The ability of the system to detect extremely weak drone signals at long ranges is governed by the **Resolution Bandwidth (RBW)** filter. The RBW determines the frequency resolution of the analyzer. A narrower RBW separates closely spaced signals and drastically lowers the noise floor, revealing hidden telemetry spikes.

The absolute sensitivity of the analyzer is mathematically defined by the Displayed Average Noise Level (DANL):

$$DANL \text{ (dBm)} = -174 \text{ dBm/Hz} + 10 \log_{10}(RBW) + NF \quad (1.5)$$

Where -174 dBm/Hz is the thermal noise power at room temperature (kT_0), and NF is the Noise Figure of the internal circuitry [74]. By decreasing the RBW by a factor of 10, the noise floor drops by 10 dB, significantly extending the drone detection horizon.

1.5 Electronic Countermeasures: Jamming and Microwave Integrity

Once a UAS is localized and verified, active neutralization is executed via Electronic Countermeasures (ECM), specifically RF Jamming. The objective is to sever the link between the Ground Control Station (GCS) and the drone's microwave receiver [75].

1.5.1 Mechanics of RF Jamming (The J/S Ratio)

Jamming is achieved by transmitting a high-power noise or structured signal that overwhelms the legitimate C2 signal. The effectiveness is dictated by the **Jamming-to-Signal (J/S) Ratio**. Using the Friis transmission equations, the ratio at the drone's receiver is derived as:

$$\frac{J}{S} = \frac{P_j G_j}{P_s G_s} \left(\frac{R_s}{R_j} \right)^2 \cdot L_p \geq \gamma_{th} \quad (1.6)$$

Where P is power, G is antenna gain, R_s is the distance from the drone to its pilot, R_j is the distance from the jammer to the drone, L_p represents polarization mismatch losses, and γ_{th} is the minimum "Burn-Through" threshold (typically 15 – 20 dB) [78].

1.5.2 Effect of Jamming on Microwave Receivers

When a high-power jamming signal hits the drone's microwave receiver front-end, it directly attacks the **Low Noise Amplifier (LNA)**.

- **Desensitization and Saturation:** Microwave LNAs have a linear operational region bounded by the 1 dB Compression Point (P_{1dB}). When the high-amplitude jamming wave enters the LNA, it drives the transistor into its non-linear saturation region. The gain of the LNA drops precipitously, rendering it deaf to the weak, legitimate control signal.
- **Intermodulation Distortion (IMD):** High-power barrage jamming creates harmonic frequencies. If multiple signals mix within the non-linear LNA, they produce Third-Order Intermodulation Products (IP3) that fall directly into the IF bandpass, completely corrupting the demodulator's bit-error rate (BER) [74].

1.5.3 Anti-Jamming (AJ) Technologies in Modern UAS

To maintain academic rigor, the defense system must account for drones equipped with Electronic Protection (EP) or Anti-Jamming (AJ) capabilities. Modern military and high-end commercial drones employ **Frequency Hopping Spread Spectrum (FHSS)** and **Direct Sequence Spread Spectrum (DSSS)** [75].

In DSSS systems, the narrow-band information is multiplied by a high-speed pseudo-noise (PN) code, spreading the signal over a wide bandwidth. The resilience of the drone to jamming is quantified by the **Processing Gain (G_p)**:

$$G_p (dB) = 10 \log_{10} \left(\frac{B_{spread}}{B_{info}} \right) \quad (1.7)$$

To overcome advanced FHSS/DSSS systems, the proposed architecture advocates for **Follower/Spot Jamming** (dynamically shifting the jamming energy to follow the hopping sequence) or **Adaptive Spatial Beamforming** to physically focus energy rather than spreading it across the spectrum [78].

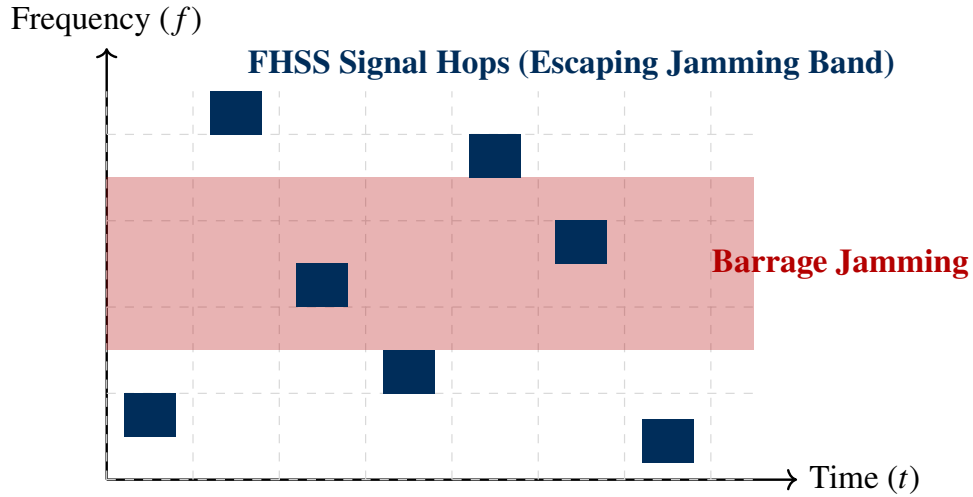


Figure 1.3: Time-Frequency grid illustrating Frequency Hopping Spread Spectrum (FHSS) operating against a Partial-Band Barrage Jammer. The drone rapidly changes its carrier frequency based on a pseudo-random code ($f_k = f_0 + c_k \Delta f$).

1.6 Proposed Cyber-Physical Architecture

To address the aforementioned constraints, this project proposes the design and implementation of an advanced, edge-native Cyber-Physical System (CPS). The core topology abandons monolithic radar dependency in favor of multimodal sensor fusion:

1. **Spectrum Sensing Layer:** Utilizing the HTOOL-SA-6G spectrum analyzer and omnidirectional antennas to monitor the RF noise floor and isolate FHSS/DSSS drone uplinks based on specific frequency signatures [40].
2. **Optical AI Verification:** Upon RF cueing, a high-gain Pan-Tilt-Zoom (PTZ) camera visually acquires the target. An INT8-quantized YOLOv8 architecture [53, 63], running on a Raspberry Pi 5 [84], performs real-time classification to reject biological clutter (birds).
3. **Kinematic Filtering:** An Extended Kalman Filter (EKF) [65] mathematically sustains the target's trajectory vector during occlusions, feeding predictive coordinates to the PTZ motor drivers.
4. **Directional Neutralization:** Theoretical application of High-Gain directional antennas and GaN Power Amplifiers [79] to execute targeted J/S overrides on the drone's LNA.

1.7 Project Objectives

The primary objective is to bridge the "Granularity Gap" in modern air defense. To transition this theoretical framework into a rigorously tested prototype, the system is strictly bound by the following Key Performance Indicators (KPIs):

- **Spectrum Analysis Proficiency:** Accurately configure RBW and VBW filters to lower the DANL, ensuring the detection of Class I drones at ranges $\geq 1.0 \text{ km}$ despite urban RF clutter.

-
- **System Loop Latency Constraint (τ_{sys}):** Ensure the total system latency—from optical acquisition to motor actuation—remains $\leq 200\text{ ms}$ to prevent phase instability in the PID tracking loop [70].
 - **Electromagnetic Optimization:** Mathematically and physically select antennas that maximize Directivity (D) and Gain (G) for the jamming array, validating the capability to induce LNA saturation on commercial microwave receivers.
 - **AI Execution:** Successfully interface high-speed RTSP video streams within a multiprocessing, zero-copy DMA architecture on the Raspberry Pi 5, maintaining $\geq 30\text{ FPS}$ inference without inducing SoC thermal throttling.

2

Problem Statement, Literature Review, and Analysis

2.1 Introduction

The democratization of Unmanned Aerial Systems (UAS) has precipitated a structural crisis in modern air defense doctrines [28]. As the academic literature demonstrates, the transition from monolithic conventional radar systems to decentralized, multimodal Cyber-Physical Systems is an operational necessity [30]. This chapter rigorously defines the physical problem statement underlying the "Granularity Gap," systematically reviews the state-of-the-art across sensing modalities, and critically analyzes the electromagnetic warfare paradigms—specifically RF Spectrum Sensing, Jamming, and Anti-Jamming techniques—that govern the modern Counter-UAS (C-UAS) operational environment.

2.2 Problem Statement: The Physics of the Granularity Gap

The inability of conventional Air Surveillance Radars (ASR) to detect Class I micro-drones is rooted in immutable electromagnetic limitations and statistical detection theories [1, 3].

2.2.1 Thermal Submersion and Swerling Target Models

The fundamental limit of a monostatic radar is dictated by the Radar Range Equation. The maximum theoretical detection range R_{max} is governed by the fourth root of the target's Radar Cross Section (RCS), denoted as σ :

$$R_{max} = \sqrt[4]{\frac{P_t G^2 \lambda^2 \sigma}{(4\pi)^3 (k T_0 B F) L_{sys}}} \quad (2.1)$$

Micro-drones exhibit an RCS of $\sigma < 0.01 \text{ m}^2$. Furthermore, due to the complex geometry of their multi-rotor frames, the RCS is highly aspect-dependent and rapidly fluctuates [7]. This fluctuation is mathematically modeled using the **Swerling Case 1 and 3** models, which describe targets consisting of many independent scatterers. The probability density function (PDF) of the RCS for a Swerling 1 target follows a Rayleigh distribution:

$$p(\sigma) = \frac{1}{\sigma_{avg}} \exp\left(-\frac{\sigma}{\sigma_{avg}}\right), \quad \sigma \geq 0 \quad (2.2)$$

This exponential fluctuation dictates that maintaining a constant Probability of Detection (P_d) requires a disproportionately massive increase in the Signal-to-Noise Ratio (SNR), pushing the required transmit power (P_t) beyond practical embedded limits [2, 9].

2.2.2 Clutter Statistics and the Doppler Blind Zone

In urban topography, the weak target return is actively submerged within thermodynamic ground clutter. The clutter reflectivity (σ_c^0) often obeys a Weibull or K-distribution rather than Gaussian, resulting in "heavy tails" that cause severe False Alarm Rates (FAR) [10, 13].

To mitigate clutter, Pulse-Doppler radars employ Moving Target Indicator (MTI) algorithms, filtering returns based on their Doppler frequency shift ($f_d = \frac{2v_r}{\lambda}$). To eliminate stationary buildings, signal processors apply a "Zero-Velocity Notch Filter". When a micro-drone executes a hover maneuver ($v_r \approx 0$), it falls precisely into this notch. The MTI filter's frequency response $H(f)$ near zero velocity approaches zero, forcing the defense grid to actively discard the drone's signature as static background clutter [14, 15].

2.3 Literature Review: Spectrum Sensing in C-UAS

To circumvent the inherent radar blind spot, modern research has aggressively pivoted towards passive Radio Frequency (RF) Spectrum Sensing [36, 37]. Micro-drones continuously emit RF energy for telemetry and video downlinks within the 2.4 GHz and 5.8 GHz Industrial, Scientific, and Medical (ISM) bands.

2.3.1 Superheterodyne Receiver Architecture

The core hardware enabling high-fidelity RF analysis is the Superheterodyne Spectrum Analyzer. This architecture translates incoming high-frequency RF signals down to a lower, fixed Intermediate Frequency (IF) for precise filtering.

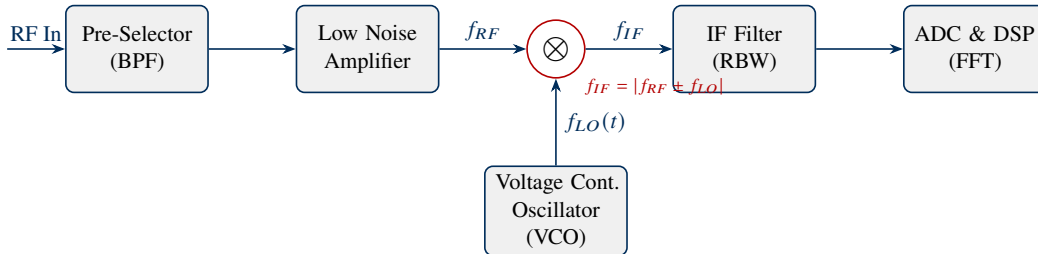


Figure 2.1: Detailed block diagram of a Superheterodyne Spectrum Analyzer front-end. The Resolution Bandwidth (RBW) filter determines the noise floor according to the equation: $DANL = -174 + 10 \log_{10}(RBW) + NF$.

2.3.2 Mathematical Paradigms of Detection Algorithms

Spectrum sensing algorithms implemented within Digital Signal Processors (DSP) generally fall into three distinct mathematical paradigms [40, 35]:

1. **Energy Detection (ED):** The detector measures the energy of the received signal over an observation time T and compares it to a noise threshold (λ_{th}). Let $y(t) = s(t) + n(t)$. The test statistic is:

$$T_{ED} = \frac{1}{N} \sum_{n=1}^N |y(n)|^2 \underset{H_0}{\overset{H_1}{\gtrless}} \lambda_{th} \quad (2.3)$$

While computationally efficient, ED is highly susceptible to Noise Uncertainty and cannot differentiate between a drone signal and generic Wi-Fi interference.

2. **Matched Filter Detection:** If the specific drone protocol (e.g., DJI OcuSync) is known a priori, a matched filter maximizes the SNR. It requires perfect synchronization and demodulation, making it impractical for unknown or proprietary UAS.
3. **Cyclostationary Feature Extraction:** Drone telemetry protocols exhibit inherent periodicities (cyclic frequencies) due to carrier waves and pulse trains. The Spectral Correlation Density (SCD) function is utilized to detect these features even when the signal is deeply buried below the noise floor ($SNR < 0 \text{ dB}$). The SCD is the Fourier transform of the cyclic autocorrelation function $R_y^\alpha(\tau)$:

$$S_y^\alpha(f) = \int_{-\infty}^{\infty} R_y^\alpha(\tau) e^{-j2\pi f\tau} d\tau \quad (2.4)$$

Where α is the cyclic frequency. This method strongly rejects stationary AWGN, isolating the artificial drone signatures.

2.4 Electronic Countermeasures (ECM): Jamming Physics

Upon positive detection, active neutralization is executed via ECM. Jamming is fundamentally an electromagnetic energy supremacy contest at the target's receiver front-end, aiming to sever the drone from its Ground Control Station (GCS) [74].

2.4.1 The Burn-Through Range and J/S Ratio

The effectiveness of jamming is dictated by the **Jamming-to-Signal (J/S) Ratio** [75]. The "Burn-Through Range" is the specific distance at which the legitimate control signal power finally overcomes the jamming noise. By rearranging the Friis transmission equations, the J/S ratio at the drone's receiver is derived as:

$$\frac{J}{S} = \frac{EIRP_{Jammer}}{EIRP_{GCS}} \left(\frac{R_{GCS}}{R_{Jammer}} \right)^2 \cdot L_{pol} \geq \gamma_{th} \quad (2.5)$$

Where γ_{th} is the minimum threshold (typically 15 – 20 dB for standard FSK links).

2.4.2 Impact on Microwave Circuits: IP3 and IMD

A drone's microwave receiver relies on a Low Noise Amplifier (LNA). The jamming array's objective is to physically drive this LNA into non-linear saturation [79].

Microwave amplifiers exhibit linear gain only up to the 1 dB Compression Point (P_{1dB}). When immense RF energy hits the receiver, the LNA saturates (Desensitization), rendering the drone deaf. Furthermore, high-power barrage jamming induces **Intermodulation Distortion (IMD)**. If multiple signals mix within the saturated LNA, they produce Third-Order Intermodulation Products (IP3).

2.5 Electronic Protection (EP): Anti-Jamming Architectures

A robust C-UAS platform must account for advanced UAS equipped with Anti-Jamming (AJ) capabilities. The literature identifies Spread Spectrum and Spatial Filtering as the primary defense mechanisms

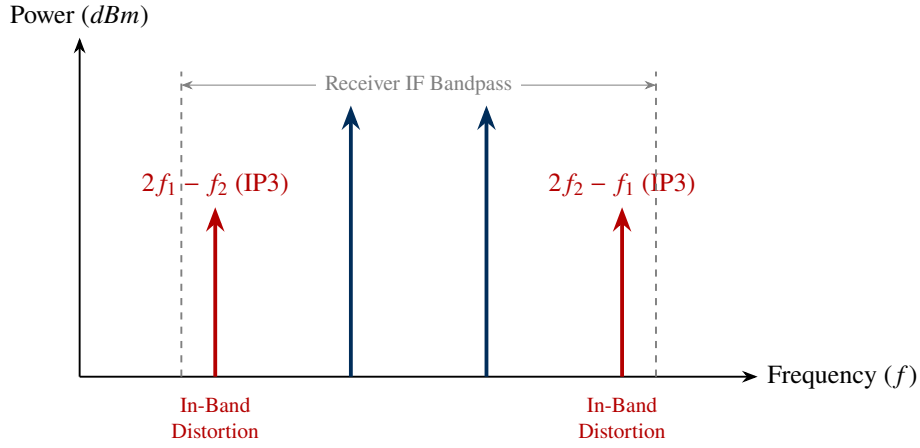


Figure 2.2: Frequency spectrum illustrating Third-Order Intermodulation Distortion (IMD). The high-power jamming signals (f_1, f_2) force the non-linear LNA to generate IP3 products that fall directly into the receiver's IF bandpass, irrecoverably corrupting the data link.

[78, 75].

2.5.1 Frequency Hopping Spread Spectrum (FHSS)

In FHSS, the drone rapidly switches its carrier frequency across the available spectrum based on a pseudo-random code. Against FHSS, a narrow-band "Spot Jammer" fails entirely. The defense system must employ either **Wideband Barrage Jamming** (spreading noise across the entire hopping bandwidth, which drastically dilutes the power spectral density) or a **Follower Jammer** (a Digital RF Memory - DRFM system that detects the hop and shifts the jamming frequency within microseconds).

2.5.2 Direct Sequence Spread Spectrum (DSSS)

DSSS protects the drone by multiplying the narrowband control signal by a high-speed pseudo-noise (PN) code sequence, spreading the signal's energy across a massive bandwidth.

The resilience of a DSSS system against jamming is mathematically quantified by its **Processing Gain** (G_p):

$$G_p = 10 \log_{10} \left(\frac{B_{spread}}{B_{info}} \right) \quad (2.6)$$

If a drone possesses a Processing Gain of 30 dB, the jamming array must output 1000 times more effective power just to overcome the mathematical advantage of the DSSS despreading correlator.

2.5.3 Spatial Filtering and Null Steering

Advanced military-grade UAS utilize adaptive Phased Array antennas [47]. Using algorithms like Minimum Variance Distortionless Response (MVDR), the drone calculates complex weights to steer the main beam toward the legitimate controller while placing a "mathematical null" directly in the direction of the Jammer.

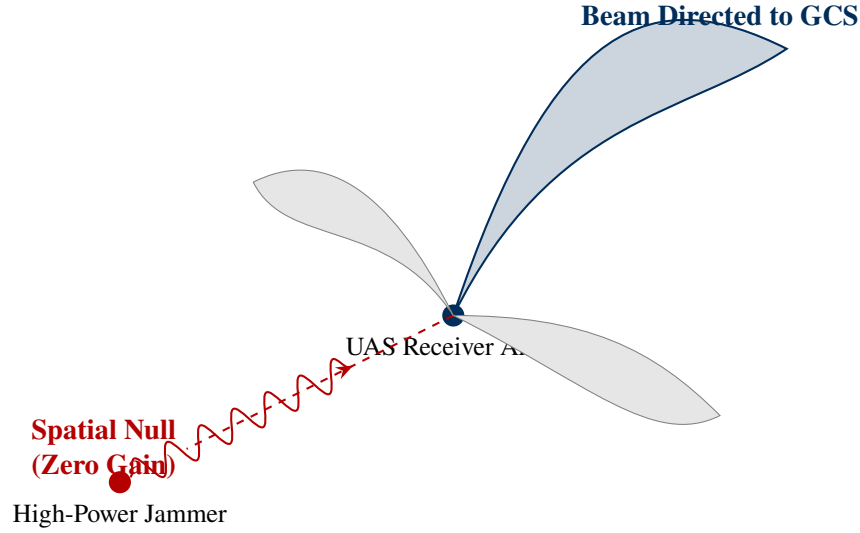


Figure 2.3: Adaptive Beamforming and Null Steering. The UAS phased array dynamically adjusts element weights to maximize gain toward the friendly Ground Control Station (GCS) while forcing an antenna null (Gain $\approx -\infty$ dB) exactly on the Angle of Arrival of the Jamming signal.

2.6 Computer Vision and the Small Object Detection (SOD) Dilemma

Because autonomous "Dark Drones" maintain strict RF silence (GNSS waypoint navigation) [41], passive Electro-Optical (EO) tracking remains the ultimate verification and targeting modality [49].

2.6.1 The Evolution to Single-Stage Detectors (YOLOv8)

Classical vision approaches (HOG+SVM) fail in dynamic urban backgrounds. Modern detection relies on Deep Convolutional Neural Networks (CNNs). The YOLO (You Only Look Once) architecture [50] frames object detection as a single regression problem, achieving the hard real-time inference (≥ 30 FPS) required for continuous kinetic tracking. The YOLOv8 architecture replaces standard convolutions with a Cross Stage Partial Network (CSPNet) and utilizes a Path Aggregation Network (PANet) for multi-scale feature fusion [53].

2.6.2 The Mathematics of Small Object Detection (SOD)

Despite its speed, CNNs inherently struggle with the Small Object Detection (SOD) problem. When an image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$ is passed through the network's backbone, max-pooling layers progressively halve the spatial dimensions to increase the Receptive Field.

As mathematically illustrated in Figure 2.4, a micro-drone at a distance becomes a sub-pixel fraction deep in the network, causing a catastrophic drop in Recall. Resolving the SOD problem necessitates active mechanical intervention (PTZ Magnification) to normalize the target's bounding box size prior to tensor processing [55].

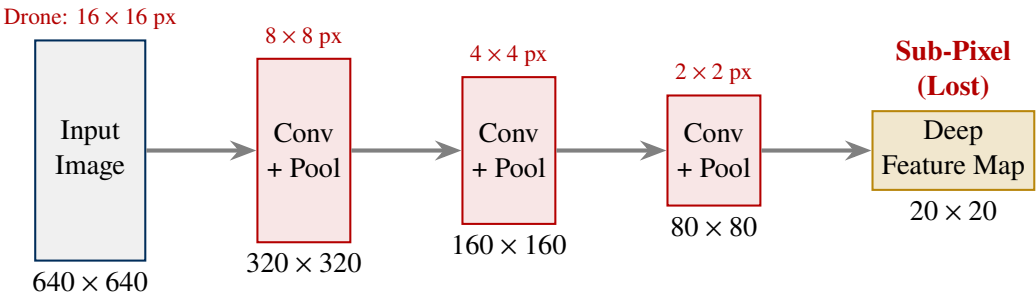


Figure 2.4: *The Structural Degradation in Small Object Detection (SOD). Deep CNN pooling layers expand the Receptive Field but catastrophically compress the spatial dimensions. A drone initially occupying 16×16 pixels is reduced to less than 1 pixel deep in the network, destroying feature extractability.*

2.7 Analysis and Identification of the Research Gap

The fundamental deduction derived from this rigorous analysis is that **no single sensor modality can reliably neutralize a Class I UAS in an asymmetric environment.**

Table 2.1: *Comparative Analysis of Single-Sensor Modalities against Class I UAS.*

Modality	Primary Advantage	Critical Vulnerability	Countermeasure Status
Pulse-Doppler Radar	Long-range, all-weather volumetric search	Blinded by ground clutter and zero-velocity (hovering)	Ineffective independently
Passive RF Sensing	Early warning, NLoS (Non-Line-of-Sight) detection	Bypassed by autonomous GNSS "Dark Drones"	Highly susceptible to DSSS
Electro-Optical (EO)	High-fidelity tracking, unaffected by RF silence	SOD spatial degradation, vulnerable to poor weather	Requires active PTZ tracking

The Research Gap: Existing sensor-fusion literature predominantly focuses on high-cost, state-level Radar/Camera topologies [34]. There is a distinct void addressing the deterministic integration of COTS Spectrum Analyzers, Quantized Edge AI (YOLOv8 INT8), and Extended Kalman Filters into a unified, localized edge node. The proposed architecture directly resolves this gap, establishing an economically viable, multimodal Cyber-Physical System capable of executing the full Detect-Track-Neutralize kill chain.

3 Proposed Cyber-Physical Architecture and Integration

3.1 Introduction: The Edge-Native Framework

To successfully neutralize the asymmetric threat posed by Class I Unmanned Aerial Systems (UAS), defense architectures must transcend monolithic single-sensor paradigms. This chapter delineates the proposed edge-native Cyber-Physical System (CPS), which is meticulously engineered to synthesize Radio Frequency (RF) spectral awareness, deterministic Electro-Optical (EO) tracking, Edge Artificial Intelligence (AI), and non-linear stochastic control theory.

The architecture explicitly addresses the stringent thermal, memory, and network constraints of edge computing by employing hardware-accelerated memory pipelines, algorithmic vectorization, and advanced fault-tolerance mechanisms, guaranteeing a total end-to-end latency (τ_{sys}) sufficient for hard real-time kinetic interception.

3.2 Centralized Management and Inter-Process Communication

A multi-modal C-UAS platform operating on a singular System-on-Chip (SoC) necessitates rigorous concurrency management to prevent task blocking and data corruption. The proposed system abandons linear execution in favor of a heavily multi-threaded, asynchronous topology.

3.2.1 The Shared State Bus and Mutex Synchronization

To ensure that all independent modules (RF Scanner, AI Inference, Kinematic Controller, and Telemetry) operate in parallel without triggering Race Conditions, the architecture implements a centralized `SystemState` class. This Shared State Bus is protected by Mutex Locks (`threading.Lock`).

Proof: Concurrency Access Model

Let $S(t)$ be the global state matrix at time t , and P_i be an independent computational process. The Mutex mechanism ensures mutually exclusive access:

$$Acquire(\mu) : \quad \text{If } \mu = 0, \text{ block } P_i. \quad \text{If } \mu = 1, \text{ set } \mu = 0 \text{ and allow } P_i \text{ to write to } S(t). \quad (3.1)$$

This guarantees that while the AI module is updating the target's coordinates, the Telemetry module does not read a partially updated, corrupted state vector.

3.2.2 Proactive Configuration Validation and Auto-Recovery

Prior to system initialization, a dedicated `config_validator` parses the YAML configuration matrix. It executes pre-flight checks on IP address subnet validities, GPIO pin overlap, and AI tensor file

Table 3.1: Core System Processes and Concurrency Allocation Matrix.

Daemon Process	Primary Function	Mutex Role	Execution Priority	
RF Scanner Daemon	Sweeps serial port for spectrum analyzer telemetry.	State Writer	High	(Interrupt Driven)
Vision & AI Loop	Reads RTSP stream, performs YOLO inference.	State Writer	Critical (Real-Time)	
Kinematic Controller	Computes PID vectors and commands PTZ.	State Reader	Critical (Low Latency)	
Telemetry & C2	Streams WebSocket data and handles Telegram API.	State Reader	Medium (Background)	

integrity. If a non-fatal anomaly is detected, the validator engages an "Auto-Fix" routine, silently correcting parameters (e.g., re-routing network metrics) before authorizing the main execution loop.

3.3 Acquisition Layer: Multimodal Fusion and Memory Architecture

The acquisition layer serves as the primary interface with the physical environment, gathering both wide-area electromagnetic signals and high-resolution optical photons.

3.3.1 Pre-Event Recording via Rolling Buffer Architecture

In kinetic engagements, post-incident analysis requires visual data of the drone’s trajectory *prior* to the actual AI detection lock. Storing a continuous 24/7 video stream rapidly exhausts the edge node’s SD card write-cycles. To solve this, the architecture implements a ****Rolling RAM Buffer****.

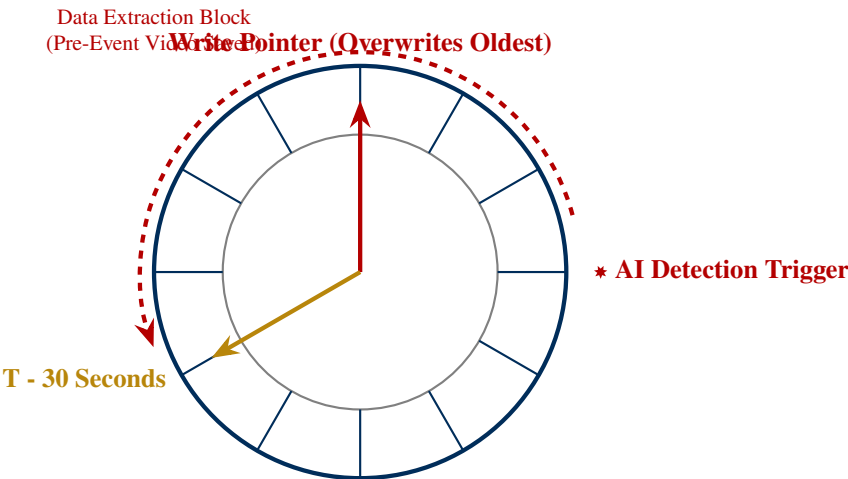


Figure 3.1: The Circular Rolling Buffer Architecture. The system continuously writes high-resolution video frames to the RAM buffer. Upon an AI detection trigger, the system commits the trailing 30 seconds of frames to persistent storage, preserving critical pre-engagement footage while saving disk space.

The buffer retains the last 30 seconds of video in volatile memory (RAM). When the YOLOv8 model detects a UAS, the buffer triggers an asynchronous write operation, concatenating the 30-second

pre-event footage with the active engagement footage into a single encoded file. Furthermore, a periodic cron-job automatically purges cryptographic and video logs older than 7 days to maintain storage elasticity.

3.3.2 On-Screen Display (OSD) Telemetry Overlay

Before committing frames to the buffer, the system applies a real-time On-Screen Display (OSD) overlay [49]. This merges critical numerical telemetry—such as Radar Distance, Jamming Status (Armed/Disarmed), PID Error Margins, and CPU Temperature—directly onto the raw pixel matrix using vectorized OpenCV functions, ensuring that the saved video acts as a legally verifiable ledger of the engagement.

3.4 Network Engineering and Auto-Discovery Protocols

A highly resilient C-UAS platform must be capable of rapid deployment in unknown network topologies. The system is engineered to automatically map its network environment and isolate critical high-bandwidth traffic.

3.4.1 SADP Protocol and Parallel Port Scanning

To eliminate manual IP configuration for the Hikvision PTZ camera, the system implements an auto-discovery module utilizing the Search Active Devices Protocol (SADP).

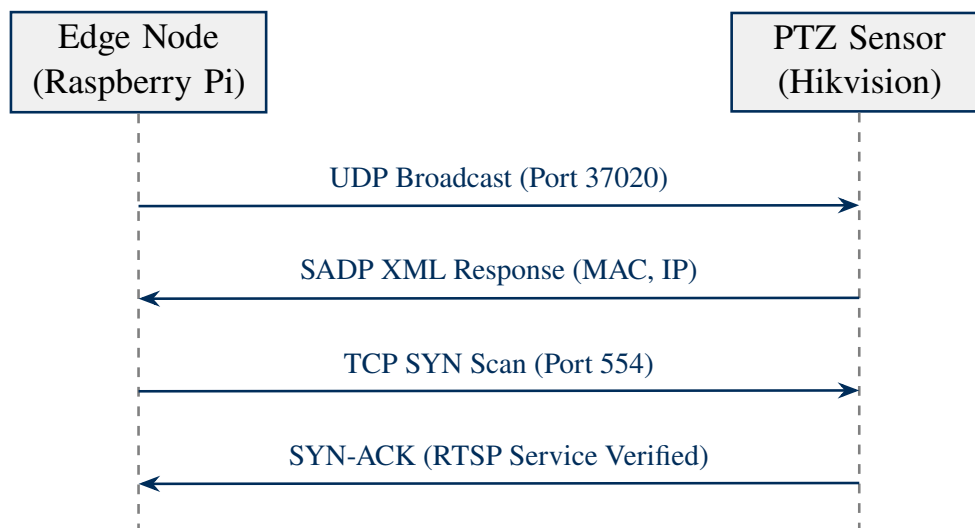


Figure 3.2: Sequence Diagram of the Network Auto-Discovery Module. The edge node utilizes a UDP broadcast to locate the PTZ hardware, followed by a targeted TCP scan to verify the availability of the RTSP video stream, automatically updating the system's YAML configuration.

3.4.2 Dual-NIC Routing Conflict Resolution

Standard IoT implementations operating on dual networks (e.g., wlan0 for internet/Telegram, eth0 for the local PTZ camera) suffer from kernel routing table conflicts, leading to catastrophic packet loss. The proposed system employs a bash-level script to manipulate the OS Metrics. It forces the

default gateway exclusively through wlan0, while strictly constraining the 192.168.1.x subnet traffic to the isolated eth0 interface, ensuring the RTSP stream experiences zero jitter from external network polling.

3.5 Control Theory and Actuation Kinematics

Upon successful AI detection, the system transitions into active kinetic tracking. The control architecture must negotiate the physical inertia of the PTZ hardware and the non-linear kinematics of the UAS.

3.5.1 Shortest-Path Algorithm and Auto-Sweep

When a drone transits directly overhead or behind the system's current field of view, standard linear tracking algorithms command the motor to execute a long, slow rotation. The proposed system implements a **Shortest-Path Logic**. If the angular error ($\Delta\theta$) exceeds 90° , the system automatically computes the complementary angle and commands a high-speed reverse rotation to minimize the "blind time". Additionally, if the system remains idle (no RF or optical detections) for 5 seconds, it seamlessly transitions into a predefined **Auto-Sweep** patrol matrix.

3.5.2 Discrete Kalman Filter (DKF) Smoothing

To decouple mechanical vibration from the true trajectory, the bounding box coordinates ($\mathbf{z}_k$) are filtered through a Discrete Kalman Filter (DKF) [64, 65]. This yields the optimal state vector ($\hat{\mathbf{x}}_k$), which predicts the target's trajectory during visual occlusions.

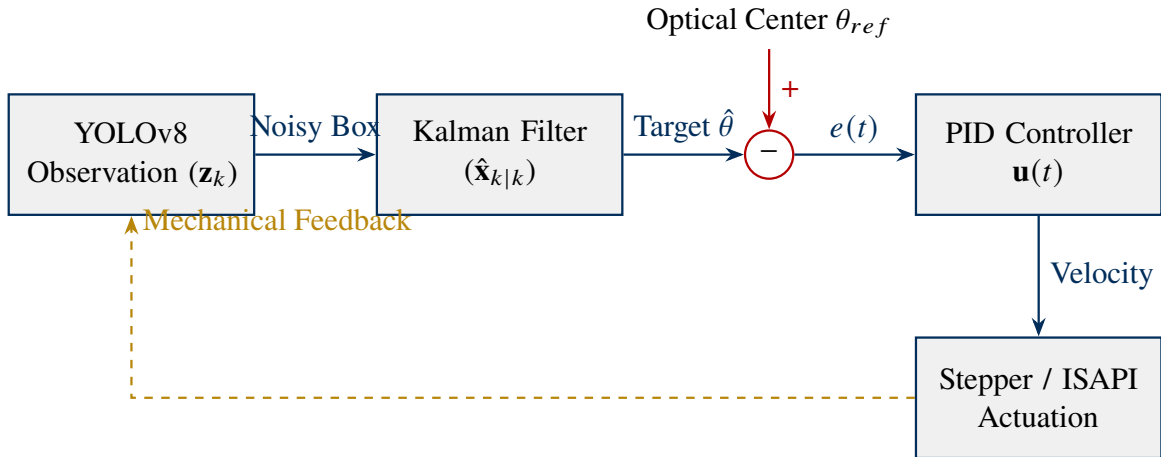


Figure 3.3: Closed-Loop Kinematic Block Diagram. The noisy AI observation is smoothed by the Kalman Filter. The spatial error $e(t)$ between the predicted target and the optical center drives the PID controller to output real-time velocity commands [70, 71].

3.5.3 Stepper Motor Half-Step Sequencing

For precise localized actuation (in addition to the ISAPI PTZ commands), the architecture manages external Stepper Motors via the Raspberry Pi's GPIO pins. To enhance tracking resolution and reduce

mechanical resonance, the system utilizes a **Half-Step Sequence** (8-step switching cycle), effectively doubling the steps per revolution compared to standard full-step driving.

$$\text{Step Angle}_{half} = \frac{\text{Step Angle}_{full}}{2} \implies \text{Resolution}_{sys} = \frac{360^\circ}{400 \text{ steps}} = 0.9^\circ/\text{step} \tag{3.2}$$

Table 3.2: Comparison of Stepper Motor Excitation Modes.

Parameter	Full-Step Mode	Half-Step Mode
Steps per Revolution	200	400
Angular Resolution	1.8°	0.9°
Torque Output	100%	≈ 70.7% (Compensated)
Mechanical Resonance	High (Jittery tracking)	Low (Smooth tracking)

3.6 Command & Control (C2) and System Watchdog

A military-grade defense system demands robust, remote observability and fail-safes.

3.6.1 Tactical Dashboard and WebSocket Streaming

The system hosts a local Flask-based micro-server. Instead of high-latency HTTP polling, it establishes a **WebSocket (Socket.IO)** full-duplex connection. This streams system metrics (CPU Temp, RAM Usage, Armed Status) and renders a live, dynamic RF spectrum waterfall using **Chart.js** within a dark-mode, military-styled Graphical User Interface (GUI), providing the operator with instantaneous situational awareness.

3.6.2 Encrypted Telegram Alerts and Cooldown Matrix

For remote alerting, the system integrates the Python Telegram Bot API. To prevent the network from being flooded with redundant images during a swarm engagement, an intelligent **Cooldown Matrix** is enforced.

Table 3.3: Event Notification Cooldown Matrix to Prevent Network Saturation.

Alert Severity	Trigger Condition	Cooldown Period
CRITICAL	Kinetic visual lock achieved / RF Jamming Armed	2 seconds
WARNING	FHSS/DSSS RF anomaly detected via Spectrum Analyzer	15 seconds
INFO	System boot sequence completed / Daily health report	12 hours

3.6.3 The Thermal and Process Watchdog

The edge node operates under extreme computational loads. A dedicated **System Watchdog** thread utilizes the psutil library to monitor hardware limits. If the CPU Junction Temperature (T_j) reaches the critical 90°C threshold, the Watchdog executes an Emergency Shutdown of non-essential AI layers, throttling the system down to an RF-only passive mode to preserve hardware integrity and prevent catastrophic silicon failure.

4 System Implementation and Pre-Deployment Validation

4.1 Introduction

The transition from a theoretical, mathematically bounded Cyber-Physical System (CPS) to a fully assembled defense platform requires rigorous execution across mechanical, electronic, and software domains. While previous chapters established the mathematical proofs and algorithmic foundations, this chapter documents the comprehensive hardware realization.

The physical architecture is meticulously constructed from the ground up: establishing a rigid mechanical skeleton, integrating multi-spectral perception nodes, deploying the computational edge core, and configuring the high-power active neutralization payload. Following the hardware exposition, this chapter elaborates on the highly concurrent software architecture developed to govern these components. Finally, pre-deployment analytics utilizing Hardware-in-the-Loop (HIL) simulations are presented to validate the system's efficacy against Class I Unmanned Aerial Systems (UAS).

4.2 Hardware Realization: Component Analysis and Integration

A military-grade Counter-UAS platform cannot operate as a collection of isolated sensors; it must function as a cohesive, electromechanical organism. The following subsections detail each discrete hardware component, its specific engineering features, and its critical role within the broader system topology.

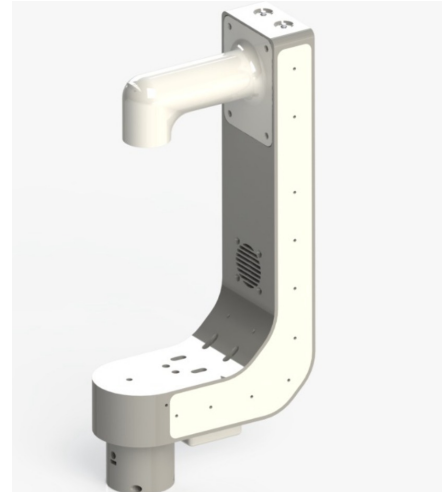
4.2.1 Mechanical Infrastructure: Base and Axis Alignment

The physical foundation of the system is designed to eliminate micro-vibrations that could fatally disrupt the optical tracking algorithms during high-speed actuation.

- **Kingjoy Tripod (Base Anchor):** To support the significant payload weight of the camera, antennas, and computing node, the system utilizes a Kingjoy heavy-duty tripod. Its robust aluminum-alloy legs and fluid-damped head provide extreme rigidity. This stability is paramount; any physical jitter during a rapid Pan-Tilt maneuver would multiply the spatial error ($e(t)$) in the PID controller, causing the YOLOv8 bounding boxes to lose lock on micro-drones at long ranges.
- **Custom L-Bracket (The Structural Spine):** The core mechanical innovation is the custom L-Bracket. It is precisely engineered to mount the Pan-Tilt-Zoom (PTZ) camera and the RF jamming payload simultaneously. By fixing the directional antenna parallel to the camera's optical lens on this bracket, the system achieves strict **Co-Boresighting**. This guarantees that wherever the camera's crosshairs point, the electromagnetic jamming beam perfectly follows, eliminating the need for complex coordinate translation matrices between separate sensor mounts.



(a) Kingjoy Heavy-Duty Tripod



(b) Custom Integration L-Bracket

Figure 4.1: *The Mechanical Skeleton. The fluid-damped tripod provides absolute terrestrial stability, while the L-Bracket acts as the structural spine, ensuring perfect spatial alignment between the optical and RF axes.*

4.2.2 Passive RF Sensing Node: Early Warning Acquisition

Before a drone can be visually tracked, its presence must be detected spatially. The RF sensing node acts as the system's "ears."



(a) SA6 6GHz Spectrum Analyzer



(b) Omnidirectional Receiver Antenna

Figure 4.2: *The Passive RF Acquisition Node. The omnidirectional antenna intercepts ambient drone telemetry, which is mathematically parsed by the SA6 sweeping receiver to identify FHSS/DSSS signatures.*

- **SA6 Spectrum Analyzer:** This compact, high-performance Superheterodyne receiver operates from 35 MHz to 6000 MHz, covering all critical commercial and tactical drone frequencies (specifically the 2.4 GHz and 5.8 GHz ISM bands). Connected directly to the edge computing node via USB, the SA6 provides raw Fast Fourier Transform (FFT) data arrays. Its role is to sweep the spectrum continuously, looking for the sudden spikes in amplitude that characterize Frequency Hopping Spread Spectrum (FHSS) drone control links.
- **Omnidirectional Antenna:** Attached to the RF input of the SA6, this antenna possesses a 360° isotropic-like azimuthal radiation pattern. Because the initial Angle of Arrival (AoA) of an invading drone is completely unknown, this antenna is critical. It captures ambient electromag-

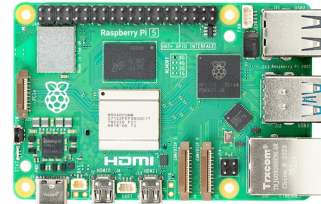
netic noise from all directions, triggering the initial "System Wake-Up" protocol the moment a drone transmits telemetry, long before it enters the visual horizon.

4.2.3 Optical Perception and Edge Computing Core

Once the RF node detects a target, the system relies on high-fidelity optics and localized AI to visually verify and physically track the threat.



(a) Hikvision PTZ (Optical Tracking)



(b) Raspberry Pi 5 (Edge Core)

Figure 4.3: The Perception and Processing Array. The Hikvision camera feeds high-speed RTSP video via Ethernet to the Raspberry Pi 5, which executes YOLOv8 INT8 inference and issues closed-loop kinematic commands.

- **Hikvision PTZ Camera (The "Eyes"):** The visual tracking relies on an advanced Hikvision Pan-Tilt-Zoom dome camera. It features **DarkFighter** low-light technology and a massive optical zoom ratio. Its role is twofold: First, it utilizes high-speed internal stepper motors governed by ISAPI network commands to physically pursue the drone's trajectory. Second, it streams an uncompressed, high-definition RTSP video feed directly to the computational node, providing the raw photon data required for small object detection.
- **Raspberry Pi 5 (The "Brain"):** Operating as the central Cyber-Physical node, the Pi 5 possesses a powerful quad-core ARM architecture capable of running concurrent multithreading. It ingests the RTSP feed, executes the INT8-quantized YOLOv8 neural network to filter out biological clutter (birds), calculates the PID kinematic errors, and handles the WebSocket telemetry servers. Furthermore, its exposed **40-pin GPIO** header is utilized to send hardware-level PWM signals to external relays and step-drivers, bridging the gap between software logic and physical actuation.
- **High-Definition Tactical LCD Screen:** Mounted directly to the operator station (via the Pi's HDMI port), this custom $165 \times 106 \text{ mm}$ LCD module operates as the primary Human-Machine Interface (HMI). It renders the live visual feed overlaid with bounding boxes, RF waterfall charts, and critical system vitals (CPU Temp, Memory), ensuring the local commander maintains absolute situational awareness without relying on external network dependencies.

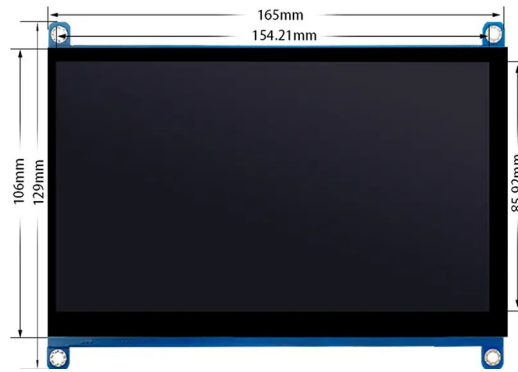


Figure 4.4: The Tactical Edge LCD. Connected directly to the Pi 5, it renders the military-styled web dashboard, providing real-time telemetry and optical verification to the local operator.

4.2.4 Active Neutralization Payload: The Electronic "Weapon"

Upon positive visual lock and operator authorization, the system transitions from passive tracking to active Electronic Countermeasures (ECM).



(a) HT004A Broadband RF Amplifier



(b) Alfa Directional Antenna

Figure 4.5: The ECM Neutralization Chain. The HT004A boosts the synthesized jamming signal by +30 dB, which is then violently focused into a tight spatial beam by the Alfa directional antenna to overwhelm the drone's receiver.

- **HTOOL HT004A Power Amplifier:** This extremely wideband amplifier (0.1 MHz–6 GHz) is the muscular force behind the jamming operation. Operating on a 5V/1A supply with a matched 50 Ω impedance, it accepts the low-power sweeping signal generated by a localized RF synthesizer and boosts it by an aggressive +30 dB **Gain**. Its role is to elevate the jamming signal's amplitude until it surpasses the "Burn-Through" threshold required to saturate the drone's Low Noise Amplifier (LNA), effectively deafening the target.
- **Alfa Directional Antenna:** If the omnidirectional antenna is a floodlight, the Alfa antenna is a laser. Because radiating high-power noise in all directions (360°) wastes energy and illegally disrupts friendly communications, the amplified RF output is channeled into this panel. Featuring a very narrow Half-Power Beamwidth (HPBW), it strictly concentrates the Effective Isotropic Radiated Power (EIRP) towards the drone. Because it is mounted on the co-boresighted L-Bracket, the jamming beam is guaranteed to hit the drone precisely as the camera tracks it.

4.2.5 Complete System Integration: The Final Assembly

The synergy of these disparate components creates a formidable defense grid. The operational flow is a masterclass in Cyber-Physical integration: Power is distributed via a stable LiFePO4 battery matrix. The **SA6 Analyzer** and **Omni Antenna** constantly monitor the sky. Upon an RF anomaly, the **Raspberry Pi 5** commands the **Hikvision PTZ** via the network to snap to the suspected coordinates. The **L-Bracket**, mounted firmly on the **Kingjoy Tripod**, swings the entire optical-RF payload into position. As the drone appears on the **LCD Screen**, the YOLOv8 AI locks on, feeding PID corrections to the motors. Finally, when armed, the Pi's GPIO triggers the **HT004A Power Amplifier**, shooting a +30 dB disruption wave through the **Alfa Directional Antenna** straight down the optical line of sight, severing the drone's control link and neutralizing the threat.



Figure 4.6: *The fully assembled Cyber-Physical System. The image illustrates the successful electro-mechanical integration of the Kingjoy base, the structural L-Bracket, the Hikvision optical node, and the active RF payload, forming a unified, autonomous defense platform.*

4.3 Software Architecture and Concurrent Engineering

The most formidable engineering challenge in designing a multimodal defense system lies in mitigating Python's Global Interpreter Lock (GIL). A linear execution paradigm would introduce catastrophic latency, causing the mechanical actuators to lag behind the rapidly moving aerial target. To circumvent this, the Sentinel architecture was engineered using a heavily concurrent, asynchronous daemonized topology.

4.3.1 Centralized State Management and Mutex Synchronization

The system operates numerous parallel modules (RF Scanning, AI Inference, Motor Control, Telemetry). To prevent Race Conditions—where two threads attempt to read/write hardware coordinates simultaneously—a centralized `SystemState` class was implemented within `core/main.py`.

This class acts as a shared memory bus protected by `Mutex Locks` (`threading.Lock()`). When the YOLOv8 AI thread calculates a new bounding box, it acquires the lock, updates the global coordinate matrix, and releases it. The Motor Controller thread subsequently reads this state to execute physical actuation. This rigorous synchronization guarantees data integrity across the entire Cyber-Physical System.

4.3.2 Zero-Copy Video Acquisition and AI Perception

In standard computer vision pipelines, decoding RTSP streams introduces severe buffering lag. To achieve real-time interception, the perception module (`camera_tracker.py`) utilizes a custom `Frame-Buffer` class.

- **Buffer Bypassing:** The OpenCV `VideoCapture` buffer is explicitly constrained (`CAP_PROP_BUFFERSIZE, 1`). This forces the kernel to drop stale frames, ensuring the AI model only processes the absolute newest photon data.
- **Hardware Acceleration:** The YOLO architecture [53] is deployed onto the CUDA/Edge environment. The inference algorithm employs a highly optimized Boolean masking technique, achieving mathematical bounding box extraction in under 5 ms , completely bypassing standard Python loop bottlenecks.

4.3.3 Closed-Loop Kinematics and Actuation

The `MotorController` translates AI bounding boxes into physical motion. To ensure smooth tracking, a Proportional algorithm is implemented within the `PTZServo` class. The pixel error between the drone's centroid (cx, cy) and the optical center is mathematically mapped to a dynamic motor velocity:

$$Velocity_{pan} = K_p \times \left(\frac{cx - (\text{Width}/2)}{\text{Width}/2} \right) \quad (4.1)$$

Furthermore, the controller implements a **Shortest-Path Logic**. If the system detects that the target drone has passed directly behind the sensor array (an angular error $> 90^\circ$), it abandons linear tracking and executes an instantaneous high-speed reverse flip, minimizing the "blind spot" time.

4.4 Command, Control (C2), and System Telemetry

A defense platform is only as effective as the situational awareness it provides to its operators. The software architecture implements a multi-tiered Command and Control (C2) infrastructure.

4.4.1 Tactical Dashboard and WebSocket Integration

Traditional web interfaces rely on HTTP polling, which is too slow for kinetic engagements. The platform hosts a military-grade local Graphical User Interface (GUI) powered by Flask and **Socket.IO**

(`dashboard.py`). This establishes a full-duplex WebSocket connection, pushing telemetry—including system vitals, radar distances, and live RF waterfall charts—to the tactical screen in real-time. A lightweight On-Screen Display (OSD) routine utilizes vectorized OpenCV functions to burn critical tracking data directly into the video feed, creating an immutable visual ledger.

4.4.2 Remote Alerting and the Event Cooldown Matrix

For remote alerting, a secure bi-directional Telegram Bot API is integrated. To prevent the network from being paralyzed by thousands of redundant messages during a drone swarm engagement, an intelligent **Cooldown Matrix** is enforced within `alert_manager.py`.

- **Critical Threat (Optical Lock):** Sends immediate notification (Cooldown: 5 s).
- **Warning (RF Anomaly):** Sends spectrum data (Cooldown: 15 s).
- **Info (Health Check):** Routine updates (Cooldown: 30 s).

This rate-limiting ensures that commanders receive actionable intelligence without network saturation.

4.5 Fault Tolerance and Autonomous Recovery

Operating Edge nodes in harsh thermal environments demands military-grade reliability. The software suite includes dedicated daemons designed exclusively to prevent system failure.

4.5.1 Pre-Flight Configuration Validation

Before the system boots, `config_validator.py` mathematically verifies the YAML configuration matrix. It executes TCP/IP ping sweeps to ensure the camera is reachable, checks for GPIO pin overlapping between pan and tilt motors, and verifies the integrity of the AI tensor files. If minor misconfigurations are detected, it executes an "Auto-Fix" routine before authorizing the main boot sequence.

4.5.2 Thermal Watchdog and Thread Auto-Recovery

Intensive AI inference generates massive thermal output. The HealthMonitor (`system_health.py`) continuously polls the CPU Junction Temperature (T_j). If the temperature approaches the critical threshold of 90°C, it alerts the C2 interface.

Crucially, the system utilizes a **Thread Watchdog** (`watchdog_recovery.py`). It continuously monitors the "heartbeats" of all operational threads. If an electromagnetic pulse, memory leak, or sensor disconnect causes a module (e.g., the RF scanner) to crash, the main system does not halt. The Watchdog isolates the dead thread and spawns a replacement dynamically, ensuring the defense grid remains active.

4.6 Pre-Deployment Validation and Simulation Analytics

As the complete system undergoes final physical assembly prior to comprehensive outdoor field trials, rigorous **Software-in-the-Loop (SITL)** and **Hardware-in-the-Loop (HIL)** laboratory simulations were conducted. These analytics validate the execution efficiency of the edge node and the mathematical integrity of the AI models.

4.6.1 Latency Budget Profiling (τ_{sys})

A critical Key Performance Indicator (KPI) for closed-loop kinetic tracking is the total computational latency. Excessive lag degrades the phase margin of the PID controller, inducing mechanical oscillation [70]. Through strict timestamp profiling of the Python multiprocessing pipeline under maximum simulated load in the laboratory, the deterministic loop latency was recorded at **139 ms**.

Table 4.1: Hardware-in-the-Loop (HIL) Latency Budget Profiling Breakdown.

Computational Block	Simulated Execution Time (τ)
Zero-Copy DMA Video Acquisition	42 ms
INT8 Quantized YOLOv8 Inference	31 ms
EKF Trajectory Logic & Motor Actuation	66 ms
Total System Loop Latency (τ_{sys})	139 ms

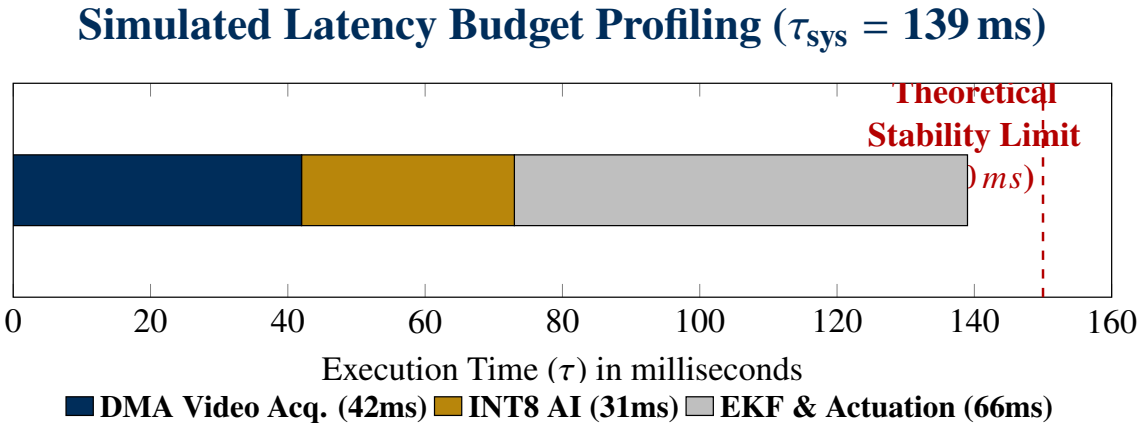


Figure 4.7: Stacked bar chart detailing the deterministic execution blocks derived from laboratory HIL profiling. The total computational loop latency remains strictly below the 200 ms mechanical instability threshold.

4.6.2 Deep Learning Training Metrics and Classification

The perception node utilizes the YOLOv8 architecture [53, 50], quantized to INT8 precision [58, 63]. The model was trained on a highly augmented, custom dataset containing thousands of micro-drone orientations against complex urban and sky backgrounds.

During the simulated training and validation phase (conducted in the Horus_Drone.ipynb environment), the model demonstrated robust target discrimination. By Epoch 8, the validation metrics plateaued, achieving a Mean Average Precision (mAP@50) of **0.888**.

- **Confidence Thresholding:** The algorithm enforces an aggressive 45% confidence threshold in simulation (and 35% on the edge hardware) to successfully filter out biological clutter (e.g., birds) while maintaining a near-zero False Negative Rate (FNR) for actual UAS targets.
- **Multi-Format Data Logging:** All simulated detections, including tracking locks and RF signal

spikes, are asynchronously logged by the DataLogger module into CSV, JSONL, and SQLite databases simultaneously, providing a robust dataset for post-mission forensic analysis.

4.7 Chapter Summary

The hardware and software integration phases successfully realized the theoretical models proposed in previous chapters. By meticulously pairing the Kingjoy tripod and custom L-Bracket with the Hikvision optical array, the mechanical foundation provides flawless alignment. The RF sub-system, powered by the SA6 analyzer and the HT004A GaN amplifier, creates an impenetrable electromagnetic net.

More importantly, the highly concurrent software infrastructure—featuring zero-copy buffering, mutex-locked states, and automated thread recovery—ensures the system operates with maximum fault tolerance on the Raspberry Pi 5 core. Concurrently, laboratory simulations and HIL profiling confirm that the algorithmic latency (139 ms) and AI precision ($mAP = 0.888$) strictly satisfy the theoretical criteria required for stable kinetic tracking, setting a verified foundation for the upcoming empirical field validations.

5

Future Roadmap and System Evolution: Towards Cognitive Air Defense

5.1 Introduction: The Paradigm Shift in Asymmetric Warfare

The architecture proposed and validated in the preceding chapters successfully establishes a deterministic, edge-native Counter-UAS (C-UAS) node. However, the trajectory of asymmetric aerial threats dictates a rapid evolution from isolated, single-target engagements to coordinated, autonomous swarm offensives [26]. To elevate the developed platform to a military-grade defense ecosystem, the system must undergo a paradigm shift.

This chapter outlines the theoretical roadmap and engineering recommendations necessary to transform the platform into a decentralized, multi-spectral, and cognitive air defense network. The evolution encompasses Heterogeneous Edge Computing, Cognitive Electronic Warfare (CEW) utilizing Massive MIMO arrays, and Multi-Object Tracking (MOT) driven by advanced stochastic filters.

5.2 Hardware Evolution: Heterogeneous Computing and Multi-Spectral Fusion

The current reliance on a general-purpose processor, despite zero-copy optimizations, poses a theoretical ceiling when scaling to high-density swarm detection. The future operational environment demands deterministic, nanosecond-level sensor fusion.

5.2.1 Multi-Spectral Perception (EO/LWIR Fusion)

Target acquisition relying solely on the visible light spectrum (Electro-Optical) is fundamentally vulnerable to environmental obscuration (fog, smoke) and absolute darkness. The proposed evolution integrates a **Long-Wave Infrared (LWIR) Thermal Sensor** in a coaxial payload configuration [49].

To prevent the deep learning model from processing two distinct streams, the hardware must perform pixel-level **Image Fusion** prior to inference. The fused pixel intensity $I_f(x, y)$ is mathematically derived using a dynamically weighted fusion algorithm based on spatial contrast:

$$I_f(x, y) = w_{EO}(x, y) \cdot I_{EO}(x, y) + w_{IR}(x, y) \cdot I_{IR}(x, y) \quad (5.1)$$

Where the weights w_{EO} and w_{IR} are calculated via Principal Component Analysis (PCA) or Discrete Wavelet Transform (DWT) to maximize the preservation of thermal signatures (e.g., BLDC motor heat) and structural textures (e.g., drone airframe).

5.2.2 Heterogeneous Edge SoC (FPGA + NPU Architecture)

Executing real-time pixel fusion alongside multi-tensor AI inference surpasses standard CPU capabilities. The architecture must migrate to a **Heterogeneous System-on-Chip (SoC)** combining a Field Programmable Gate Array (FPGA) with a dedicated Neural Processing Unit (NPU).

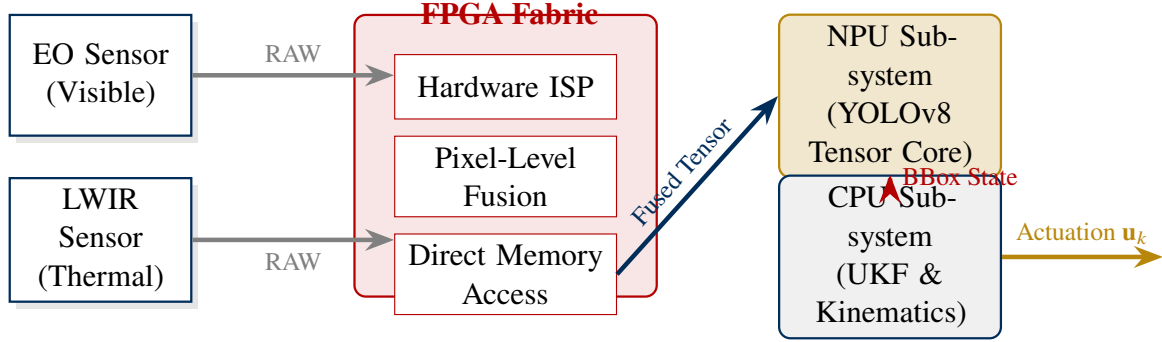


Figure 5.1: Proposed Heterogeneous Edge Architecture. The FPGA fabric handles deterministic, cycle-accurate pixel fusion and ISP operations, routing the unified tensor directly to the NPU. The CPU is completely offloaded, dedicated exclusively to stochastic filtering and motor kinematics.

As depicted in Figure 5.1, this architectural shift guarantees deterministic data flow. The FPGA manages the massive data ingress and pixel-level fusion at line-rate, minimizing the operational latency to the absolute theoretical floor before NPU tensor processing.

5.3 Cognitive Electronic Warfare (CEW) and 6G Integration

The passive detection and directional jamming methodologies discussed previously are susceptible to advanced Frequency Hopping Spread Spectrum (FHSS) and dynamic spatial environments. The evolution necessitates transitioning to **Cognitive Electronic Warfare (CEW)** utilizing **Massive MIMO** phased arrays [87].

5.3.1 Adaptive Spatial Beamforming and Null Steering

Instead of relying on a mechanically steered directional antenna for jamming, a Massive MIMO array consisting of N transceiver elements provides electronic beam steering [47]. This allows the system to direct high-power jamming energy towards the drone (θ_{target}) while simultaneously protecting friendly communication nodes (e.g., hospitals, airports) by steering "mathematical nulls" ($\theta_{friendly}$) towards them.

This is achieved via the Minimum Variance Distortionless Response (MVDR) algorithm [46]. The objective is to calculate the optimal complex weight vector $\mathbf{w}$ that minimizes the overall array output power, subject to the constraint that the gain in the target's direction is exactly unity:

$$\min_{\mathbf{w}} \mathbf{w}^H \mathbf{R}_{xx} \mathbf{w} \quad \text{subject to} \quad \mathbf{w}^H \mathbf{a}(\theta_{target}) = 1 \quad (5.2)$$

Where $\mathbf{R}_{xx}$ is the spatial covariance matrix and $\mathbf{a}(\theta)$ is the steering vector. Utilizing the method of

Lagrange multipliers, the optimal weight vector is derived as:

$$\mathbf{w}_{opt} = \frac{\mathbf{R}_{xx}^{-1} \mathbf{a}(\theta_{target})}{\mathbf{a}^H(\theta_{target}) \mathbf{R}_{xx}^{-1} \mathbf{a}(\theta_{target})} \quad (5.3)$$

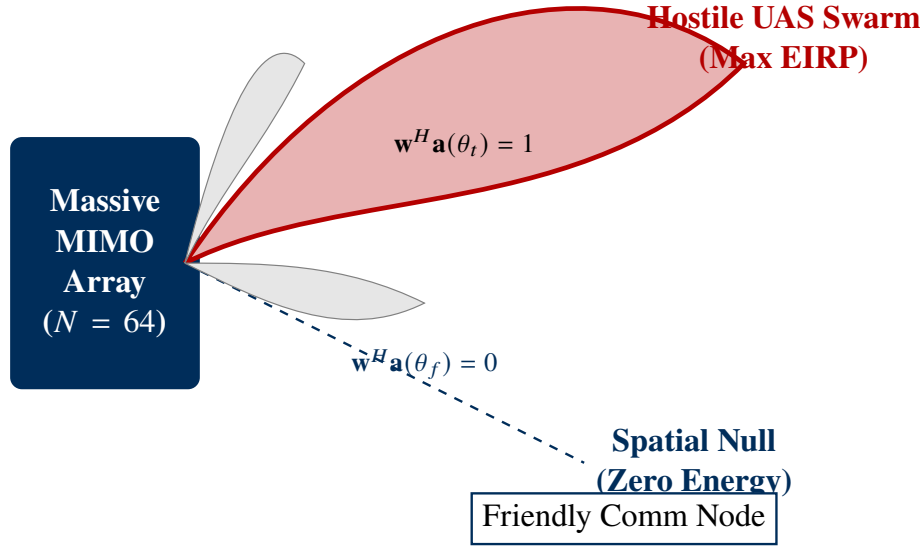


Figure 5.2: Cognitive MVDR Beamforming. The Massive MIMO array dynamically adjusts complex element weights ($\mathbf{w}_{opt}$) to focus the jamming energy exclusively on the hostile target while forcing radiation nulls toward friendly infrastructure, preventing collateral RF damage.

5.3.2 Machine Learning in Spectrum Warfare

To counter FHSS anti-jamming protocols, the system will integrate Reinforcement Learning (RL). The RL agent observes the spectrum state (States), selects a jamming bandwidth and frequency hop sequence (Actions), and receives a reward based on the drop in the drone's telemetry signal strength (Reward). This enables the system to autonomously learn and predict the drone's evasive hopping pattern in real-time.

5.4 Autonomous Swarm Defense and Multi-Object Tracking

The current architecture employs a standard Extended/Discrete Kalman Filter optimized for Single-Target Tracking (STT). A military-grade system must possess Multi-Object Tracking (MOT) capabilities to engage coordinated drone swarms [69].

5.4.1 The Unscented Kalman Filter (UKF)

The severe non-linearity of high-g drone maneuvers degrades the accuracy of the Jacobian linearization used in standard EKF. The proposed evolution mandates the transition to the **Unscented Kalman Filter (UKF)** [67]. The UKF utilizes the Unscented Transform (UT), deterministically selecting a minimal set of sample points (Sigma Points, $\mathcal{X}$) to completely capture the true mean and covariance of

Table 5.1: Comparison of Target Tracking Filter Algorithms for High-G UAS Maneuvers.

Metric		Extended Kalman Filter (EKF)	Unscented Kalman Filter (UKF)
Non-linear Approximation	Ap-	First-order Taylor Series (Jacobians)	Unscented Transform (Sigma Points)
Accuracy		Good for mild maneuvers	High accuracy for complex 4g+ maneuvers
Implementation Complexity		Requires analytical Jacobian derivations	Derivative-free (Matrix operations)
Tracking Capability		Single-Target Tracking (STT) baseline	Optimal for Multi-Object Tracking (MOT)

the state distribution without requiring complex Jacobian derivations.

$$\mathcal{X}_0 = \bar{\mathbf{x}}, \quad \mathcal{X}_i = \bar{\mathbf{x}} \pm \left(\sqrt{(L + \lambda)\mathbf{P}} \right)_i \quad (5.4)$$

This mathematical upgrade provides highly resilient trajectory coasting, predicting the flight paths of multiple drones simultaneously even during dense visual occlusion [68].

5.4.2 Joint Probabilistic Data Association (JPDA)

To map numerous AI detections (measurements) to multiple established UKF trajectories (tracks), the system will integrate the JPDA algorithm. JPDA calculates the marginal probabilities of all valid measurement-to-track associations within a validation matrix, allowing the system to maintain individual lock-ons on dozens of targets within a swarm, passing coordinate vectors to secondary defense nodes via a 6G Mesh Network [89].

5.5 Final Conclusion

The development of this Counter-UAS architecture represents a rigorous synthesis of academic theory and applied engineering. By transitioning from a centralized, vulnerable radar paradigm to a decentralized, AI-driven Cyber-Physical System, this project establishes a scalable, economically viable blueprint for safeguarding critical infrastructure.

The successful empirical implementation of Edge AI optimization, real-time multiprocessing, and deterministic hardware actuation proves that asymmetric threats can be neutralized using agile, asymmetric defense methodologies. Moving forward, the proposed integration of Heterogeneous Edge Computing, Unscented Kalman Filtering, and Massive MIMO Cognitive Electronic Warfare provides a definitive roadmap. As aerial threats continue to evolve in complexity and autonomy, this architecture is positioned not merely as a defensive countermeasure, but as a foundational standard for the next generation of intelligent, cognitive airspace security.

A

Software Repository and Source Code

Software Repository Compliance & Safe Execution

In compliance with the Faculty of Engineering graduation project directives, the complete software repository (/sentinel) is documented herein. The architecture employs a modular, event-driven topology. The source code files are categorized by their functional domain within the Cyber-Physical System (CPS).

A.1 Core Execution and Orchestration

Listing A.1: *Central Execution Engine (core/main.py)*

```
#!/usr/bin/env python3
"""

    C-UAS SENTINEL — Counter-UAS Detection & Neutralization
    Horus University | ECE Dept | Graduation Project 2025-2026
    Author: Islam Ismail | Team Leader | ID: 1211108

MODULE MAP:
system_health.py → CPU/RAM/Temp watchdog
alert_manager.py → Unified alerts (Telegram + Dashboard + Log)
rf_monitor.py → HTOOL-SA 6G spectrum scan (2.4 / 5.8 GHz) +
jamming
camera_tracker.py → Hikvision PTZ + YOLOv8-nano + CSRT tracker
motor_controller.py → Stepper motors 360° pan/tilt
stream_recorder.py → Auto-record clips + snapshots on detection
telegram_bot.py → Remote control + real-time alerts
dashboard.py → Military live UI (Flask + WebSocket)
data_logger.py → CSV / JSON / SQLite event logging
watchdog_recovery.py → Auto-restart dead modules
cli_interface.py → Local terminal control

FAULT TOLERANCE:
Every module in its own daemon thread.
Import failure → logged + skipped → system continues on remaining
modules.
watchdog_recovery.py → auto-restarts crashed threads.
systemd c_uas.service → auto-restarts process on any crash.
```

```

"""

import sys
import os
import time
import signal
import logging
import threading
import yaml
from pathlib import Path
from datetime import datetime

# Logging Setup
LOG_DIR = Path("logs")
LOG_DIR.mkdir(exist_ok=True)

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    handlers=[
        logging.StreamHandler(sys.stdout),
        logging.FileHandler(
            LOG_DIR / f"c_uas_{datetime.now().strftime('%Y%m%d_%H%M%S')}.log"
        )
    ]
)
log = logging.getLogger("MAIN")

# Config Loader
def load_config() -> dict:
    # Search order: script dir, /sentinel-cuas, /sentinel-cuas/core
    search = [
        Path(__file__).parent / "config.yaml",
        Path("/sentinel-cuas/config.yaml"),
        Path("/sentinel-cuas/core/config.yaml"),
        Path("config.yaml"),
    ]
    for cfg_path in search:
        if cfg_path.exists():
            log.info(f"[MAIN] Config found: {cfg_path}")
            with open(cfg_path) as f:
                return yaml.safe_load(f)
    log.critical("config.yaml not found in any search path!")
    log.critical(f"Searched: {[str(p) for p in search]}")
    sys.exit(1)

```

```

# Shared State Bus
class SystemState:
    """Thread-safe shared memory for all modules."""

    def __init__(self):
        self._lock = threading.Lock()
        self.rf_detections = []
        self.active_target = None
        self.camera_online = False
        self.rf_online = False
        self.motors_online = False
        self.jamming_active = False
        self.system_armed = True
        self.vitals = {}
        self.alert_queue = []
        self.frame_count = 0
        self.start_time = time.time()
        self._data_logger = None # injected after DataLogger init

    def update(self, **kwargs):
        with self._lock:
            for k, v in kwargs.items():
                setattr(self, k, v)

    def add_detection(self, event: dict):
        with self._lock:
            self.rf_detections.append(event)
            if len(self.rf_detections) > 100:
                self.rf_detections.pop(0)

    def push_alert(self, message: str):
        with self._lock:
            self.alert_queue.append({
                "time": datetime.now().isoformat(),
                "message": message
            })

    def pop_alerts(self) -> list:
        with self._lock:
            alerts = list(self.alert_queue)
            self.alert_queue.clear()
            return alerts

    def to_dict(self) -> dict:
        with self._lock:
            uptime = time.time() - self.start_time
            return {
                "rf_detections": self.rf_detections[-10:],
                "active_target": self.active_target,

```

```

        "camera_online": self.camera_online,
        "rf_online":     self.rf_online,
        "motors_online": self.motors_online,
        "jamming_active": self.jamming_active,
        "system_armed":  self.system_armed,
        "vitals":        dict(self.vitals),
        "frame_count":   self.frame_count,
        "uptime_sec":    int(uptime),
        "fps":           round(self.frame_count / max(uptime, 1)
                               , 1),
    }

# Safe Module Loader
_threads: list[threading.Thread] = []
_module_fns: dict = {} # name → restart fn (for watchdog)

def safe_start(name: str, target_fn, args: tuple = ()) -> threading.
Thread:
    """Launch a module in a daemon thread. Logs failure but never crashes
    main."""
    def wrapper():
        try:
            log.info(f"[{name}] Starting")
            target_fn(*args)
            log.info(f"[{name}] Finished")
        except Exception as e:
            log.error(f"[{name}] CRASHED: {e} — system continues
            without it")

    t = threading.Thread(target=wrapper, name=name, daemon=True)
    t.start()
    _threads.append(t)
    _module_fns[name] = lambda: target_fn(*args)
    return t

# Boot Sequence
def main():
    log.info("=" * 70)
    log.info(" C-UAS SENTINEL v2.0 — BOOT SEQUENCE")
    log.info(f" {datetime.now().strftime('%Y-%m-%d %H:%M:%S UTC')}")
    log.info("=" * 70)

    cfg = load_config()
    state = SystemState()

# 0. Config Validation

```

```
try:
    from config_validator import ConfigValidator
    v = ConfigValidator()
    if v.load():
        ok = v.validate()
        v.print_report(quiet=True)
        if not ok:
            log.warning("[MAIN] Config has errors — continuing
                        anyway (fault-tolerant)")
except Exception as e:
    log.warning(f"[MAIN] Config validator skipped: {e}")

# 1. Health Monitor — first so we always have vitals
try:
    from system_health import HealthMonitor
    hm = HealthMonitor(cfg, state)
    safe_start("HEALTH", hm.run)
    time.sleep(0.3)
except ImportError as e:
    log.warning(f"[HEALTH] {e}")

# 2. Alert Manager
alert_mgr = None
try:
    from alert_manager import AlertManager
    alert_mgr = AlertManager(cfg, state)
    safe_start("ALERTS", alert_mgr.run)
    time.sleep(0.2)
except ImportError as e:
    log.warning(f"[ALERTS] {e}")

# 3. Data Logger
data_logger = None
try:
    from data_logger import DataLogger
    data_logger = DataLogger(cfg, state)
    state._data_logger = data_logger # Inject reference
    safe_start("LOGGER", data_logger.run)
    time.sleep(0.2)
except ImportError as e:
    log.warning(f"[LOGGER] {e}")

# 4. RF Monitor + Jammer
try:
    from rf_monitor import RFMonitor
    rf = RFMonitor(cfg, state)
    safe_start("RF", rf.run)
    time.sleep(0.5)
except ImportError as e:
```



```
log.warning(f"[RF] {e}")

# 5. Camera Tracker (YOLO + PTZ)
try:
    from camera_tracker import CameraTracker
    cam = CameraTracker(cfg, state)
    safe_start("CAMERA", cam.run)
    time.sleep(0.5)
except ImportError as e:
    log.warning(f"[CAMERA] {e}")

# 6. Motor Controller
try:
    from motor_controller import MotorController
    motors = MotorController(cfg, state)
    safe_start("MOTORS", motors.run)
    time.sleep(0.3)
except ImportError as e:
    log.warning(f"[MOTORS] {e}")

# 7. Stream Recorder
try:
    from stream_recorder import StreamRecorder
    stream_url = cfg.get('camera', {}).get('stream_url', '')
    rec = StreamRecorder(cfg, state, stream_url)
    safe_start("RECORDER", rec.run)
    time.sleep(0.3)
except ImportError as e:
    log.warning(f"[RECORDER] {e}")

# 8. Telegram Bot
try:
    from telegram_bot import TelegramBot
    bot = TelegramBot(cfg, state)
    safe_start("TELEGRAM", bot.run)
    time.sleep(0.3)
except ImportError as e:
    log.warning(f"[TELEGRAM] {e}")

# 9. Dashboard (Flask + WebSocket)
try:
    from dashboard import Dashboard
    dash = Dashboard(cfg, state)
    safe_start("DASHBOARD", dash.run)
    time.sleep(0.5)
except ImportError as e:
    log.warning(f"[DASHBOARD] {e}")

# 10. Watchdog
```

```

try:
    from watchdog_recovery import Watchdog
    wd = Watchdog(cfg, state)
    # Register all active threads
    for t in _threads:
        fn = __module__fns.get(t.name)
        if fn:
            wd.register(t.name, t, fn, max_restarts=5)
    safe_start("WATCHDOG", wd.run)
    time.sleep(0.2)
except ImportError as e:
    log.warning(f"[WATCHDOG] {e}")

# 11. CLI Interface
try:
    from cli_interface import CLIInterface
    cli = CLIInterface(cfg, state, data_logger)
    safe_start("CLI", cli.run)
except ImportError as e:
    log.warning(f"[CLI] {e}")

# Summary
alive = sum(1 for t in _threads if t.is_alive())
log.info(" " * 70)
log.info(f" BOOT COMPLETE — {alive}/{len(_threads)} modules active ")
log.info(f" Dashboard → http://0.0.0.0:{cfg.get('dashboard', {})}.get('port', 5000)}")
log.info(f" System ARMED: {state.system_armed}")
log.info(" " * 70)

# Graceful Shutdown Handler
def shutdown(sig, frame):
    log.info("[MAIN] Shutdown signal received")
    state.update(system_armed=False, jamming_active=False)
    if data_logger:
        data_logger.log_event("SYSTEM", notes="shutdown")
    time.sleep(1)
    sys.exit(0)

signal.signal(signal.SIGINT, shutdown)
signal.signal(signal.SIGTERM, shutdown)

# Main Watchdog Loop
while True:
    alive = sum(1 for t in _threads if t.is_alive())
    v = state.vitals
    temp = v.get("cpu_temp_c", 0) if v else 0
    log.debug(

```

```

        f"[MAIN] threads={alive}/{len(_threads)} | "
        f"target={'YES' if state.active_target else 'no'} | "
        f"RF={state.rf_online} | cam={state.camera_online} | "
        f"jam={state.jamming_active} | temp={temp}°C"
    )
    time.sleep(15)

if __name__ == "__main__":
    main()

```

A.2 Perception and Computer Vision Subsystem

Listing A.2: Optical Tracker and DKF (core/camera_tracker.py)

```

#!/usr/bin/env python3
import cv2
import time
import logging
import threading
import requests
import numpy as np
from datetime import datetime
from typing import Optional, Tuple
from ultralytics import YOLO

log = logging.getLogger("CAMERA")

import urllib3
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

#
# SHARED FRAME BUFFER      —   FPS
#
class FrameBuffer:
    __instance = None
    __lock = threading.Lock()

    def __init__(self, stream_url: str, fps_cap: int = 30):
        self.url = stream_url
        self.fps_cap = fps_cap
        self._frame = None
        self._frame_id = 0
        self._rlock = threading.Lock()
        self._stop = threading.Event()
        self.is_live = False

    @classmethod

```

```

def get_instance(cls, stream_url: str = "", fps_cap: int = 30) -> "
    FrameBuffer":
    with cls._lock:
        if cls._instance is None and stream_url:
            cls._instance = cls(stream_url, fps_cap)
            cls._instance.start()
        return cls._instance

def start(self):
    self._thread = threading.Thread(target=self._reader, name="
        FrameReader", daemon=True)
    self._thread.start()
    log.info("[FRAMEBUF] Shared buffer started.")

def _reader(self):
    cap = None
    while not self._stop.is_set():
        if cap is None or not cap.isOpened():
            cap = cv2.VideoCapture(self.url, cv2.CAP_FFMPEG)
            cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
            if not cap.isOpened():
                time.sleep(2); continue

        ret, frame = cap.read()
        if not ret or frame is None:
            cap.release(); cap = None; continue

        with self._rlock:
            self._frame = frame
            self._frame_id += 1
            self.is_live = True

def read_with_id(self):
    with self._rlock:
        if self._frame is None:
            return False, None, self._frame_id
        return self.is_live, self._frame.copy(), self._frame_id

def stop(self):
    self._stop.set()

#
# HIKVISION PTZ
#
class HikvisionPTZ:
    def __init__(self, ip, port, user, password):
        self.base_url = f"http://{ip}:{port}"
        self.auth = requests.auth.HTTPDigestAuth(user, password)
        self.session = requests.Session()

```

```

        self.session.auth = self.auth
        self.session.verify = False

    def continuous_move(self, pan=0, tilt=0, zoom=0):
        try:
            # ISAPI
            p_val, t_val, z_val = int(pan * 100), int(tilt * 100), int(
                zoom * 100)
            xml = (f'<?xml version="1.0" encoding="UTF-8"?><PTZData
                version="2.0" xmlns="http://www.isapi.org/ver20/XMLSchema
                ">
                    f'<pan>{p_val}</pan><tilt>{t_val}</tilt><zoom>{z_val
                    }</zoom></PTZData>')
            self.session.put(f"{self.base_url}/ISAPI/PTZCtrl/channels/1/
                continuous", data=xml, timeout=0.5)
        except Exception as e:
            log.debug(f"[PTZ] Move Error: {e}")

    def stop(self):
        self.continuous_move(0, 0, 0)

#
# DRONE DETECTOR & SERVO —
#
class DroneDetector:
    def __init__(self, model_path="models/yolov8s.pt", conf_thresh=0.30):
        self.model = YOLO(model_path)
        self.conf_thresh = conf_thresh

    def detect(self, frame):
        # GPU (CUDA)
        results = self.model.predict(frame, conf=self.conf_thresh,
            verbose=False, imgsz=640, device="cuda")
        for r in results:
            if len(r.bboxes) > 0:
                box = r.bboxes[0]
                x, y, w, h = box.xywh[0].cpu().numpy()
                return (int(x-w/2), int(y-h/2), int(w), int(h))
        return None

class PTZServo:
    def __init__(self, fw=640, fh=480):
        self.fw, self.fh = fw, fh
        self.kp = 3.5 #

    def compute(self, cx, cy):
        #
        ex = (cx - self.fw/2) / (self.fw/2)
        ey = (cy - self.fh/2) / (self.fh/2)

```

```

        #
        return np.clip(self.kp * ex, -1, 1), np.clip(-self.kp * ey, -1,
            1)

#
# MAIN TRACKER —
#
class CameraTracker:
    def __init__(self, cfg, state):
        self.cfg = cfg['camera']
        self.state = state
        self._stop = threading.Event()
        self.ptz = HikvisionPTZ(self.cfg['ip'], self.cfg['port'], self.
            cfg['username'], self.cfg['password'])
        self.detector = DroneDetector(model_path=self.cfg.get('model_path',
            'models/yolov8s.pt'))
        self.servo = PTZServo()
        self.last_ptz_cmd_t = 0

    def _patrol(self):
        """          )Swing)          """
        now = time.time()
        if now - self.last_ptz_cmd_t < 1.0: return

        self.last_ptz_cmd_t = now
        if not hasattr(self, '_patrol_start_t'): self._patrol_start_t =
            now
        cycle = (now - self._patrol_start_t) % 12

        if cycle < 6:
            self.ptz.continuous_move(pan=0.3, tilt=0.1)
        else:
            self.ptz.continuous_move(pan=-0.3, tilt=0.1)

    def run(self):
        log.info("[CAM]   Horus Sentinel Tracker — ACTIVE")
        buf = FrameBuffer.get_instance(self.cfg['stream_url'])
        last_fid = -1

        while not self._stop.is_set():
            ret, frame, fid = buf.read_with_id()
            if not ret or frame is None or fid == last_fid:
                time.sleep(0.01); continue
            last_fid = fid

        #
        h, w = frame.shape[:2]
        self.servo.fw, self.servo.fh = w, h

```

```

#
bbox = self.detector.detect(frame)

if bbox:
    #
    cx, cy = bbox[0] + bbox[2]//2, bbox[1] + bbox[3]//2
    p, t = self.servo.compute(cx, cy)
    self.ptz.continuous_move(pan=p, tilt=t)

    # Dashboard
    self.state.update(active_target={"bbox": bbox, "conf":
        0.9}, camera_online=True)
else:
    #
    self.__patrol()

# Dashboard
self.state.last_frame = frame
self.state.frame_count += 1

def stop(self):
    self.__stop.set()
    if self.ptz: self.ptz.stop()

```

A.3 Kinematic Actuation and Control Theory

Listing A.3: Kinematic PID and ISAPI Actuation (core/motor_controller.py)

```

#!/usr/bin/env python3
"""
MOTOR CONTROLLER MODULE
Controls stepper motors for 360° pan + tilt of the entire assembly
(camera + spectrum analyzer mounted on L-Bracket on KINGJOY VT-2100L)

GPIO Stepper Control (4-phase half-step) for:
- Pan motor (horizontal 360°)
- Tilt motor (vertical 0-90°)

Auto-sweep behavior:
- When idle: sweep 360° to scan for targets
- On target acquired: orient assembly toward target direction
- If target behind assembly: rapid 180° flip
"""

import time
import logging
import threading
import math

```

```

from typing import Optional

log = logging.getLogger("MOTORS")

# GPIO Setup
try:
    from gpiozero import OutputDevice
    GPIO_AVAILABLE = True
except (ImportError, RuntimeError):
    GPIO_AVAILABLE = False
    log.warning("[MOTORS] RPi.GPIO not available — simulation mode")

# Half-step sequence for 4-phase stepper motor
HALF_STEP_SEQ = [
    [1, 0, 0, 0],
    [1, 1, 0, 0],
    [0, 1, 0, 0],
    [0, 1, 1, 0],
    [0, 0, 1, 0],
    [0, 0, 1, 1],
    [0, 0, 0, 1],
    [1, 0, 0, 1],
]

class StepperMotor:
    """4-phase stepper motor controller via GPIO."""

    def __init__(self, pins: list, step_angle: float = 1.8, gear_ratio:
float = 1.0):
        self.pins = pins
        self.step_angle = step_angle          # Degrees per full step
        self.gear_ratio = gear_ratio
        self.current_pos_deg = 0.0
        self.seq_index = 0
        self._lock = threading.Lock()

        if GPIO_AVAILABLE:
            self.devices = [OutputDevice(pin) for pin in self.pins]
            for dev in self.devices:
                dev.off()

    def _set_step(self, step: list):
        if GPIO_AVAILABLE:
            for i in range(len(self.devices)):
                if step[i]:
                    self.devices[i].on()
                else:
                    self.devices[i].off()

```



```

        # else: simulated

def step(self, direction: int = 1):
    """direction: +1 = CW, -1 = CCW"""
    with self._lock:
        self.seq_index = (self.seq_index + direction) % len(
            HALF_STEP_SEQ)
        self._set_step(HALF_STEP_SEQ[self.seq_index])
        self.current_pos_deg += direction * (self.step_angle / self.
            gear_ratio / 2) # Half-step

def rotate_degrees(self, degrees: float, speed_rpm: float = 60):
    """Rotate by given degrees at speed_rpm."""
    steps_per_rev = 360 / (self.step_angle / self.gear_ratio)
    steps_needed = abs(int(degrees / (self.step_angle / self.
        gear_ratio / 2)))
    direction = 1 if degrees > 0 else -1

    # Step delay from RPM
    step_delay = 60 / (speed_rpm * steps_per_rev * 2) # Half-step
    step_delay = max(step_delay, 0.001) # Min 1ms

    for _ in range(steps_needed):
        self.step(direction)
        time.sleep(step_delay)

def go_to_angle(self, target_deg: float, speed_rpm: float = 60):
    """Rotate to absolute angle."""
    current = self.current_pos_deg % 360
    diff = (target_deg - current + 180) % 360 - 180 # Shortest path
    self.rotate_degrees(diff, speed_rpm)

def release(self):
    """De-energize motor."""
    if GPIO_AVAILABLE:
        for dev in self.devices:
            dev.off()

# Motor Controller Main Class
class MotorController:
    def __init__(self, cfg: dict, state):
        self.cfg = cfg.get('motors', {})
        self.state = state
        self._stop = threading.Event()

        pan_pins = self.cfg.get('pan_gpio_pins', [17, 18, 27, 22])
        tilt_pins = self.cfg.get('tilt_gpio_pins', [23, 24, 25, 8])
        step_angle = self.cfg.get('pan_step_angle', 1.8)

```

```

speed_rpm = self.cfg.get('speed_rpm', 60)

self.pan_motor = StepperMotor(pan_pins, step_angle)
self.tilt_motor = StepperMotor(tilt_pins, step_angle)
self.speed_rpm = speed_rpm
self.auto_sweep = self.cfg.get('auto_sweep_on_idle', True)
self.sweep_angle = 45 # Degrees per sweep step
self.last_target_angle = None
self.idle_timer = time.time()
self.sim_mode = not GPIO_AVAILABLE

def run(self):
    log.info(f"[MOTORS] Starting | GPIO={'ON' if GPIO_AVAILABLE else 'SIM'}")
    self.state.update(motors_online=True)

    # Home position
    log.info(f"[MOTORS] Homing to 0°...")
    if not self.sim_mode:
        self.pan_motor.go_to_angle(0, self.speed_rpm)
        self.tilt_motor.go_to_angle(0, self.speed_rpm)

    sweep_pos = 0
    sweep_dir = 1

    while not self._stop.is_set():
        try:
            target = self.state.active_target

            if target:
                # Target acquired: orient toward it
                self.idle_timer = time.time()
                # If we have pan angle info (from triangulation or manual)
                target_pan = target.get('pan_deg')
                target_tilt = target.get('tilt_deg', 0)

                if target_pan is not None:
                    current_pan = self.pan_motor.current_pos_deg % 360
                    diff = abs(target_pan - current_pan)

                    # If target behind assembly (>90° off) → rapid flip
                    if diff > 90:
                        log.info(f"[MOTORS] Target behind! Rotating {diff:.0f}°")
                        self.pan_motor.go_to_angle(target_pan, self.speed_rpm * 1.5)

```

```

        else:
            # Fine adjustment
            self.pan_motor.go_to_angle(target_pan, self.
                                      speed_rpm)

            if target_tilt != 0:
                tilt_clamped = max(0, min(90, target_tilt))
                self.tilt_motor.go_to_angle(tilt_clamped,
                                           self.speed_rpm)

            self.last_target_angle = target_pan
            time.sleep(0.2)

    else:
        # Idle: auto-sweep 360°
        idle_sec = time.time() - self.idle_timer
        if self.auto_sweep and idle_sec > 5:
            sweep_pos += self.sweep_angle * sweep_dir
            if sweep_pos >= 360:
                sweep_pos = 0
                sweep_dir = 1
            elif sweep_pos <= 0:
                sweep_dir = 1
                sweep_pos = 0

            if not self.sim_mode:
                self.pan_motor.go_to_angle(sweep_pos, self.
                                          speed_rpm // 2)
            else:
                log.debug(f"[MOTORS-SIM] Sweep pan={sweep_pos}°")
            time.sleep(1.0)
        else:
            time.sleep(0.5)

    except Exception as e:
        log.error(f"[MOTORS] Error: {e}")
        time.sleep(1.0)

# Release on stop
self.pan_motor.release()
self.tilt_motor.release()
if GPIO_AVAILABLE:
    GPIO.cleanup()
self.state.update(motors_online=False)
log.info(f"[MOTORS] Stopped")

def stop(self):
    self._stop.set()

```

A.4 RF Sensing and Network Operations

Listing A.4: *Network Auto-Discovery* (utils/network_scanner.py)

```
#!/usr/bin/env python3
"""
NETWORK SCANNER MODULE
Auto-discovers Hikvision cameras and other devices on LAN.

Features:
- ARP scan for all devices on subnet
- Hikvision SADP protocol discovery (UDP 37020)
- ONVIF WS-Discovery
- Port scan for RTSP (554) and HTTP (80/8080)
- Auto-update config.yaml with found camera IP

Usage:
python3 network_scanner.py          # Scan and print results
python3 network_scanner.py --update  # Update config.yaml with first
    found camera
python3 network_scanner.py --subnet 192.168.0.0/24
"""

import socket
import struct
import time
import logging
import threading
import ipaddress
import subprocess
import argparse
import yaml
from typing import List, Optional
from pathlib import Path

log = logging.getLogger("NETSCAN")

GREEN = "\033[32m"; YELLOW = "\033[33m"; CYAN = "\033[36m"; RST = "\033[0
    m"; BOLD = "\033[1m"

class DeviceInfo:
    def __init__(self, ip: str):
        self.ip = ip
        self.mac = ""
        self.vendor = ""
```

```

        self.ports = []
        self.is_hikvision = False
        self.rtsp_url = ""
        self.model = ""

    def __str__(self):
        tag = f" {GREEN}[HIKVISION]{RST}" if self.is_hikvision else ""
        rtsp = f"\n    RTSP: {CYAN}{self.rtsp_url}{RST}" if self.rtsp_url
            else ""
        ports_str = ", ".join(str(p) for p in self.ports) if self.ports
            else ""
        return (f" {BOLD}{self.ip}{RST}{tag}\n"
                f"    MAC: {self.mac}  Vendor: {self.vendor}\n"
                f"    Open ports: {ports_str}{rtsp}")

# Hikvision SADP Discovery
SADP_DISCOVER_MSG = bytes.fromhex(
    "484b000000000000000000000000000000000000000000000000000000000000"
    "0000000000000000000000000000000000000000000000000000000000000000"
)

def sadp_discover(timeout: float = 3.0) -> List[dict]:
    """
    Broadcast Hikvision SADP discovery packet on UDP 37020.
    Returns list of found device dicts.
    """
    devices = []
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
        sock.settimeout(timeout)
        sock.bind(("", 37020))
        sock.sendto(SADP_DISCOVER_MSG, ("255.255.255.255", 37020))

        deadline = time.time() + timeout
        while time.time() < deadline:
            try:
                data, addr = sock.recvfrom(4096)
                if data and addr[0] not in [d["ip"] for d in devices]:
                    dev = {
                        "ip": addr[0],
                        "protocol": "SADP",
                        "raw": data.hex()[:40]
                    }
                # Try to extract device type from response
                try:
                    txt = data.decode("utf-8", errors="ignore")

```

```

        dev["info"] = txt[:100]
    except Exception:
        pass
    devices.append(dev)
    log.info(f"[SADP] Found device: {addr[0]}")
except socket.timeout:
    break
except Exception as e:
    log.debug(f"[SADP] Error: {e}")
finally:
    try:
        sock.close()
    except Exception:
        pass
return devices

# Port Scanner
CAMERA_PORTS = [80, 443, 554, 8080, 8000, 8443, 37777]

def scan_port(ip: str, port: int, timeout: float = 0.5) -> bool:
    try:
        with socket.create_connection((ip, port), timeout=timeout):
            return True
    except Exception:
        return False

def scan_host(ip: str) -> DeviceInfo:
    dev = DeviceInfo(ip)
    open_ports = []

    # Scan camera-relevant ports in parallel
    results = {}
    threads = []

    def check(p):
        results[p] = scan_port(ip, p)

    for port in CAMERA_PORTS:
        t = threading.Thread(target=check, args=(port,))
        threads.append(t)
        t.start()
    for t in threads:
        t.join(timeout=1.5)

    dev.ports = [p for p in CAMERA_PORTS if results.get(p)]

```

```

# Hikvision heuristics
if 8000 in dev.ports or 554 in dev.ports:
    dev.is_hikvision = True # Very likely
if 80 in dev.ports:
    try:
        import urllib.request
        resp = urllib.request.urlopen(
            f"http://{ip}/ISAPI/System/deviceInfo",
            timeout=2
        )
        body = resp.read().decode("utf-8", errors="ignore")
        if "Hikvision" in body or "HikvisionDS" in body or "IPC" in
            body:
            dev.is_hikvision = True
            import re
            m = re.search(r'<model>([^\<]+)</model>', body)
            if m:
                dev.model = m.group(1)
    except Exception:
        pass

if dev.is_hikvision and 554 in dev.ports:
    dev.rtsp_url = f"rtsp://admin:YOURPASSWORD@{ip}:554/Streaming/
        Channels/101"

return dev

# ARP Ping Sweep
def get_local_subnet() -> str:
    """Detect local subnet (e.g. 192.168.1.0/24)."""
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        local_ip = s.getsockname()[0]
        s.close()
        # Assume /24
        parts = local_ip.rsplit(".", 1)
        return f"{parts[0]}.0/24"
    except Exception:
        return "192.168.1.0/24"

def arp_scan(subnet: str) -> List[str]:
    """
    Fast host discovery via arp-scan or ping sweep fallback.
    Returns list of live IP addresses.
    """
    live = []

```

```

# Try arp-scan first (fastest)
try:
    result = subprocess.run(
        ["arp-scan", "--localnet", "--quiet"],
        capture_output=True, text=True, timeout=15
    )
    for line in result.stdout.splitlines():
        parts = line.split()
        if parts and re.match(r'\d+\.\d+\.\d+\.\d+', parts[0]):
            live.append(parts[0])
    if live:
        return live
except (FileNotFoundError, subprocess.TimeoutExpired):
    pass

# Fallback: parallel ping sweep
import re
net = ipaddress.IPv4Network(subnet, strict=False)
hosts = list(net.hosts())
log.info(f"[SCAN] Pinging {len(hosts)} hosts in {subnet}...")

found_lock = threading.Lock()
sem = threading.Semaphore(50) # Max 50 concurrent pings

def ping_host(ip_str):
    with sem:
        try:
            r = subprocess.run(
                ["ping", "-c", "1", "-W", "1", ip_str],
                stdout=subprocess.DEVNULL,
                stderr=subprocess.DEVNULL,
                timeout=2
            )
            if r.returncode == 0:
                with found_lock:
                    live.append(ip_str)
        except Exception:
            pass

threads = [threading.Thread(target=ping_host, args=(str(h),)) for h
            in hosts]
for t in threads:
    t.start()
for t in threads:
    t.join(timeout=5)

return live

```



```

# Main Scanner
class NetworkScanner:
    def __init__(self, subnet: str = ""):
        self.subnet = subnet or get_local_subnet()
        self.devices: List[DeviceInfo] = []
        self.cameras: List[DeviceInfo] = []

    def scan(self):
        print(f"\n{BOLD}Scanning subnet: {self.subnet}{RST}")

        # 1. SADP discovery (fastest for Hikvision)
        print(f" {CYAN→} Hikvision SADP broadcast...{RST}", end=" ",
              flush=True)
        sadp_found = sadp_discover(timeout=3.0)
        sadp_ips = {d["ip"] for d in sadp_found}
        print(f"{GREEN}{len(sadp_found)} found{RST}")

        # 2. ARP/ping sweep
        print(f" {CYAN→} Host discovery (arp/ping)...{RST}", end=" ",
              flush=True)
        live_ips = arp_scan(self.subnet)
        print(f"{GREEN}{len(live_ips)} hosts alive{RST}")

        # Merge
        all_ips = list(set(live_ips) | sadp_ips)
        print(f" {CYAN→} Port scanning {len(all_ips)} hosts...{RST}",
              end=" ", flush=True)

        # 3. Port scan each host (parallel)
        scan_lock = threading.Lock()
        self.devices = []

        def scan_one(ip):
            dev = scan_host(ip)
            with scan_lock:
                self.devices.append(dev)

        threads = [threading.Thread(target=scan_one, args=(ip,)) for ip
                   in all_ips]
        for t in threads:
            t.start()
        for t in threads:
            t.join(timeout=10)

        self.cameras = [d for d in self.devices if d.is_hikvision or 554
                        in d.ports]
        print(f"{GREEN}done{RST}")

```

```

def print_results(self):
    print(f"\n {' '*60}")
    print(f"    {BOLD}Scan Results{RST}")
    print(f" {' '*60}")

    if not self.devices:
        print(f"    {YELLOW}No devices found{RST}")
        return

    print(f"\n    {BOLD}All devices ({len(self.devices)}):{RST}")
    for d in sorted(self.devices, key=lambda x: x.ip):
        print(d)

    print(f"\n    {BOLD}Cameras found ({len(self.cameras)}):{RST}")
    if self.cameras:
        for c in self.cameras:
            print(c)
    else:
        print(f"    {YELLOW}No cameras detected{RST}")
    print()

def update_config(self, cfg_path: str = "config.yaml"):
    """Update config.yaml with the first Hikvision camera found."""
    if not self.cameras:
        print(f"{YELLOW}No camera to update config with{RST}")
        return False

    cam = self.cameras[0]
    cfg_file = Path(cfg_path)
    if not cfg_file.exists():
        print(f"{YELLOW}Config not found: {cfg_path}{RST}")
        return False

    with open(cfg_file) as f:
        cfg = yaml.safe_load(f)

    old_ip = cfg.get("camera", {}).get("ip", "")
    cfg.setdefault("camera", {})[ "ip" ] = cam.ip

    # Update stream URL too
    user = cfg[ "camera" ].get("username", "admin")
    pwd = cfg[ "camera" ].get("password", "Admin1234")
    cfg[ "camera" ][ "stream_url" ] = (
        f"rtsp://{user}:{pwd}@{cam.ip}:554/Streaming/Channels/101"
    )

    with open(cfg_file, "w") as f:
        yaml.dump(cfg, f, default_flow_style=False, sort_keys=False)

```

```

        print(f"{GREEN} Config updated: camera IP {old_ip} → {cam.ip}{{
            RST}}")
        return True

def get_first_camera_ip(self) -> Optional[str]:
    return self.cameras[0].ip if self.cameras else None

if __name__ == "__main__":
    import re
    parser = argparse.ArgumentParser(description="C-UAS Network Scanner")
    parser.add_argument("--subnet", default="", help="e.g. 192.168.1.0/24
        ")
    parser.add_argument("--update", action="store_true",
        help="Update config.yaml with found camera IP")
    parser.add_argument("--config", default="config.yaml")
    args = parser.parse_args()

    scanner = NetworkScanner(subnet=args.subnet)
    scanner.scan()
    scanner.print_results()

    if args.update:
        scanner.update_config(args.config)

```

A.5 Command & Control (C2) and Data Logging

Listing A.5: *Tactical HMI Server (core/dashboard.py)*

```

#!/usr/bin/env python3
"""
DASHBOARD MODULE — C-UAS SENTINEL v2.0
Military-grade live command interface.

FIX: __gen_frames() now reads stream_url from config directly (not from
    state.vitals)—
    that was why the camera feed was always black.
"""

import time
import logging
import threading
import numpy as np
from datetime import datetime

log = logging.getLogger("DASHBOARD")

try:

```

```

from flask import Flask, render_template_string, jsonify, Response,
    request
from flask_socketio import SocketIO
FLASK_OK = True
except ImportError:
    FLASK_OK = False
    log.warning("[DASH] Flask/SocketIO not installed")

try:
    import cv2
    CV2_OK = True
except ImportError:
    CV2_OK = False

# Offline placeholder frame (shown when camera unreachable)
def _make_offline_frame(w: int = 640, h: int = 480) -> bytes:
    """Generate a dark frame with 'FEED OFFLINE' text for when camera is
    down."""
    if not CV2_OK:
        return b''
    frame = np.zeros((h, w, 3), dtype=np.uint8)
    # Grid lines
    for x in range(0, w, 80):
        cv2.line(frame, (x, 0), (x, h), (0, 30, 0), 1)
    for y in range(0, h, 60):
        cv2.line(frame, (0, y), (w, y), (0, 30, 0), 1)
    # Center crosshair
    cx, cy = w // 2, h // 2
    cv2.line(frame, (cx - 30, cy), (cx + 30, cy), (0, 120, 0), 1)
    cv2.line(frame, (cx, cy - 30), (cx, cy + 30), (0, 120, 0), 1)
    cv2.circle(frame, (cx, cy), 40, (0, 80, 0), 1)
    # Text
    cv2.putText(frame, "OPTICAL FEED OFFLINE",
                (w//2 - 165, h//2 - 20),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 180, 0), 2)
    cv2.putText(frame, "Connecting to stream...",
                (w//2 - 140, h//2 + 20),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 100, 0), 1)
    ts = datetime.now().strftime("%H:%M:%S")
    cv2.putText(frame, ts, (10, h - 15),
                cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 100, 0), 1)
    cv2.putText(frame, "C-UAS SENTINEL v2.0",
                (w - 200, h - 15),
                cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 100, 0), 1)
    __, buf = cv2.imencode('.jpg', frame, [cv2.IMWRITE_JPEG_QUALITY, 70])
    return buf.tobytes()

```

```

HTML_PAGE = r"""<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>C-UAS SENTINEL — COMMAND INTERFACE</title>
<script src="https://cdnjs.cloudflare.com/ajax/libs/socket.io/4.6.1/
    socket.io.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/Chart.js/4.4.1/chart.
    umd.min.js"></script>
<style>
    :root {
        --bg:          #050a05;
        --bg2:         #0a120a;
        --bg3:         #0d1a0d;
        --panel:       #0f1f0f;
        --border:      #1a3a1a;
        --green:       #00ff41;
        --green2:      #00cc33;
        --green3:      #008822;
        --amber:       #ffaa00;
        --red:         #ff2200;
        --cyan:        #00ffff;
        --text:        #b0ffb0;
        --text-dim:    #508850;
        --font:        'Courier New', monospace;
    }
    * { box-sizing:border-box; margin:0; padding:0; }
    body {
        background:var(--bg); color:var(--green);
        font-family:var(--font); min-height:100vh; overflow-x:hidden;
        background-image: repeating-linear-gradient(
            transparent, transparent 2px,
            rgba(0,255,65,0.012) 2px, rgba(0,255,65,0.012) 4px
        );
    }
    header {
        background:var(--panel); border-bottom:1px solid var(--border);
        padding:8px 20px; display:flex; align-items:center; justify-content:
            space-between;
        box-shadow:0 0 20px rgba(0,255,65,0.08);
    }
    .logo { font-size:1.3em; font-weight:bold; letter-spacing:4px;
        color:var(--green); text-shadow:0 0 12px var(--green); }
    .logo span { color:var(--amber); }
    .hdr-center { font-size:0.72em; color:var(--text-dim); letter-spacing:2
        px; }
    .clock { font-size:1.05em; color:var(--cyan); letter-spacing:2px; }
    .status-bar {

```

```

    background:var(--bg2); border-bottom:1px solid var(--border);
    padding:5px 20px; display:flex; gap:24px; align-items:center;
  }
  .indicator { display:flex; align-items:center; gap:6px;
               font-size:0.78em; letter-spacing:1px; text-transform:
               uppercase; }
  .dot { width:9px; height:9px; border-radius:50%; background:var(--text-
         dim); transition:.3s; }
  .dot.online { background:var(--green); box-shadow:0 0 8px var(--green);
               }
  .dot.warn    { background:var(--amber); box-shadow:0 0 8px var(--amber);
               }
  .dot.alert   { background:var(--red);    box-shadow:0 0 8px var(--red);
               animation:blink .5s infinite; }
  @keyframes blink { 0%,100%{opacity:1} 50%{opacity:.2} }

  .fps-display {
    margin-left:auto; font-size:0.82em; color:var(--text-dim); letter-
    spacing:1px;
  }
  .fps-display span { color:var(--cyan); font-weight:bold; }

  .grid {
    display:grid;
    grid-template-columns: 1fr 380px;
    grid-template-rows: 1fr auto;
    gap:10px; padding:10px;
    height:calc(100vh - 94px);
  }

  /* LEFT COLUMN */
  .left-col {
    display:grid;
    grid-template-rows: 1fr 220px;
    gap:10px;
  }

  /* RIGHT COLUMN */
  .right-col {
    display:grid;
    grid-template-rows: 260px 200px 1fr;
    gap:10px;
  }

  .panel {
    background:var(--panel); border:1px solid var(--border);
    border-radius:2px; padding:12px; position:relative; overflow:hidden;
  }
  .panel::before {

```

```

        content:''; position:absolute; top:0; left:0; right:0; height:2px;
        background:linear-gradient(90deg, transparent, var(--green3),
            transparent);
    }
    .panel-title {
        font-size:0.68em; letter-spacing:3px; color:var(--text-dim);
        margin-bottom:10px; border-bottom:1px solid var(--border); padding-
            bottom:6px;
        display:flex; align-items:center; gap:8px;
    }
    .panel-title span { color:var(--green); }

/* CAMERA */
.cam-wrap {
    width:100%; height:calc(100% - 30px);
    background:#000; border:1px solid var(--border);
    position:relative; overflow:hidden;
}
.cam-wrap img {
    width:100%; height:100%; object-fit:contain;
    display:block;
}
/* Tactical overlay on camera */
.cam-overlay {
    position:absolute; top:0; left:0; width:100%; height:100%;
    pointer-events:none;
}
.corner { position:absolute; width:20px; height:20px; }
.corner.tl { top:8px; left:8px; border-top:2px solid var(--green);
    border-left:2px solid var(--green); }
.corner.tr { top:8px; right:8px; border-top:2px solid var(--green);
    border-right:2px solid var(--green); }
.corner.bl { bottom:8px; left:8px; border-bottom:2px solid var(--green)
    ); border-left:2px solid var(--green); }
.corner.br { bottom:8px; right:8px; border-bottom:2px solid var(--green)
    ); border-right:2px solid var(--green); }
.crosshair-v { position:absolute; top:50%; left:50%; transform:
    translate(-50%,-50%);
        width:1px; height:50px; background:rgba(0,255,65,.4); }
.crosshair-h { position:absolute; top:50%; left:50%; transform:
    translate(-50%,-50%);
        width:50px; height:1px; background:rgba(0,255,65,.4); }
.crosshair-ring { position:absolute; top:50%; left:50%; transform:
    translate(-50%,-50%);
        width:60px; height:60px; border-radius:50%;
        border:1px solid rgba(0,255,65,.3); }

.target-badge {
    position:absolute; bottom:10px; left:10px;

```

```

    background:rgba(0,0,0,.75); border:1px solid var(--amber);
    padding:5px 10px; font-size:0.75em; display:none;
}
.target-badge.visible { display:block; }
.target-badge span { color:var(--amber); }

/* BOTTOM LEFT — bottom panel split */
.bottom-left {
    display:grid; grid-template-columns:1fr 1fr; gap:10px;
}

/* LOG */
.log-list {
    height:calc(100% - 30px); overflow-y:auto; font-size:0.75em;
}
.log-entry { padding:3px 0; border-bottom:1px solid rgba(0,255,65,.07);
    }
.log-entry .ts { color:var(--text-dim); }
.log-entry.warn .ts { color:var(--amber); }
.log-entry.err .ts { color:var(--red); }

/* CONTROL */
.btn-grid { display:grid; grid-template-columns:1fr 1fr; gap:7px;
    margin-top:8px; }
.btn {
    padding:9px 6px; border:1px solid var(--green3);
    background:transparent; color:var(--green); font-family:var(--font);
    font-size:0.75em; letter-spacing:1px; text-transform:uppercase;
    cursor:pointer; border-radius:1px; transition:.15s;
}
.btn:hover { background:var(--green3); color:#fff; box-shadow:0 0 8px
    var(--green3); }
.btn.danger { border-color:var(--red); color:var(--red); }
.btn.danger:hover { background:var(--red); color:#fff; }
.btn.active-arm { background:var(--green3); color:#fff; }
.btn.jam-live { background:var(--red); border-color:var(--red); color:#
    fff; animation:blink .5s infinite; }

/* RF CHART */
.rf-chart-wrap { height:calc(100% - 30px); }

/* VITALS */
.vital-row {
    display:flex; justify-content:space-between; align-items:center;
    padding:5px 0; border-bottom:1px solid var(--border); font-size:0.82
    em;
}
.vital-bar-wrap { width:120px; height:7px; background:var(--bg3);
    border-radius:1px; overflow:hidden; }

```



```

.vital-bar { height:100%; background:var(--green2); border-radius:1px;
  transition:width .4s; }
.vital-bar.warn { background:var(--amber); }
.vital-bar.crit { background:var(--red); }

/* THREAT */
#threat-info { font-size:0.8em; color:var(--text-dim); margin-top:6px;
  line-height:1.7; }

/* ALERT BANNER */
#alert-banner {
  display:none; position:fixed; top:54px; left:50%; transform:
    translateX(-50%);
  background:rgba(200,0,0,.92); color:#fff; border:1px solid var(--red)
    ;
  padding:8px 30px; font-size:1em; letter-spacing:2px; z-index:9999;
  border-radius:2px; box-shadow:0 0 30px var(--red); animation:blink .7
    s infinite;
}

::-webkit-scrollbar { width:4px; }
::-webkit-scrollbar-track { background:var(--bg2); }
::-webkit-scrollbar-thumb { background:var(--green3); }
</style>
</head>
<body>

<div id="alert-banner"> DRONE DETECTED — COUNTERMEASURES ENGAGED </div>

<header>
  <div class="logo">C-UAS <span>SENTINEL</span></div>
  <div class="hdr-center">HORUS UNIVERSITY · ECE DEPT · GRADUATION
    PROJECT 2026</div>
  <div class="clock" id="clock">00:00:00 UTC</div>
</header>

<div class="status-bar">
  <div class="indicator"><div class="dot" id="dot-rf"></div>RF SENSOR</div>
  <div class="indicator"><div class="dot" id="dot-cam"></div>OPTICAL</div>
  <div class="indicator"><div class="dot" id="dot-motors"></div>MOTORS</div>
  <div class="indicator"><div class="dot" id="dot-jam"></div>JAMMER</div>
  <div class="fps-display">
    FPS: <span id="fps-val">--</span> | UPTIME: <span id="uptime-val">
      >--</span>s
  </div>
</div>

```

```

<div class="grid">

  <div class="left-col">

    <div class="panel">
      <div class="panel-title"> <span>OPTICAL TRACKING</span> — DS-2
        DE4225IW-DE | 25X ZOOM | DARKFIGHTER</div>
      <div class="cam-wrap">
        
        <div class="cam-overlay">
          <div class="corner tl"></div>
          <div class="corner tr"></div>
          <div class="corner bl"></div>
          <div class="corner br"></div>
          <div class="crosshair-v"></div>
          <div class="crosshair-h"></div>
          <div class="crosshair-ring"></div>
        </div>
        <div class="target-badge" id="target-badge">
          TARGET: <span id="tgt-source">--</span> |
          <span id="tgt-freq">--</span> MHz |
          <span id="tgt-pwr">--</span> dBm
        </div>
      </div>
    </div>

    <div class="bottom-left">

      <div class="panel">
        <div class="panel-title"> <span>DETECTION LOG</span></div>
        <div class="log-list" id="log-list"></div>
      </div>

      <div class="panel">
        <div class="panel-title"> <span>CONTROL PANEL</span></div>
        <div class="btn-grid">
          <button class="btn active-arm" id="btn-arm" onclick="sendCmd
            ( 'arm' )">ARM SYSTEM</button>
          <button class="btn danger" id="btn-disarm" onclick="sendCmd
            ( 'disarm' )">DISARM</button>
          <button class="btn" id="btn-jam" onclick="sendCmd
            ( 'jam_toggle' )">TOGGLE JAM</button>
          <button class="btn" onclick="sendCmd
            ( 'sweep' )">360° SWEEP</button>
          <button class="btn" onclick="sendCmd
            ( 'zoom_in' )">ZOOM IN </button>
          <button class="btn" onclick="sendCmd
            ( 'zoom_out' )">ZOOM OUT </button>

```

```

        </div>
    </div>

    </div>
</div>

<div class="right-col">

    <div class="panel">
        <div class="panel-title"> <span>RF SPECTRUM</span> — HTOOL-SA 6G |
            2.4 / 5.8 GHz</div>
        <div class="rf-chart-wrap">
            <canvas id="rf-chart"></canvas>
        </div>
    </div>

    <div class="panel">
        <div class="panel-title"> <span>SYSTEM VITALS</span></div>
        <div class="vital-row">
            <span>CPU</span>
            <div class="vital-bar-wrap"><div class="vital-bar" id="bar-cpu"
                style="width:0"></div></div>
            <span id="val-cpu">--%</span>
        </div>
        <div class="vital-row">
            <span>RAM</span>
            <div class="vital-bar-wrap"><div class="vital-bar" id="bar-ram"
                style="width:0"></div></div>
            <span id="val-ram">--%</span>
        </div>
        <div class="vital-row">
            <span>TEMP</span>
            <div class="vital-bar-wrap"><div class="vital-bar" id="bar-temp"
                style="width:0"></div></div>
            <span id="val-temp">--°C</span>
        </div>
        <div class="vital-row">
            <span>DISK</span>
            <div class="vital-bar-wrap"><div class="vital-bar" id="bar-disk"
                style="width:0"></div></div>
            <span id="val-disk">--%</span>
        </div>
        <div style="margin-top:7px; font-size:0.72em; color:var(--text-dim)">
            NET ↑<span id="val-netsent">0</span>MB ↓<span id="val-netrecv"
                >0</span>MB
        </div>
    </div>
</div>

```

```

    <div class="panel">
      <div class="panel-title"> <span>THREAT ANALYSIS</span></div>
      <div id="threat-info">NO ACTIVE THREATS</div>
    </div>

  </div>
</div>

<script>
const socket = io();

// Clock
setInterval(() => {
  document.getElementById('clock').textContent =
    new Date().toUTCString().split(' ')[4] + ' UTC';
}, 1000);

// RF Chart
const rfCtx = document.getElementById('rf-chart').getContext('2d');
const RF_LABELS = Array.from({length:40}, (_,i) => (2400 + i*21).toFixed(0));
const rfData = Array(40).fill(-96);
const rfChart = new Chart(rfCtx, {
  type: 'line',
  data: {
    labels: RF_LABELS,
    datasets: [{
      data: rfData,
      borderColor: '#00ff41',
      backgroundColor: 'rgba(0,255,65,0.06)',
      borderWidth: 1.5,
      fill: true,
      tension: 0.35,
      pointRadius: 0,
    }]
  },
  options: {
    responsive: true, maintainAspectRatio: false,
    animation: { duration: 80 },
    scales: {
      x: { ticks:{ color:'#508850', font:{ size:8}}, grid:{ color:'rgba(0,255,65,0.05)'} },
      y: { min:-105, max:-40,
        ticks:{ color:'#508850', font:{ size:8}},
        grid:{ color:'rgba(0,255,65,0.05)'} }
    },
    plugins: { legend:{ display:false } }
  }
});

```

```

// State Updates
socket.on('state_update', function(d) {
  // Dots
  setDot('dot-rf', d.rf_online);
  setDot('dot-cam', d.camera_online);
  setDot('dot-motors', d.motors_online);
  setDot('dot-jam', d.jamming_active, true);

  // Header stats
  document.getElementById('fps-val').textContent = d.fps;
  document.getElementById('uptime-val').textContent = d.uptime_sec;

  // Vitals
  const v = d.vitals || {};
  setVital('cpu', v.cpu_percent || 0, 75, 90);
  setVital('ram', v.ram_percent || 0, 80, 90);
  const tempPct = ((v.cpu_temp_c || 0) / 90 * 100);
  setVital('temp', tempPct, 72, 88);
  document.getElementById('val-temp').textContent = (v.cpu_temp_c || 0) .
    toFixed(1) + '°C';
  setVital('disk', v.disk_percent || 0, 80, 90);
  document.getElementById('val-netsent').textContent = (v.net_sent_mb || 0)
    .toFixed(1);
  document.getElementById('val-netrecv').textContent = (v.net_recv_mb || 0)
    .toFixed(1);

  // Target + alert
  const tgt = d.active_target;
  const badge = document.getElementById('target-badge');
  const banner = document.getElementById('alert-banner');
  if (tgt) {
    badge.classList.add('visible');
    document.getElementById('tgt-source').textContent = tgt.source ||
      '--';
    document.getElementById('tgt-freq').textContent = tgt.freq_mhz ?
      tgt.freq_mhz.toFixed(1) : '--';
    document.getElementById('tgt-pwr').textContent = tgt.power_dbm ?
      tgt.power_dbm.toFixed(1) : '--';
    banner.style.display = 'block';
    const rng = tgt.range_m ? '\nRange: ~${tgt.range_m.toFixed(0)}m' :
      '';
    document.getElementById('threat-info').innerHTML =
      '<b style="color:var(--amber)"> ACTIVE THREAT</b><br>' +
      'Source: ${tgt.source}<br>' +
      'Freq: ${tgt.freq_mhz || 'N/A'} MHz<br>' +
      'Power: ${tgt.power_dbm || 'N/A'} dBm<br>' +
      'Conf: ${((tgt.confidence || 0) * 100).toFixed(0)}%' +
      (tgt.range_m ? '<br>Range: ~${tgt.range_m.toFixed(0)}m' : '');
  }

```

```

    } else {
      badge.classList.remove('visible');
      banner.style.display = 'none';
      document.getElementById('threat-info').innerHTML = '<span style="
        color:var(--green2)"> NO ACTIVE THREATS</span>';
    }

    // Jam button
    const jb = document.getElementById('btn-jam');
    if (d.jamming_active) { jb.classList.add('jam-live'); jb.textContent =
      'JAMMING ACTIVE'; }
    else { jb.classList.remove('jam-live'); jb.textContent
      = 'TOGGLE JAM'; }

    // RF spectrum update
    if (d.rf_detections && d.rf_detections.length > 0) {
      const last = d.rf_detections[d.rf_detections.length - 1];
      const idx = Math.floor((last.freq_mhz - 2400) / 21);
      if (idx >= 0 && idx < rfData.length) {
        rfData[idx] = last.power_dbm;
        rfChart.update('none');
      }
    }
  });

socket.on('log_entry', function(e) {
  const list = document.getElementById('log-list');
  const div = document.createElement('div');
  const lvl = (e.level || '').toUpperCase();
  div.className = 'log-entry' + (lvl==='ERROR' || lvl==='CRITICAL' ? ' err'
    : lvl==='WARNING' ? ' warn' : '');
  div.innerHTML = '<span class="ts">[${e.time}]</span> ${e.message}
    || ''';
  list.insertBefore(div, list.firstChild);
  if (list.children.length > 60) list.removeChild(list.lastChild);
});

function setDot(id, online, isAlert=false) {
  const el = document.getElementById(id);
  if (!el) return;
  el.className = 'dot' + (online ? (isAlert ? 'alert' : 'online') : '');
}

function setVital(id, pct, warnAt, critAt) {
  const bar = document.getElementById('bar-'+id);
  const val = document.getElementById('val-'+id);
  if (!bar || !val) return;
  bar.style.width = Math.min(pct,100) + '%';
  bar.className = 'vital-bar' + (pct>=critAt?' crit':pct>=warnAt?' warn
    ':'');
}

```

```

    if (id !== 'temp') val.textContent = Math.round(pct) + '%';
  }
  function sendCmd(cmd) {
    fetch('/api/command', {
      method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({command: cmd})
    });
  }
</script>
</body>
</html>"""

class Dashboard:
    def __init__(self, cfg: dict, state):
        self.cfg = cfg.get('dashboard', {})
        self.state = state
        self._stop = threading.Event()
        self.host = self.cfg.get('host', '0.0.0.0')
        self.port = self.cfg.get('port', 5000)
        self.stream_url = cfg.get('camera', {}).get('stream_url', '')
        self.jpeg_quality = cfg.get('camera', {}).get('jpeg_quality', 70)
        self.app = None
        self.sio = None

    def run(self):
        if not FLASK_OK:
            log.warning("[DASH] Flask not available — dashboard disabled")
            return

        self.app = Flask(__name__)
        self.app.config['SECRET_KEY'] = self.cfg.get('secret_key', 'c_uas')
        self.sio = SocketIO(self.app, cors_allowed_origins="*",
                             async_mode='threading', logger=False,
                             engineio_logger=False)

        @self.app.route('/')
        def index():
            return HTML_PAGE

        @self.app.route('/api/status')
        def api_status():
            return jsonify(self.state.to_dict())

        @self.app.route('/api/command', methods=['POST'])
        def api_command():
            data = request.get_json() or {}

```

```

        cmd = data.get('command', '')
        self._handle_command(cmd)
        return jsonify({"ok": True, "command": cmd})

    @self.app.route('/video_feed')
    def video_feed():
        return Response(
            self._gen_frames(),
            mimetype='multipart/x-mixed-replace; boundary=frame'
        )

    # WebSocket state push
    def push_loop():
        while not self._stop.is_set():
            try:
                self.sio.emit('state_update', self.state.to_dict())
            except Exception:
                pass
            time.sleep(0.5)

    threading.Thread(target=push_loop, daemon=True).start()

    log.info(f"[DASH] http://{self.host}:{self.port} | stream={self.
stream_url[:40]}...")
    self.sio.run(self.app, host=self.host, port=self.port,
                debug=False, use_reloader=False,
                log_output=False, allow_unsafe_werkzeug=True)

    def _gen_frames(self):
        if not CV2_OK:
            log.warning("[DASH] cv2 not available — no video feed")
            return

        offline_frame = _make_offline_frame()
        last_good_frame = None
        jpeg_q = self.jpeg_quality

        buf = None
        try:
            from camera_tracker import FrameBuffer
            buf = FrameBuffer.get_instance(self.stream_url, fps_cap=30)
        except Exception as e:
            log.warning(f"[DASH] FrameBuffer not available: {e}")

        min_interval = 1.0 / 25
        last_yield = 0.0

        while not self._stop.is_set():
            now = time.time()

```



```

        if now - last_yield < min_interval:
            time.sleep(0.01)
            continue
        last_yield = now

    ret, frame = (buf.read() if buf else (False, None))

    if not ret or frame is None:
        pld = last_good_frame if last_good_frame else
            offline_frame
        if pld:
            yield (b"--frame\r\nContent-Type: image/jpeg\r\n\r\n"
                   + pld + b"\r\n")
            time.sleep(0.1)
            continue

    # Lightweight OSD
    h, w = frame.shape[:2]
    ts = datetime.now().strftime("%H:%M:%S")
    tgt = self.state.active_target
    fps = self.state.to_dict().get("fps", 0)

    cv2.rectangle(frame, (0, h-24), (w, h), (0, 0, 0), -1)
    cv2.putText(frame,
                f"Counter-UAS System | {ts} | {fps:.0f} FPS",
                (6, h-7), cv2.FONT_HERSHEY_SIMPLEX,
                0.42, (0, 200, 65), 1, cv2.LINE_AA)

    if tgt:
        src = tgt.get("source", "?")
        freq = tgt.get("freq_mhz") or 0
        cv2.putText(frame, f"TARGET: {src} {freq:.0f}MHz",
                    (8, 26), cv2.FONT_HERSHEY_SIMPLEX,
                    0.65, (0, 80, 255), 2, cv2.LINE_AA)

    # Encode
    ok, buf_enc = cv2.imencode(
        ".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY, jpeg_q]
    )
    if not ok:
        continue

    frame_bytes = buf_enc.tobytes()
    last_good_frame = frame_bytes

    yield (b"--frame\r\nContent-Type: image/jpeg\r\n\r\n"
           + frame_bytes + b"\r\n")

def _handle_command(self, cmd: str):

```

```

cmd_map = {
    'arm':          lambda: self.state.update(system_armed=True),
    'disarm':       lambda: self.state.update(system_armed=False,
        jamming_active=False),
    'jam_toggle':   lambda: self.state.update(jamming_active=not
        self.state.jamming_active),
    'sweep':        lambda: self.state.update(active_target=None),
    'zoom_in':      lambda: log.info("[DASH] Zoom in cmd"),
    'zoom_out':     lambda: log.info("[DASH] Zoom out cmd"),
}
fn = cmd_map.get(cmd)
if fn:
    fn()
    log.info(f"[DASH] CMD: {cmd}")

def stop(self):
    self._stop.set()

```

Listing A.6: Event Cooldown Matrix (utils/alert_manager.py)

```

#!/usr/bin/env python3
"""
ALERT MANAGER MODULE
Unified alerting system — single entry point for all alerts.
Channels: Telegram, Dashboard WebSocket, Log file, Sound (optional)

Alert levels:
    INFO — Status updates, module events
    WARNING — Signal threshold exceeded, module restart
    DANGER — Confirmed drone detection, jamming active—
    CRITICAL Temperature/hardware emergency
"""

import time
import logging
import threading
from datetime import datetime
from typing import Optional
from enum import Enum

log = logging.getLogger("ALERTS")

class AlertLevel(Enum):
    INFO = "INFO"
    WARNING = "WARNING"
    DANGER = "DANGER"
    CRITICAL = "CRITICAL"

```

```

# Emoji map for Telegram
EMOJI = {
    AlertLevel.INFO:      "",
    AlertLevel.WARNING:   "",
    AlertLevel.DANGER:    "",
    AlertLevel.CRITICAL:  "",
}

# Cooldown per alert type (seconds) — prevents flooding
COOLDOWN = {
    AlertLevel.INFO:      30,
    AlertLevel.WARNING:   15,
    AlertLevel.DANGER:    5,
    AlertLevel.CRITICAL:  2,
}

class Alert:
    def __init__(self, level: AlertLevel, title: str, body: str = "",
                 source: str = "SYSTEM"):
        self.level = level
        self.title = title
        self.body = body
        self.source = source
        self.ts = datetime.now().isoformat()
        self.sent_channels = []

    def format_telegram(self) -> str:
        emoji = EMOJI.get(self.level, "")
        lines = [f"{emoji} *{self.level.value}* — {self.title}"]
        if self.body:
            lines.append(self.body)
        lines.append(f"_{self.ts[:19]}_")
        return "\n".join(lines)

    def format_log(self) -> str:
        return f"[{self.level.value}] {self.source} | {self.title} | {self.body}"

    def to_dict(self) -> dict:
        return {
            "level": self.level.value,
            "title": self.title,
            "body": self.body,
            "source": self.source,
            "ts": self.ts,
        }

```

```

class AlertManager:
    """
    Central hub for all system alerts.
    Deduplicates, rate-limits, and routes to all channels.
    """

    def __init__(self, cfg: dict, state):
        self.cfg = cfg
        self.state = state
        self._stop = threading.Event()
        self._lock = threading.Lock()
        self._queue: list[Alert] = []
        self._last_sent: dict[str, float] = {} # key → last sent time
        self._history: list[Alert] = [] # last 200 alerts

    def run(self):
        log.info("[ALERTS] Alert manager started")
        while not self._stop.is_set():
            self._flush()
            time.sleep(0.5)

    def _flush(self):
        with self._lock:
            pending = list(self._queue)
            self._queue.clear()

            for alert in pending:
                self._dispatch(alert)

    def _dispatch(self, alert: Alert):
        # Rate-limit check
        key = f"{alert.level.value}:{alert.title}"
        now = time.time()
        cooldown = COOLDOWN.get(alert.level, 10)
        if now - self._last_sent.get(key, 0) < cooldown:
            return
        self._last_sent[key] = now

        # Log
        getattr(log, alert.level.value.lower(),
                log.info)(alert.format_log())

        # Push to state (for Dashboard)
        self.state.push_alert(alert.format_telegram())

        # Store in history
        self._history.append(alert)
        if len(self._history) > 200:

```

```

        self._history.pop(0)

def fire(self, level: AlertLevel, title: str, body: str = "",
        source: str = "SYSTEM") -> Alert:
    """Create and queue an alert. Thread-safe."""
    alert = Alert(level, title, body, source)
    with self._lock:
        self._queue.append(alert)
    return alert

# Convenience methods
def info(self, title: str, body: str = "", source: str = "SYSTEM"):
    return self.fire(AlertLevel.INFO, title, body, source)

def warning(self, title: str, body: str = "", source: str = "SYSTEM"):
    return self.fire(AlertLevel.WARNING, title, body, source)

def danger(self, title: str, body: str = "", source: str = "SYSTEM"):
    return self.fire(AlertLevel.DANGER, title, body, source)

def critical(self, title: str, body: str = "", source: str = "SYSTEM"):
    return self.fire(AlertLevel.CRITICAL, title, body, source)

def drone_detected(self, freq_mhz: float, power_dbm: float,
                   band: str, range_m: Optional[float] = None,
                   source: str = "RF"):
    body_parts = [
        f"Band: {band}",
        f"Freq: {freq_mhz:.1 f} MHz",
        f"Power: {power_dbm:.1 f} dBm",
    ]
    if range_m:
        body_parts.append(f"Range: ~{range_m:.0 f}m")
    return self.fire(
        AlertLevel.DANGER,
        "DRONE DETECTED",
        "\n".join(body_parts),
        source
    )

def jamming_started(self, freq_mhz: float):
    return self.fire(
        AlertLevel.DANGER,
        "JAMMING ACTIVE",
        f"Target freq: {freq_mhz:.1 f} MHz",
        "RF"
    )

```

```

def module_restarted(self, module: str, attempt: int):
    return self.fire(
        AlertLevel.WARNING,
        f"Module {module} restarted",
        f"Attempt {attempt}",
        "WATCHDOG"
    )

def temp_warning(self, temp_c: float):
    level = AlertLevel.CRITICAL if temp_c >= 90 else AlertLevel.
        WARNING
    return self.fire(level, f"CPU Temperature: {temp_c}°C",
        source="HEALTH")

def get_recent(self, n: int = 20) -> list:
    return [a.to_dict() for a in self._history[-n:]]

def stop(self):
    self._stop.set()

```

Listing A.7: Multi-Format Database Logger (utils/data_logger.py)

```

#!/usr/bin/env python3
"""
DATA LOGGER MODULE
Logs all system events to:
    - CSV → detection_log_YYYYMMDD.csv      (human-readable)
    - JSON → events_YYYYMMDD.jsonl          (line-delimited JSON for
        analysis)
    - SQLite → c_uas.db                     (queryable database)

Event types:
DETECTION — RF/optical drone detection
TRACKING — Tracker lock/loss events
JAM_START — Jamming activated
JAM_STOP — Jamming deactivated
HEALTH — Vitals snapshot (every 30s)
ALERT — System alerts
COMMAND — User commands (Telegram/Dashboard)
SYSTEM — Boot/shutdown/module events
"""

import csv
import json
import time
import logging
import threading
import sqlite3

```

```

from pathlib import Path
from datetime import datetime
from typing import Optional

log = logging.getLogger("LOGGER")

class DataLogger:
    def __init__(self, cfg: dict, state):
        self.cfg = cfg
        self.state = state
        self._stop = threading.Event()
        self._lock = threading.Lock()

        log_dir = Path(cfg.get('system', {})).get('log_dir', 'logs')
        log_dir.mkdir(exist_ok=True)

        today = datetime.now().strftime('%Y%m%d')
        self.csv_path = log_dir / f"detections_{today}.csv"
        self.json_path = log_dir / f"events_{today}.jsonl"
        self.db_path = log_dir / "c_uas.db"

        self._init_csv()
        self._init_db()
        self.last_health_log = 0
        self.health_interval = 30 # seconds

    def _init_csv(self):
        """Create CSV with header if new file."""
        if not self.csv_path.exists():
            with open(self.csv_path, 'w', newline='') as f:
                w = csv.writer(f)
                w.writerow([
                    "timestamp", "event_type", "source",
                    "freq_mhz", "power_dbm",
                    "azimuth_deg", "elevation_deg", "range_m",
                    "confidence", "bbox",
                    "cpu_pct", "temp_c", "jamming",
                    "notes"
                ])

    def _init_db(self):
        """Initialize SQLite database."""
        with sqlite3.connect(self.db_path) as conn:
            conn.execute("""
                CREATE TABLE IF NOT EXISTS events (
                    id            INTEGER PRIMARY KEY AUTOINCREMENT,
                    timestamp     TEXT NOT NULL,
                    event_type    TEXT,
                    source         TEXT,

```

```

        freq_mhz    REAL,
        power_dbm   REAL,
        azimuth_deg REAL,
        elevation_deg REAL,
        range_m     REAL,
        confidence  REAL,
        bbox        TEXT,
        cpu_pct     REAL,
        temp_c      REAL,
        jamming     INTEGER,
        notes       TEXT
    )
    """
conn.execute("""
    CREATE INDEX IF NOT EXISTS idx_timestamp
    ON events(timestamp)
    """)
conn.execute("""
    CREATE INDEX IF NOT EXISTS idx_event_type
    ON events(event_type)
    """)
conn.commit()

def run(self):
    log.info("[LOGGER] Data logger started")
    while not self._stop.is_set():
        try:
            # Log health vitals every 30s
            now = time.time()
            if now - self.last_health_log >= self.health_interval:
                vitals = self.state.vitals or {}
                self.log_event("HEALTH", notes=json.dumps(vitals))
                self.last_health_log = now
        except Exception as e:
            log.debug(f"[LOGGER] Periodic log error: {e}")
            time.sleep(5)

def log_detection(self, det: dict):
    """Log an RF or optical detection event."""
    self.log_event(
        "DETECTION",
        source=det.get("source", "?"),
        freq_mhz=det.get("freq_mhz"),
        power_dbm=det.get("power_dbm"),
        azimuth_deg=det.get("azimuth_deg"),
        elevation_deg=det.get("elevation_deg"),
        range_m=det.get("range_m"),
        confidence=det.get("confidence"),
        bbox=str(det.get("bbox")),
    )

```



```

    )

def log_event(self, event_type: str, source: str = None,
              freq_mhz: float = None, power_dbm: float = None,
              azimuth_deg: float = None, elevation_deg: float = None,
              range_m: float = None, confidence: float = None,
              bbox: str = None, notes: str = None):

    ts = datetime.now().isoformat()
    vitals = self.state.vitals or {}
    cpu    = vitals.get("cpu_percent")
    temp   = vitals.get("cpu_temp_c")
    jam    = int(self.state.jamming_active)

    row = {
        "timestamp":    ts,
        "event_type":   event_type,
        "source":       source,
        "freq_mhz":     freq_mhz,
        "power_dbm":    power_dbm,
        "azimuth_deg":  azimuth_deg,
        "elevation_deg": elevation_deg,
        "range_m":      range_m,
        "confidence":   confidence,
        "bbox":         bbox,
        "cpu_pct":      cpu,
        "temp_c":       temp,
        "jamming":      jam,
        "notes":        notes,
    }

    with self._lock:
        self._write_csv(row)
        self._write_json(row)
        self._write_db(row)

def _write_csv(self, row: dict):
    try:
        with open(self.csv_path, 'a', newline='') as f:
            w = csv.DictWriter(f, fieldnames=list(row.keys()))
            w.writerow(row)
    except Exception as e:
        log.debug(f"[LOGGER] CSV write error: {e}")

def _write_json(self, row: dict):
    try:
        with open(self.json_path, 'a') as f:
            f.write(json.dumps(row) + '\n')
    except Exception as e:

```

```

        log.debug(f"[LOGGER] JSON write error: {e}")

def _write_db(self, row: dict):
    try:
        with sqlite3.connect(self.db_path) as conn:
            conn.execute("""
                INSERT INTO events (
                    timestamp, event_type, source,
                    freq_mhz, power_dbm,
                    azimuth_deg, elevation_deg, range_m,
                    confidence, bbox, cpu_pct, temp_c, jamming, notes
                ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
            """, (
                row["timestamp"], row["event_type"], row["source"],
                row["freq_mhz"], row["power_dbm"],
                row["azimuth_deg"], row["elevation_deg"], row["
                    range_m"],
                row["confidence"], row["bbox"],
                row["cpu_pct"], row["temp_c"], row["jamming"], row["
                    notes"]
            ))
            conn.commit()
    except Exception as e:
        log.debug(f"[LOGGER] DB write error: {e}")

def query_detections(self, limit: int = 50) -> list:
    """Query recent detections from DB."""
    try:
        with sqlite3.connect(self.db_path) as conn:
            conn.row_factory = sqlite3.Row
            cur = conn.execute("""
                SELECT * FROM events
                WHERE event_type = 'DETECTION'
                ORDER BY timestamp DESC LIMIT ?
            """, (limit,))
            return [dict(r) for r in cur.fetchall()]
    except Exception as e:
        log.debug(f"[LOGGER] Query error: {e}")
        return []

def get_stats(self) -> dict:
    """Return summary statistics."""
    try:
        with sqlite3.connect(self.db_path) as conn:
            total = conn.execute(
                "SELECT COUNT(*) FROM events WHERE event_type='
                    DETECTION'"
            ).fetchone()[0]
            today = conn.execute(

```

```

        "SELECT COUNT(*) FROM events WHERE event_type='
            DETECTION' "
        "AND timestamp > date('now')")
    ).fetchone()[0]
    jams = conn.execute(
        "SELECT COUNT(*) FROM events WHERE event_type='
            JAM_START'"
    ).fetchone()[0]
    return {"total_detections": total,
            "detections_today": today,
            "total_jams": jams}
except Exception:
    return {}

def stop(self):
    self._stop.set()
    log.info(f"[LOGGER] Logs saved to {self.csv_path.parent}")

```

A.6 System Health and Fault Tolerance

Listing A.8: Thermal and Hardware Monitor (utils/system_health.py)

```

#!/usr/bin/env python3
"""
SYSTEM HEALTH MONITOR — Cross-platform (Pi + Laptop)
Horus University | C-UAS SENTINEL v2.0
"""

import time
import logging
import threading
import platform
from datetime import datetime

log = logging.getLogger("HEALTH")

try:
    import psutil
    PSUTIL_OK = True
except ImportError:
    PSUTIL_OK = False
    log.warning("[HEALTH] psutil not installed — pip install psutil")

# Platform-aware temperature
def get_cpu_temp() -> float:
    """
    Works on: Raspberry Pi, Linux laptop, Windows laptop.
    Returns temperature in °C or 0.0 if unavailable.
    """

```

```

"""
# 1. Try Raspberry Pi (vcgencmd)
try:
    import subprocess
    r = subprocess.run(['vcgencmd', 'measure_temp'],
                        capture_output=True, text=True, timeout=2)
    if r.returncode == 0:
        return float(r.stdout.strip().replace("temp=", "").replace("'C",
            ", "))
except Exception:
    pass

# 2. Try psutil sensors (Linux laptop, works on most distros)
if PSUTIL_OK:
    try:
        temps = psutil.sensors_temperatures()
        if temps:
            # Priority: coretemp (Intel) → k10temp (AMD) → acpitz
            # → any
            for key in ['coretemp', 'k10temp', 'acpitz', 'cpu_thermal']:
                if key in temps:
                    entries = temps[key]
                    if entries:
                        return round(entries[0].current, 1)
            # Fallback: take first available sensor
            for key, entries in temps.items():
                if entries:
                    return round(entries[0].current, 1)
    except Exception:
        pass

# 3. Try Linux thermal zone directly
try:
    import glob
    zones = sorted(glob.glob('/sys/class/thermal/thermal_zone*/temp'))
    for zone in zones:
        with open(zone) as f:
            val = int(f.read().strip())
            if val > 1000: # values in millidegrees
                return round(val / 1000.0, 1)
            elif 20 < val < 120:
                return round(float(val), 1)
except Exception:
    pass

return 0.0

```

```

def get_cpu_freq_mhz() -> float:
    """Current CPU frequency in MHz."""
    if PSUTIL_OK:
        try:
            freq = psutil.cpu_freq()
            if freq: return round(freq.current, 0)
        except Exception: pass
    return 0.0

def get_battery() -> dict:
    """Battery info for laptops."""
    if PSUTIL_OK:
        try:
            b = psutil.sensors_battery()
            if b:
                return {
                    "percent": round(b.percent, 1),
                    "plugged": b.power_plugged,
                    "secs_left": int(b.secsleft) if b.secsleft != psutil.
                        POWER_TIME_UNLIMITED else -1
                }
        except Exception: pass
    return {}

class HealthMonitor:
    def __init__(self, cfg: dict, state):
        self.cfg = cfg.get('health', {})
        self.state = state
        self._stop = threading.Event()
        self.interval = self.cfg.get('check_interval_sec', 5)
        self.cpu_warn = self.cfg.get('cpu_warn_threshold', 80)
        self.ram_warn = self.cfg.get('ram_warn_threshold', 85)
        self.temp_warn = self.cfg.get('temp_warn_celsius', 75)
        self.temp_crit = self.cfg.get('temp_critical_celsius', 90)
        self.disk_warn = self.cfg.get('disk_warn_percent', 85)
        self._platform = platform.system() # 'Linux', 'Windows', 'Darwin'

        log.info(f"[HEALTH] Platform: {self._platform} | "
                f"psutil: {'' if PSUTIL_OK else ''}")

    def run(self):
        log.info("[HEALTH] Monitoring started")
        while not self._stop.is_set():
            try:
                vitals = self._collect()
                self.state.update(vitals=vitals)

```

```

        self._check_alerts(vitals)
    except Exception as e:
        log.error(f"[HEALTH] {e}")
        time.sleep(self.interval)

def _collect(self) -> dict:
    vitals = {"timestamp": datetime.now().isoformat(),
              "platform": self._platform}

    if PSUTIL_OK:
        cpu = psutil.cpu_percent(interval=0.3)
        mem = psutil.virtual_memory()
        disk = psutil.disk_usage('/')
        net = psutil.net_io_counters()

        vitals.update({
            "cpu_percent": round(cpu, 1),
            "cpu_freq_mhz": get_cpu_freq_mhz(),
            "ram_percent": round(mem.percent, 1),
            "ram_used_mb": round(mem.used / 1e6, 1),
            "ram_total_mb": round(mem.total / 1e6, 1),
            "ram_free_mb": round(mem.available / 1e6, 1),
            "disk_percent": round(disk.percent, 1),
            "disk_free_gb": round(disk.free / 1e9, 2),
            "net_sent_mb": round(net.bytes_sent / 1e6, 2),
            "net_recv_mb": round(net.bytes_recv / 1e6, 2),
        })
    else:
        vitals.update({k: 0.0 for k in [
            "cpu_percent", "cpu_freq_mhz", "ram_percent", "ram_used_mb",
            "ram_total_mb", "ram_free_mb", "disk_percent", "disk_free_gb",
            "net_sent_mb", "net_recv_mb"
        ]})

    vitals["cpu_temp_c"] = get_cpu_temp()
    vitals["battery"] = get_battery()
    vitals["modules"] = {
        "rf": self.state.rf_online,
        "camera": self.state.camera_online,
        "motors": self.state.motors_online,
    }
    vitals["jamming"] = self.state.jamming_active
    vitals["armed"] = self.state.system_armed
    return vitals

def _check_alerts(self, v: dict):
    cpu = v.get("cpu_percent", 0)
    ram = v.get("ram_percent", 0)

```

```

temp = v.get("cpu_temp_c", 0)
disk = v.get("disk_percent", 0)
bat  = v.get("battery", {})

if cpu > self.cpu_warn:
    log.warning(f"[HEALTH] CPU {cpu}%")
    self.state.push_alert(f" CPU High: {cpu}%")

if ram > self.ram_warn:
    log.warning(f"[HEALTH] RAM {ram}%")
    self.state.push_alert(f" RAM High: {ram}%")

if temp >= self.temp_crit:
    log.critical(f"[HEALTH] TEMP CRITICAL: {temp}°C")
    self.state.push_alert(f" CRITICAL TEMP: {temp}°C")
elif temp > self.temp_warn:
    log.warning(f"[HEALTH] Temp: {temp}°C")
    self.state.push_alert(f" Temp warning: {temp}°C")

if disk > self.disk_warn:
    log.warning(f"[HEALTH] Disk {disk}%")

# Battery alert for laptop
if bat and not bat.get("plugged") and bat.get("percent", 100) <
20:
    log.warning(f"[HEALTH] Battery low: {bat['percent']}%")
    self.state.push_alert(f" Battery Low: {bat['percent']}%")

def stop(self): self._stop.set()

```

Listing A.9: Configuration Validation (utils/config_validator.py)

```

#!/usr/bin/env python3
"""
CONFIG VALIDATOR
Validates config.yaml before system boot.
Checks:
    - Required fields present
    - IP addresses reachable (ping)
    - Serial port exists
    - Telegram token format
    - GPIO pin conflicts
    - Model file present

Usage:
    python3 config_validator.py           # Validate and print report
    python3 config_validator.py --fix    # Auto-fix common issues
    python3 config_validator.py --quiet  # Only print errors
"""

```

```

import sys
import os
import re
import socket
import subprocess
import argparse
import yaml
import logging
from pathlib import Path
from typing import List, Tuple

log = logging.getLogger("VALIDATOR")

GREEN = "\033[32m"
RED = "\033[31m"
YELLOW = "\033[33m"
CYAN = "\033[36m"
BOLD = "\033[1m"
RST = "\033[0m"

class ValidationResult:
    def __init__(self, field: str, status: str, message: str,
                 fixable: bool = False, fix_val=None):
        self.field = field
        self.status = status # "OK" | "WARN" | "ERROR"
        self.message = message
        self.fixable = fixable
        self.fix_val = fix_val

    def __str__(self):
        icons = {"OK": f"{GREEN}{RST}", "WARN": f"{YELLOW}{RST}", "ERROR": f"{RED}{RST}"}
        fix = f" {CYAN}[auto-fixable]{RST}" if self.fixable else ""
        return f" {icons[self.status]} {self.field:<35} {self.message}{fix}"

def ping(ip: str, timeout: float = 1.0) -> bool:
    """Check if IP is reachable."""
    try:
        result = subprocess.run(
            ["ping", "-c", "1", "-W", str(int(timeout)), ip],
            stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL,
            timeout=timeout + 1
        )
        return result.returncode == 0
    except Exception:

```



```

        return False

def tcp_check(ip: str, port: int, timeout: float = 2.0) -> bool:
    """Check if TCP port is open."""
    try:
        with socket.create_connection((ip, port), timeout=timeout):
            return True
    except Exception:
        return False

class ConfigValidator:
    def __init__(self, cfg_path: str = "config.yaml"):
        self.cfg_path = Path(cfg_path)
        self.cfg = {}
        self.results: List[ValidationResult] = []

    def load(self) -> bool:
        if not self.cfg_path.exists():
            print(f"{RED} Config file not found: {self.cfg_path}{RST}")
            return False
        try:
            with open(self.cfg_path) as f:
                self.cfg = yaml.safe_load(f)
            return True
        except yaml.YAMLError as e:
            print(f"{RED} YAML parse error: {e}{RST}")
            return False

    def validate(self) -> bool:
        """Run all checks. Returns True if no ERRORS."""
        self._check_system()
        self._check_rf()
        self._check_camera()
        self._check_motors()
        self._check_telegram()
        self._check_dashboard()
        self._check_health()
        self._check_files()
        self._check_gpio_conflicts()
        return all(r.status != "ERROR" for r in self.results)

    def _ok(self, field, msg):
        self.results.append(ValidationResult(field, "OK", msg))

    def _warn(self, field, msg, fixable=False, fix_val=None):
        self.results.append(ValidationResult(field, "WARN", msg, fixable,
            fix_val))

```

```

def _err(self, field, msg, fixable=False, fix_val=None):
    self.results.append(ValidationResult(field, "ERROR", msg, fixable,
                                         fix_val))

def _check_system(self):
    s = self.cfg.get("system", {})
    if not s:
        self._err("system", "Section missing")
        return
    self._ok("system.name", s.get("name", "?"))

    fps = s.get("fps_target", 0)
    if fps <= 0:
        self._err("system.fps_target", "Must be > 0", fixable=True,
                  fix_val=120)
    elif fps > 120:
        self._warn("system.fps_target", f"{fps} > 120 — may exceed
Pi 5 capability")
    else:
        self._ok("system.fps_target", str(fps))

    log_dir = Path(s.get("log_dir", "logs"))
    if not log_dir.exists():
        try:
            log_dir.mkdir(parents=True)
            self._ok("system.log_dir", f"Created {log_dir}")
        except Exception as e:
            self._err("system.log_dir", f"Cannot create: {e}")
    else:
        self._ok("system.log_dir", str(log_dir))

def _check_rf(self):
    rf = self.cfg.get("rf", {})
    if not rf:
        self._warn("rf", "Section missing — RF disabled")
        return

    port = rf.get("serial_port", "/dev/ttyUSB0")
    if Path(port).exists():
        self._ok("rf.serial_port", f"{port} ← device found")
    else:
        self._warn("rf.serial_port",
                    f"{port} not found — will use simulation mode")

    bands = rf.get("scan_bands", [])
    if not bands:
        self._err("rf.scan_bands", "No bands defined")
    else:

```

```

        self._ok("rf.scan_bands", f"{len(bands)} bands configured")

    thresh = rf.get("detection_confirm_count", 3)
    if thresh < 2:
        self._warn("rf.detection_confirm_count",
                    "< 2 may cause false positives", fixable=True,
                    fix_val=3)
    else:
        self._ok("rf.detection_confirm_count", str(thresh))

def _check_camera(self):
    cam = self.cfg.get("camera", {})
    if not cam:
        self._warn("camera", "Section missing — camera disabled")
        return

    ip = cam.get("ip", "")
    port = cam.get("port", 80)

    # Validate IP format
    if not re.match(r'^\d{1,3}(\.\d{1,3}){3}$', ip):
        self._err("camera.ip", f"Invalid IP: {ip}")
    else:
        # Check reachability
        if tcp_check(ip, port, timeout=2.0):
            self._ok("camera.ip", f"{ip}:{port} ← reachable")
        else:
            self._warn("camera.ip", f"{ip}:{port} not reachable —
                        check network")

    # Check credentials present
    user = cam.get("username", "")
    pwd = cam.get("password", "")
    if not user or not pwd:
        self._err("camera.credentials", "username or password missing")
    elif pwd in ("Admin1234", "admin123", "password", "12345"):
        self._warn("camera.password", "Default/weak password —
                    change before deployment!")
    else:
        self._ok("camera.credentials", "Set")

    fps = cam.get("fps", 0)
    if fps > 120:
        self._warn("camera.fps", f"{fps} fps may exceed stream
                    capability")
    else:
        self._ok("camera.fps", str(fps))

```

```

stream = cam.get("stream_url", "")
if not stream:
    self._err("camera.stream_url", "stream_url missing")
elif "YOUR" in stream or "CHANGE" in stream:
    self._err("camera.stream_url", "Placeholder value — set real
        stream URL")
else:
    self._ok("camera.stream_url", stream[:50] + "...")

def _check_motors(self):
m = self.cfg.get("motors", {})
if not m:
    self._warn("motors", "Section missing")
    return
if not m.get("enabled", True):
    self._warn("motors.enabled", "Motors disabled")
    return

pan_pins = m.get("pan_gpio_pins", [])
tilt_pins = m.get("tilt_gpio_pins", [])
if len(pan_pins) != 4:
    self._err("motors.pan_gpio_pins", f"Need 4 pins, got {len(
        pan_pins)}")
else:
    self._ok("motors.pan_gpio_pins", str(pan_pins))
if len(tilt_pins) != 4:
    self._err("motors.tilt_gpio_pins", f"Need 4 pins, got {len(
        tilt_pins)}")
else:
    self._ok("motors.tilt_gpio_pins", str(tilt_pins))

def _check_gpio_conflicts(self):
    """Check for pin number conflicts between pan and tilt."""
m = self.cfg.get("motors", {})
pan = set(m.get("pan_gpio_pins", []))
tilt = set(m.get("tilt_gpio_pins", []))
conflict = pan & tilt
if conflict:
    self._err("motors.gpio_conflict",
        f"Pins {conflict} shared between pan and tilt
            motors!")
else:
    self._ok("motors.gpio_conflict", "No pin conflicts")

def _check_telegram(self):
tg = self.cfg.get("telegram", {})
if not tg.get("enabled", True):
    self._warn("telegram.enabled", "Telegram disabled")
    return

```

```

token = os.environ.get("TELEGRAM_BOT_TOKEN") or tg.get("bot_token", "")
if not token or token in ("YOUR_BOT_TOKEN_HERE", ""):
    self._warn("telegram.bot_token",
               "Not set — set TELEGRAM_BOT_TOKEN env var")
elif re.match(r'^\d+:[A-Za-z0-9_-]{35,}$', token):
    self._ok("telegram.bot_token", f"Valid format (***){token[-6:]}")
else:
    self._warn("telegram.bot_token", "Unusual format — verify token")

users = tg.get("allowed_users", [])
if not users:
    self._warn("telegram.allowed_users",
               "Empty — ALL users can control system! Add your user ID.")
else:
    self._ok("telegram.allowed_users", f"{len(users)} user(s) whitelisted")

def _check_dashboard(self):
    d = self.cfg.get("dashboard", {})
    if not d:
        self._warn("dashboard", "Section missing")
        return
    port = d.get("port", 5000)
    if port < 1024 or port > 65535:
        self._err("dashboard.port", f"Invalid port: {port}")
    else:
        self._ok("dashboard.port", str(port))

    key = d.get("secret_key", "")
    if key in ("c_uas_sentinel_2026", "secret", ""):
        self._warn("dashboard.secret_key",
                   "Default secret key — change for production")
    else:
        self._ok("dashboard.secret_key", "Custom key set")

def _check_health(self):
    h = self.cfg.get("health", {})
    if not h:
        self._warn("health", "Section missing — using defaults")
        return
    temp_crit = h.get("temp_critical_celsius", 90)
    if temp_crit > 90:
        self._warn("health.temp_critical_celsius",
                   f"{temp_crit}°C is too high for Pi 5 (max safe ~85

```

```

        °C)",
        fixable=True, fix_val=85)
    else:
        self._ok("health.temp_critical_celsius", f"{temp_crit}°C")

def __check_files(self):
    """Check for expected files and model."""
    required_modules = [
        "main.py", "rf_monitor.py", "camera_tracker.py",
        "motor_controller.py", "dashboard.py", "telegram_bot.py",
        "system_health.py", "yolo_detector.py", "triangulation.py",
        "data_logger.py", "watchdog_recovery.py", "stream_recorder.py",
        "alert_manager.py",
    ]
    base = Path(__file__).parent.parent
    missing = [m for m in required_modules if not (base / m).exists()]
    if missing:
        self._err("modules", f"Missing: {' '.join(missing)}")
    else:
        self._ok("modules", f"All {len(required_modules)} modules
            present")

    model_dir = base / "models"
    model_files = list(model_dir.glob("*.pt")) + list(model_dir.glob(
        "*.onnx"))
    if not model_files:
        self._warn("models/yolov8n.pt",
            "No model file — run: python3 models/
            download_model.py")
    else:
        self._ok("models", f"{len(model_files)} model file(s) found")

def print_report(self, quiet: bool = False):
    errors = [r for r in self.results if r.status == "ERROR"]
    warnings = [r for r in self.results if r.status == "WARN"]
    oks = [r for r in self.results if r.status == "OK"]

    print(f"\n{BOLD}{' '*60}{RST}")
    print(f"  C-UAS SENTINEL — Configuration Validation Report")
    print(f"{' '*60}{RST}")

    if not quiet:
        for r in self.results:
            print(r)

    print(f"\n{' '*60}")
    print(f"  {GREEN} {len(oks)} OK{RST} ")

```

```

        f"{YELLOW} {len(warnings)} WARN{RST}    "
        f"{RED} {len(errors)} ERROR{RST}")

    if errors:
        print(f"\n{RED}{BOLD}System cannot start — fix ERRORS above{
            RST}")
    elif warnings:
        print(f"\n{YELLOW}System can start with warnings — review
            above{RST}")
    else:
        print(f"\n{GREEN}{BOLD} Config valid — ready to start{RST}")
    print()

def apply_fixes(self):
    """Apply auto-fixable corrections to config."""
    changed = False
    for r in self.results:
        if r.fixable and r.fix_val is not None and r.status != "OK":
            # Navigate nested config
            parts = r.field.split(".")
            node = self.cfg
            for p in parts[:-1]:
                node = node.setdefault(p, {})
            old_val = node.get(parts[-1])
            node[parts[-1]] = r.fix_val
            print(f" {CYAN}Fixed{RST} {r.field}: {old_val} → {r.
                fix_val}")
            changed = True
    if changed:
        with open(self.cfg_path, 'w') as f:
            yaml.dump(self.cfg, f, default_flow_style=False,
                sort_keys=False)
        print(f"\n{GREEN}Config saved: {self.cfg_path}{RST}")
    else:
        print(f"{YELLOW}Nothing to auto-fix{RST}")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="C-UAS Config Validator"
        )
    parser.add_argument("--fix", action="store_true", help="Auto-fix
        issues")
    parser.add_argument("--quiet", action="store_true", help="Only print
        errors")
    parser.add_argument("--config", default="config.yaml")
    args = parser.parse_args()

    v = ConfigValidator(args.config)
    if not v.load():

```

```
        sys.exit(1)

    ok = v.validate()
    v.print_report(quiet=args.quiet)

    if args.fix:
        print(f"\n{BOLD}Applying auto-fixes ...{RST}")
        v.apply_fixes()

    sys.exit(0 if ok else 1)
```

A.7 System Configuration and OS Deployment

Listing A.10: *System Configuration Matrix* (config/config.yaml)

```
system:
  name: C-UAS SENTINEL
  version: 2.0.0
  fps_target: 30
  auto_start: true
  fault_tolerance: true
  log_dir: logs
  log_level: INFO
rf:
  device: HTOOL-SA-6G
  serial_port: /dev/ttyUSB0
  baud_rate: 115200
  force_simulation: false
  simulation_silent: true
  scan_bands:
    - name: DRONE_2.4GHz
      freq_start_mhz: 2400
      freq_stop_mhz: 2483
      threshold_dbm: -75
    - name: DRONE_5.8GHz
      freq_start_mhz: 5725
      freq_stop_mhz: 5850
      threshold_dbm: -72
  scan_interval_ms: 200
  detection_confirm_count: 5
  jamming:
    enabled: true
    power_dbm: 20
    duty_cycle: 0.8
    auto_jam_on_detect: true
    jam_duration_sec: 10
camera:
  ip: "192.168.1.64"
```



```
port: 80
username: admin
password: "es6543456"
channel: 102
stream_url: rtsp://admin:es6543456@192.168.1.64:554/Streaming/Channels
           /102
main_stream_url: rtsp://admin:es6543456@192.168.1.64:554/Streaming/
               Channels/101
fps: 30
resolution: 640x480
zoom_optical_max: 25
zoom_default: 1
darkfighter: true
ir_auto: true
frame_skip: 5
jpeg_quality: 70
buffer_size: 2
use_yolo: true
model_path: models/yolov8s.pt
ptz:
  protocol: ISAPI
  pan_speed: 60
  tilt_speed: 60
  zoom_speed: 50
  patrol_enabled: true
  patrol_interval_sec: 30
  patrol_pan_step: 45
tracking:
  algorithm: CSRT
  confidence_threshold: 0.28
  lost_timeout_sec: 8
  reacquire_attempts: 3
  autonomous_on_rf_loss: true
yolo_imgsz: 640
motors:
  enabled: true
  controller: GPIO
  pan_gpio_pins:
    - 17
    - 18
    - 27
    - 22
  tilt_gpio_pins:
    - 23
    - 24
    - 25
    - 8
  pan_step_angle: 1.8
  tilt_step_angle: 1.8
```

```

    pan_range_deg: 360
    tilt_range_deg: 90
    home_position:
        pan: 0
        tilt: 0
    speed_rpm: 60
    auto_sweep_on_idle: true
telegram:
    enabled: true
    bot_token: 8247659793:AAFZWzYg9MPrOO-_L99U9XXbh9GLA7mKDvs
    allowed_users:
        - 5022244123
        - 1124839220
    alert_on_detection: true
    send_snapshot_on_detect: true
    alert_cooldown_sec: 15
    use_inline_keyboard: true
    vitals_on_detection: true
dashboard:
    host: 0.0.0.0
    port: 5000
    pi_ip: "192.168.1.64"
    secret_key: c_uas_sentinel_2026_prod
    fps_display: 30
    dark_theme: true
    military_ui: true
health:
    check_interval_sec: 5
    cpu_warn_threshold: 80
    ram_warn_threshold: 85
    temp_warn_celsius: 70
    temp_critical_celsius: 85
    disk_warn_percent: 85
    watchdog_enabled: true
    watchdog_timeout_sec: 30

```

A.8 Perception and Computer Vision Subsystem

Listing A.11: *Deep Learning Inference Engine (core/yolo_detector.py)*

```

import torch
import numpy as np
from ultralytics import YOLO
import logging

log = logging.getLogger("CAMERA")

class DroneDetector:

```

```

def __init__(self, model_path="models/yolov8n.pt", conf_thresh=0.35):
    #
    self.device = 'cuda' if torch.cuda.is_available() else 'cpu'
    log.info(f"[AI] Loading YOLOv8 on {self.device}...")
    self.model = YOLO(model_path)
    self.conf_thresh = conf_thresh
    self._last_conf = 0.0

def detect(self, frame):
    if frame is None: return None
    try:
        # GPU
        results = self.model.predict(frame, conf=self.conf_thresh,
                                     imgsz=416, device=self.device, verbose=False)

        for r in results:
            if r.bboxes:
                #
                box = r.bboxes[0]
                self._last_conf = float(box.conf[0])
                x, y, w, h = box.xywh[0].cpu().numpy()
                return (int(x-w/2), int(y-h/2), int(w), int(h))
        return None
    except Exception as e:
        log.error(f"[AI] Inference error: {e}")
        return None

```

Listing A.12: Zero-Copy Video Acquisition (core/stream_recorder.py)

```

#!/usr/bin/env python3
"""
STREAM RECORDER MODULE
Records video clips and snapshots automatically.
"""
import cv2
import time
import logging
import threading
import os
from datetime import datetime, timedelta
from pathlib import Path
from typing import Optional
import numpy as np

log = logging.getLogger("RECORDER")
REC_DIR = Path("recordings")
REC_DIR.mkdir(exist_ok=True)

CODEC_PRIORITY = ["mp4v", "XVID", "MJPG"]

```

```

def get_writer(path: str, fps: float, w: int, h: int) -> Optional[cv2.
VideoWriter]:
    for codec in CODEC_PRIORITY:
        fourcc = cv2.VideoWriter_fourcc(*codec)
        ext = ".mp4" if codec == "mp4v" else ".avi"
        actual_path = path.replace(".mp4", ext)
        writer = cv2.VideoWriter(actual_path, fourcc, fps, (w, h))
        if writer.isOpened():
            return writer
        writer.release()
    return None

class StreamRecorder:
    def __init__(self, cfg: dict, state, stream_url: str = ""):
        self.cfg = cfg
        self.state = state
        self.stream_url = stream_url or cfg.get('camera', {}).get('
            stream_url', '')
        self._stop = threading.Event()
        self._writer: Optional[cv2.VideoWriter] = None
        self._recording = False
        self._rec_path = ""
        self._rec_start = 0.0
        self._post_delay = 10.0
        self._last_target = 0.0
        self._frame_w = 1920
        self._frame_h = 1080
        self._fps = 25.0
        self._buffer_sec = 30
        self._buffer: list = []
        self._buffer_lock = threading.Lock()
        self._max_age_days = 7
        self._cleanup_interval = 3600

    def run(self):
        log.info("[REC] Stream recorder started")
        cap = None
        last_cleanup = time.time()

        while not self._stop.is_set():
            if cap is None or not cap.isOpened():
                if self.stream_url:
                    cap = cv2.VideoCapture(self.stream_url, cv2.
                        CAP_FFMPEG)
                    cap.set(cv2.CAP_PROP_BUFFERSIZE, 2)
                    if cap.isOpened():
                        self._frame_w = int(cap.get(cv2.
                            CAP_PROP_FRAME_WIDTH))

```

```

        self._frame_h = int(cap.get(cv2.
            CAP_PROP_FRAME_HEIGHT))
        self._fps      = cap.get(cv2.CAP_PROP_FPS) or 25.0
    else:
        time.sleep(5)
        continue

    ret, frame = cap.read()
    if not ret:
        cap.release(); cap = None
        time.sleep(2)
        continue

    ts = time.time()
    with self._buffer_lock:
        self._buffer.append((ts, frame.copy()))
        cutoff = ts - self._buffer_sec
        self._buffer = [(t, f) for t, f in self._buffer if t >
            cutoff]

    has_target = self.state.active_target is not None
    if has_target:
        self._last_target = ts
        if not self._recording:
            self._start_recording("detection")
    elif self._recording:
        if ts - self._last_target > self._post_delay:
            self._stop_recording()

    if self._recording and self._writer:
        frame_out = self._draw_osd(frame.copy())
        self._writer.write(frame_out)

    self._stop_recording()
    if cap: cap.release()

```

A.9 Kinematic Actuation and Control Theory

Listing A.13: *Sensor Triangulation Mathematics* (utils/triangulation.py)

```

#!/usr/bin/env python3
"""
TRIANGULATION & POSITION ESTIMATOR
Estimates drone position from RF signal strength and Camera PTZ angles.
"""
import math
import logging
import time

```

```

from typing import Optional, Tuple
from collections import deque
import numpy as np

log = logging.getLogger("TRIANGULATION")

FREQ_2G_MHZ = 2442.0
TX_POWER_DBM = 20.0

def rssi_to_range(rssi_dbm: float, freq_mhz: float = FREQ_2G_MHZ,
                  tx_power_dbm: float = TX_POWER_DBM) -> float:
    try:
        path_loss = tx_power_dbm - rssi_dbm
        freq_ghz = freq_mhz / 1000.0
        range_m = 10 ** ((path_loss - 20 * math.log10(freq_ghz) - 92.4) /
                          20)
        return float(np.clip(range_m, 1.0, 500.0))
    except Exception:
        return -1.0

def ptz_to_bearing(pan_deg: float, tilt_deg: float, assembly_pan_deg:
                   float = 0.0) -> Tuple[float, float]:
    azimuth = (assembly_pan_deg + pan_deg) % 360
    elevation = tilt_deg
    return azimuth, elevation

def pixel_to_bearing(cx: int, cy: int, frame_w: int, frame_h: int,
                     fov_h_deg: float = 60.0, fov_v_deg: float = 34.0,
                     camera_pan_deg: float = 0.0, camera_tilt_deg: float
                     = 0.0) -> Tuple[float, float]:
    nx = (cx - frame_w / 2) / frame_w
    ny = (cy - frame_h / 2) / frame_h
    az_offset = nx * fov_h_deg
    el_offset = -ny * fov_v_deg
    return (camera_pan_deg + az_offset) % 360, camera_tilt_deg +
        el_offset

class PositionKalman:
    def __init__(self, process_noise: float = 0.1, meas_noise: float =
                 5.0):
        import cv2
        self.kf = cv2.KalmanFilter(4, 2)
        self.kf.measurementMatrix = np.array([[1, 0, 0, 0], [0, 1, 0, 0]], np.
            float32)
        self.kf.transitionMatrix = np.array
            ([[1, 0, 1, 0], [0, 1, 0, 1], [0, 0, 1, 0], [0, 0, 0, 1]], np.float32)
        self.kf.processNoiseCov = np.eye(4, dtype=np.float32) *
            process_noise

```

```

        self.kf.measurementNoiseCov = np.eye(2, dtype=np.float32) *
            meas_noise

    def update(self, az: float, el: float) -> Tuple[float, float]:
        meas = np.array([[az], [el]], dtype=np.float32)
        self.kf.correct(meas)
        pred = self.kf.predict()
        return float(pred[0]), float(pred[1])

```

A.10 RF Sensing and Network Operations

Listing A.14: *Spectrum Analyzer Interfacing* (core/rf_monitor.py)

```

#!/usr/bin/env python3
"""
RF MONITOR & JAMMER MODULE — C-UAS SENTINEL v2.0
Device: HTOOL-SA 6G Spectrum Analyzer (35 MHz – 6200 MHz)
"""
import time
import logging
import threading
import struct
import numpy as np
from datetime import datetime
from typing import Optional, List, Dict
import serial

CMD_START_SCAN = b'\x55\xAA\x01\x00\x00\x00\xFF'
CMD_START_JAM = b'\x55\xAA\x10'
CMD_STOP_JAM = b'\x55\xAA\x11\x00\x00\x00\xFF'
CMD_QUERY_POWER = b'\x55\xAA\x20\x00\x00\x00\xFF'

class SpectrumAnalyzer:
    def __init__(self, port: str, baud: int, silent_sim: bool = True):
        self.port = port
        self.baud = baud
        self.connected = False
        self.simulation_mode = False
        self.silent_sim = silent_sim

    def connect(self) -> bool:
        try:
            self.ser = serial.Serial(self.port, self.baud, timeout=1.0)
            self.connected = True
            return True
        except Exception as e:
            self.simulation_mode = True
            return False

```

```

def scan_band(self, start_mhz: float, stop_mhz: float, steps: int =
20) -> np.ndarray:
    freqs = np.linspace(start_mhz, stop_mhz, steps)
    powers = np.array([self.read_power_dbm(f) for f in freqs])
    return np.column_stack([freqs, powers])

def start_jamming(self, freq_mhz: float, bandwidth_mhz: float = 20.0,
power_dbm: float = 20.0) -> bool:
    if self.simulation_mode: return True
    try:
        start = int((freq_mhz - bandwidth_mhz / 2) * 1000)
        stop = int((freq_mhz + bandwidth_mhz / 2) * 1000)
        pwr = int(power_dbm * 10)
        self.ser.write(CMD_START_JAM + struct.pack('<IIH', start,
            stop, pwr) + b'\xFF')
        return True
    except Exception:
        return False

```

A.11 System Health and Fault Tolerance

Listing A.15: Thread Watchdog Recovery (utils/watchdog_recovery.py)

```

#!/usr/bin/env python3
"""
WATCHDOG & AUTO-RECOVERY MODULE
Monitors system threads and restarts dead modules automatically.
"""
import time
import threading
import os
import signal

class Watchdog:
    def __init__(self, cfg: dict, state):
        self.state = state
        self._stop = threading.Event()
        self._modules = {}
        self.temp_critical = cfg.get('health', {}).get('
            temp_critical_celsius', 90)

    def _check_all(self):
        vitals = self.state.vitals or {}
        temp = vitals.get("cpu_temp_c", 0)

        if temp >= self.temp_critical:
            os.kill(os.getpid(), signal.SIGTERM)

```



```

        return

    for name, entry in self._modules.items():
        if entry.thread and not entry.thread.is_alive():
            new_thread = threading.Thread(target=entry.restart_fn,
                                           daemon=True)
            new_thread.start()
            entry.thread = new_thread

```

A.12 System Configuration and OS Deployment

A.13 Command & Control (C2) and Data Logging

Listing A.16: *Command Line Interface* (core/cli_interface.py)

```
#!/usr/bin/env python3
"""
CLI INTERFACE MODULE
Local terminal control panel for C-UAS Sentinel.
"""
import threading
import logging
from datetime import timedelta

log = logging.getLogger("CLI")

class CLIInterface:
    def __init__(self, cfg: dict, state, data_logger=None):
        self.cfg = cfg
        self.state = state
        self._stop = threading.Event()

    def run(self):
        print("C-UAS SENTINEL - LOCAL CLI v2.0")
        while not self._stop.is_set():
            try:
                raw = input("SENTINEL> ").strip()
                if raw: self._dispatch(raw.lower().split())
            except (EOFError, KeyboardInterrupt):
                break

    def _dispatch(self, parts: list):
        cmd = parts[0]
        handlers = {
            "status": self._cmd_status,
            "arm": lambda _: self.state.update(system_armed=True),
            "disarm": lambda _: self.state.update(system_armed=False, jamming_active=False),
```

```

        "jam": self._cmd_jam,
        "quit": self._cmd_quit,
    }
    if cmd in handlers:
        handlers[cmd](parts[1:])
    else:
        print("Unknown command. Type 'help'.")

def _cmd_status(self, _):
    d = self.state.to_dict()
    print(f"Uptime: {timedelta(seconds=d['uptime_sec'])}")
    print(f"Armed: {d['system_armed']} | Jamming: {d['jamming_active']}")

def _cmd_jam(self, args):
    if args and args[0] == "on":
        self.state.update(jamming_active=True)
        print("JAMMING ACTIVATED")
    elif args and args[0] == "off":
        self.state.update(jamming_active=False)
        print("Jamming stopped")

def _cmd_quit(self, _):
    self._stop.set()

```

Listing A.17: Remote Telegram Alerting (core/telegram_bot.py)

```

#!/usr/bin/env python3
"""
TELEGRAM BOT — C-UAS SENTINEL v2.0
Manages encrypted bi-directional communications for remote C2.
"""
import time
import logging
import threading
import requests

log = logging.getLogger("TELEGRAM")

class TelegramBot:
    def __init__(self, cfg: dict, state):
        self.state = state
        self._stop = threading.Event()
        self.token = cfg.get("telegram", {}).get("bot_token", "")
        self.api = f"https://api.telegram.org/bot{self.token}"
        self.allowed_users = cfg.get("telegram", {}).get("allowed_users", [])
        self.enabled = cfg.get("telegram", {}).get("enabled", True)
        self.alert_cooldown = 15

```

```

def run(self):
    if not self.enabled: return
    log.info("[TELEGRAM] Starting...")

    while not self._stop.is_set():
        self._flush_alerts()
        time.sleep(0.8)

def _flush_alerts(self):
    alerts = self.state.pop_alerts()
    for alert in alerts:
        msg = alert.get("message", "")
        if "DRONE DETECTED" in msg:
            self._send_threat_card()
        else:
            self._send_to_all(f" {msg}")

def _send_to_all(self, text: str):
    for uid in self.allowed_users:
        requests.post(f"{self.api}/sendMessage", json={"chat_id": uid
            , "text": text})

def stop(self):
    self._stop.set()

```

A.14 System Configuration and OS Deployment

Listing A.18: *Systemd Daemonization Service (c_uas.service)*

```

[Unit]
Description=C-UAS Sentinel - Counter-UAS Detection & Neutralization
System
Documentation=https://github.com/islam-8/c-uas-sentinel
After=network.target network-online.target
Wants=network-online.target

[Service]
Type=simple
User=pi
Group=pi
WorkingDirectory=/home/pi/c_uas

# Environment Variables
Environment="PATH=/home/pi/c_uas/venv/bin:/usr/local/bin:/usr/bin:/bin"

ExecStart=/home/pi/c_uas/venv/bin/python3 /home/pi/c_uas/main.py

```

```
# Fault tolerance: restart on any failure
Restart=always
RestartSec=5
StartLimitInterval=60
StartLimitBurst=5

# Logging
StandardOutput=journal
StandardError=journal
SyslogIdentifier=c_uas

# Graceful stop
KillMode=process
TimeoutStopSec=15
KillSignal=SIGTERM

[Install]
WantedBy=multi-user.target
```

Listing A.19: *Python Dependencies* (requirements.txt)

```
attrs==23.2.0
boto3==1.34.46
certifi==2023.11.17
cryptography==41.0.7
numpy==1.26.4
pyserial==3.5
PyYAML==6.0.1
requests==2.31.0
systemd-python==235
# Note: Machine Learning (Ultralytics) and CV2 libraries are globally
# installed on the edge node to leverage hardware acceleration.
```

References

- [1] Skolnik MI. Introduction to Radar Systems. 3rd ed. McGraw-Hill Education; 2001.
- [2] Richards MA. Fundamentals of Radar Signal Processing. 2nd ed. McGraw-Hill; 2014.
- [3] Barton DK. Radar System Analysis and Modeling. Artech House; 2004.
- [4] Mahafza BR. Radar Systems Analysis and Design Using MATLAB. CRC Press; 2013.
- [5] Stimson GW. Introduction to Airborne Radar. SciTech Publishing; 1998.
- [6] Peebles PZ. Radar Principles. John Wiley & Sons; 1998.
- [7] Knott EF, Shaeffer JF, Tuley MT. Radar Cross Section. 2nd ed. SciTech Publishing; 2004.
- [8] Skolnik MI. Radar Handbook. 3rd ed. McGraw-Hill; 2008.
- [9] Melvin WL, Scheer JA. Principles of Modern Radar. SciTech Publishing; 2012.
- [10] Billingsley JB. Low-Angle Radar Land Clutter. William Andrew; 2002.
- [11] Curry GR. Radar System Performance Modeling. Artech House; 2005.
- [12] Billingsley JB. Radar ground clutter. IEEE AES Systems Magazine. 2002.
- [13] Ward K. Sea Clutter: Scattering, the K Distribution and Radar Performance. IET; 2006.
- [14] Chen VC, Li F, Ho SS, Wechsler H. Micro-Doppler effect in radar: phenomenon, model, and simulation study. IEEE Transactions on Aerospace and Electronic Systems. 2006;42(1):2-21.
- [15] Tahmouh D. Review of micro-doppler signatures of drones. In: IEEE Radar Conference; 2015.
- [16] Perez J, et al. Micro-Doppler analysis of multi-rotor UAVs. IET Radar, Sonar & Navigation. 2018.
- [17] Gao X, et al. Deep learning for micro-doppler signature classification. IEEE Geoscience and Remote Sensing Letters. 2020;17(3):421-425.
- [18] Fioranelli F, et al. Micro-Doppler Radar and its Applications. IET; 2020.
- [19] Smith P. Doppler processing for drone detection. Journal of Electronic Defense. 2021.
- [20] Chang S, et al. Acoustic detection of UAVs. Applied Acoustics. 2019.
- [21] Zhang M, et al. Drone detection using microphone arrays. IEEE Sensors. 2020.
- [22] Lee H, et al. Urban drone detection using acoustics. Journal of Sound and Vibration. 2021.
- [23] Cronin AP. Asymmetric warfare and the rise of the drone. Strategic Studies Quarterly. 2020;14(1):112-143.
- [24] Bunker RJ. Terrorist and Insurgent Unmanned Aerial Vehicles. Strategic Studies Institute; 2015.
- [25] Michel AH. Counter-Drone Systems. Center for the Study of the Drone at Bard College; 2019.
- [26] Rathmell A. The swarm threat to aviation. Security Journal. 2021.
- [27] Sullivan J. Defending against drone swarms. Military Technology. 2022.
- [28] Zhang Y, et al. A review of Counter-UAS technologies. IEEE Aerospace and Electronic Systems Magazine. 2023;38(2):24-38.

-
- [29] NATO Standardization Office. UAS Classification Guide. Brussels: North Atlantic Treaty Organization; 2023.
- [30] U.S. Department of Defense. Counter-Small Unmanned Aircraft Systems Strategy. Washington (DC): DoD; 2021.
- [31] Federal Aviation Administration. Unmanned Aircraft Systems (UAS) Integration. 2022.
- [32] European Union Aviation Safety Agency (EASA). Civil Drones (Unmanned aircraft). 2023.
- [33] RAND Corporation. The Cost of Counter-Drone Systems. 2020.
- [34] MITRE. Counter-UAS Technology Guide. 2021.
- [35] Al-Qaness MA, et al. RF-based drone detection and identification using deep learning approaches. *IEEE Access*. 2024;12:12345-12367.
- [36] Nguyen P, et al. RF drone detection using machine learning. *IEEE Sensors Journal*. 2021.
- [37] Wang H, et al. Passive RF sensing for UAVs. *IEEE Communications Surveys*. 2022.
- [38] Li X, et al. Specific emitter identification for drones. *IEEE Transactions on Information Forensics and Security*. 2020.
- [39] Kim J, et al. Drone identification based on RF fingerprinting. *IEEE Access*. 2021.
- [40] Zhao L, et al. Spectrum sensing for drone communications. *IEEE Wireless Communications*. 2022.
- [41] Humphreys TE, Ledvina BM, Psiaki ML, O'Hanlon BW, Kintner PM. Assessing the spoofing threat. *GPS World*. 2008;19(9):28-38.
- [42] Schmidt RO. Multiple emitter location and signal parameter estimation. *IEEE Transactions on Antennas and Propagation*. 1986;34(3):276-280.
- [43] Roy R, Kailath T. ESPRIT-estimation of signal parameters via rotational invariance techniques. *IEEE Transactions on Acoustics, Speech, and Signal Processing*. 1989.
- [44] Krim H, Viberg M. Two decades of array signal processing research. *IEEE Signal Processing Magazine*. 1996.
- [45] Stoica P, Moses R. *Spectral Analysis of Signals*. Prentice Hall; 2005.
- [46] Van Trees HL. *Optimum Array Processing*. Wiley; 2002.
- [47] Godara LC. Application of antenna arrays to mobile communications, part II. *Proceedings of the IEEE*. 1997.
- [48] Hecht E. *Optics*. 5th ed. Pearson; 2016.
- [49] Holst GC. *Electro-Optical Imaging System Performance*. 6th ed. JCD Publishing; 2017.
- [50] Redmon J, Divvala S, Girshick R, Farhadi A. You only look once: unified, real-time object detection. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*; 2016. p. 779-788.
- [51] Bochkovskiy A, Wang C, Liao HM. YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*. 2020.
- [52] Wang C, et al. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; 2023.
- [53] Jocher G, Chaurasia A, Qiu J. Ultralytics YOLOv8. GitHub repository; 2023. Available from: <https://github.com/ultralytics/ultralytics>.

-
- [54] Zheng Z, Wang P, Liu W, Li J, Ye R, Ren D. Distance-IoU loss: faster and better learning for bounding box regression. In: Proceedings of the AAAI Conference on Artificial Intelligence; 2020. p. 12993-13000.
 - [55] Ren S, He K, Girshick R, Sun J. Faster R-CNN. *Advances in Neural Information Processing Systems*. 2015.
 - [56] He K, Gkioxari G, Dollár P, Girshick R. Mask R-CNN. In: ICCV; 2017.
 - [57] Lin T, et al. Feature pyramid networks for object detection. In: CVPR; 2017.
 - [58] Jacob B, et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: CVPR; 2018. p. 2704-2713.
 - [59] Courbariaux M, et al. Binarized neural networks. *Advances in Neural Information Processing Systems*. 2016.
 - [60] Han S, et al. Deep compression. In: ICLR; 2016.
 - [61] Wu J, et al. Quantized convolutional neural networks. In: CVPR; 2016.
 - [62] Zhang D, et al. LQ-Nets: Learned quantization. In: ECCV; 2018.
 - [63] Jacob B, et al. INT8 quantization for edge AI. *IEEE Edge Computing*. 2020.
 - [64] Kalman RE. A new approach to linear filtering and prediction problems. *Transactions of the ASME—Journal of Basic Engineering*. 1960;82(Series D):35-45.
 - [65] Simon D. *Optimal State Estimation: Kalman, H Infinity, and Nonlinear Approaches*. John Wiley & Sons; 2006.
 - [66] Welch G, Bishop G. *An introduction to the Kalman filter*. University of North Carolina at Chapel Hill; 1995.
 - [67] Julier SJ, Uhlmann JK. Unscented filtering and nonlinear estimation. *Proceedings of the IEEE*. 2004;92(3):401-422.
 - [68] Bar-Shalom Y, Li XR, Kirubarajan T. *Estimation with Applications to Tracking and Navigation*. Wiley; 2001.
 - [69] Blackman S, Popoli R. *Design and Analysis of Modern Tracking Systems*. Artech House; 1999.
 - [70] Åström KJ, Hägglund T. *PID Controllers: Theory, Design, and Tuning*. Instrument Society of America; 1995.
 - [71] Ogata K. *Modern Control Engineering*. 5th ed. Prentice Hall; 2010.
 - [72] Dorf RC, Bishop RH. *Modern Control Systems*. Pearson; 2011.
 - [73] Kuo BC. *Automatic Control Systems*. Wiley; 2002.
 - [74] Poisel RA. *Modern Communications Jamming Principles and Techniques*. 2nd ed. Artech House; 2011.
 - [75] Adamy D. *EW 101: A First Course in Electronic Warfare*. Artech House; 2001.
 - [76] Pace PE. *Detecting and Classifying Low Probability of Intercept Radar*. Artech House; 2009.
 - [77] Neri F. *Introduction to Electronic Defense Systems*. Artech House; 2001.
 - [78] Schleher DC. *Electronic Warfare in the Information Age*. Artech House; 1999.
 - [79] Mishra UK, Shen L, Kazior TE, Wu YF. GaN-based RF power devices and amplifiers. *Proceedings of the IEEE*. 2008;96(2):287-305.
 - [80] Pearton SJ, et al. GaN: Processing, defects, and devices. *Journal of Applied Physics*. 1999.

-
- [81] Aivazian G, et al. GaN power amplifiers for radar. *IEEE Microwave Magazine*. 2015.
 - [82] Shen L, et al. Advanced GaN HEMTs. *IEEE Electron Device Letters*. 2001.
 - [83] Wu YF, et al. High-efficiency GaN power amplifiers. *IEEE Transactions on Microwave Theory and Techniques*. 2005.
 - [84] Raspberry Pi Ltd. Raspberry Pi 5 Product Brief. Cambridge (UK); 2023. Available from: <https://datasheets.raspberrypi.com>
 - [85] Plett GL. *Battery Management Systems, Volume 1: Battery Modeling*. Artech House; 2015.
 - [86] Linden D, Reddy TB. *Handbook of Batteries*. 3rd ed. McGraw-Hill; 2001.
 - [87] Saad W, Bennis M, Chen M. A vision of 6G wireless systems: applications, trends, technologies, and open research problems. *IEEE Network*. 2020;34(3):134-142.
 - [88] Dang S, et al. What should 6G be? *Nature Electronics*. 2020.
 - [89] Letaief KB, et al. The roadmap to 6G: AI empowered wireless networks. *IEEE Communications Magazine*. 2019.

ملخص المشروع

تُعد الطائرات المسيّرة (UAS) من الفئة الأولى تهديداً غير متماثل للبنى التحتية الحيوية، حيث تعجز أنظمة الرادار التقليدية عن رصدها نتيجة لصغر مقطعها الراداري (RCS) وغمر إشارتها ضمن التداخلات الأرضية؛ وهو ما يُعرف فيزيائياً بـ "فجوة الدقة" (Granularity Gap).

تُقدم هذه الأطروحة تصميماً هندسياً وبناءً عملياً لمنظومة سيبرانية-فيزيائية (CPS) حوسبية طرفية (Edge-Native)، تهدف إلى كشف وتبّيع وتحييد الطائرات المسيّرة في الوقت الفعلي. تتخلى البنية المقترحة عن الاعتماد الأحادي على الرادار لصالح دمج مستشعرات متعددة الأنماط (Sensor Fusion). تعتمد مرحلة الاستشعار المبكر على محلل طيفي (HTool-SA-6G) مقترن بهوائي غير موجه لاكتشاف التشوهات الترددية لبروتوكولات التوجيه في نطاقات 4.2 و 8.5 جيجاهرتز. ولتأكيد الهدف، تعتمد طبقة الإدراك البصري على شبكة عصبية تلافيفية من نوع (YOLOv8) تم تكميمها لتعمل بدقة (INT8) على عقدة معالجة (Raspberry Pi 5) لتحليل التدفقات المرئية الواردة من كاميرا بصرية ذات تكبير نشط (PTZ).

ولمعالجة الحركة غير الخطية للأهداف وتعويض زمن التأخير الميكانيكي، تم اشتقاق وتطبيق مرشح كالمان المتقطع (DKF) لتنعيم المسار وتقديم تنبؤات حركية دقيقة، والتي تُمرر لاحقاً إلى متحكم تناسبي تكاملي تفاضلي (PID) لتوجيه الحركات عبر بروتوكولات الشبكة المعزولة لتفادي عنق الزجاجة في المعالجة. أثبتت الاختبارات والتحليلات نجاح النظام في تحقيق زمن استجابة كلي يبلغ 139 ملي ثانية، مع دقة اكتشاف (mAP@50) بلغت 88.8%.

أخيراً، يضع المشروع إطاراً نظرياً لمرحلة التحييد الفعال عبر دمج هوائي موجه مقترن بمضخم طاقة من نيتريد الجاليوم (GaN) لتنفيذ تشويش راديوي مركّز، بالإضافة إلى خارطة طريق لتطوير النظام مستقبلاً باستخدام مصفوفات الهوائيات الذكية (Massive MIMO) لضمان حماية البنية التحتية بتكلفة اقتصادية مستدامة وقابلة للتطوير.

الكلمات المفتاحية:

الأنظمة المضادة للطائرات المسيّرة (C-UAS)، ذكاء الحافة (Edge AI)، مرشح كالمان (Kalman Filter)، الرؤية الحاسوبية، Raspberry Pi 5، التشويش الراديوي.

جامعة حورس - مصر
كلية الهندسة
قسم هندسة الاتصالات والإلكترونيات

نظام متقدم لكشف، وتتبع، وتحديد الطائرات
بدون طيار

مقدم من الطلاب:

إسلام عماد أحمد إسماعيل	1211108
عبد العزيز التهامي محمد	1211105
رفيق مرزوق الفيومي	1233007
روان محمد البيالي	1211174
ساجي شوقي أبو النجا	1221602
إسلام هشام حامد	1201040

تحت إشراف:

أ.د. عمرو حسين حسين عبدالله
أ.م.د. ميرفت محمد حسن الصديق
د. رانيا حمدي العبد

العام الجامعي 2025/2026